Simula Training Data Framework Specification

SAP Controls Architecture Team

April 2026

Frontmatter

Executive Summary

This specification defines the **Simula Training Data Framework**a comprehensive system for generating high-quality synthetic training data to enhance SAP AI agents' reasoning capabilities. The framework transforms production schemas and business logic into structured training examples suitable for supervised fine-tuning (SFT) and reinforcement learning (GRPO).

Primary Objectives

1. **Schema-Driven Generation**: Extract taxonomy from HANA Cloud schemas via MCP
2. **Multi-Turn Reasoning**: Generate progressive reasoning chains with verification
3. **Quality Assurance**: Implement double-critic filtering and diversity scoring
4. **SAP Integration**: Native compatibility with AI Core, BTP, and GRC platforms

Key Deliverables

- **TrainingExample Schema**: Canonical data model for all generated examples
- **Taxonomy Builder**: Algorithm for extracting domain concepts from HANA schemas
- **Generation Engine**: Multi-turn reasoning generator with critic validation
- **Evaluation Framework**: Elo-based quality scoring and coverage metrics
- **CLI Tools**: Reproducible pipeline for dataset generation

Document Conventions

Requirement Classification

All requirements use MoSCoW prioritization:

Priority	Symbol	Definition
MUST	❗	Non-negotiable for v1.0 release
SHOULD	✅	Important but can slip to v1.1
COULD	❓	Nice to have if time permits
WON'T	❌	Explicitly deferred (this phase)

Table 1: MoSCoW Priority Definitions

Colored Boxes

This specification uses colored boxes to highlight different types of content:

Source

"Source quotations from research papers or reference documents appear in green boxes with quotation marks."

Requirement: D

erived requirements appear in blue boxes and include formal requirement IDs.

Implementation Note

Implementation notes provide technical guidance for developers in purple boxes.

Warning

Warnings highlight critical constraints or potential pitfalls in red boxes.

Future Work

Future enhancements (v1.3+) are marked in gold boxes to distinguish them from current scope.

Requirement ID Conventions

How to Read This Specification

This specification is organized into five parts plus appendices:

1. **Part I: Introduction and Context** – Establishes the problem domain, business requirements, and stakeholder needs

Prefix	Category	Example
SIM-DATA	Data Schema	SIM-DATA-001: TrainingExample format
SIM-TAX	Taxonomy Engine	SIM-TAX-001: Concept extraction
SIM-GEN	Generation Engine	SIM-GEN-001: Multi-turn generation
SIM-EVAL	Evaluation	SIM-EVAL-001: Elo scoring
SIM-CLI	CLI/Pipeline	SIM-CLI-001: Build command
SIM-INT	Integration	SIM-INT-001: HANA connectivity
SIM-HANA	HANA-specific	SIM-HANA-001: Schema extraction

Table 2: Requirement ID Prefixes

2. **Part II: Architecture and Core Design** – Defines data schemas, extraction pipelines, and generation algorithms

3. **Part III: Quality Assurance and Evaluation** – Covers evaluation metrics, testing, and worked examples

4. **Part IV: Implementation and Governance** – Provides implementation guidance and cross-specification dependencies

5. **Part V: Planning and Future Evolution** – Presents roadmaps and future enhancement plans

6. **Appendices** – Contains the implementation gap review, SAP integration guide, and references

Reading Guide

Chapter Relationship Flowchart

The following flowchart shows how the planning and future directions chapters (15–17) relate to other parts of the specification. This centralized view reduces the need to repeat integration context within each chapter.

Key Reading Paths:

- **Understanding v1.2 scope**: Ch.2 → Ch.15 → App.A1 → Ch.16

- **Gap remediation**: App.A1 (P0 gaps) → Ch.15 (milestones blocked) → Ch.16 (enterprise phases)

- **Future planning**: Ch.16 (Phase 3 complete) → Ch.17 (v1.3+ enhancements)

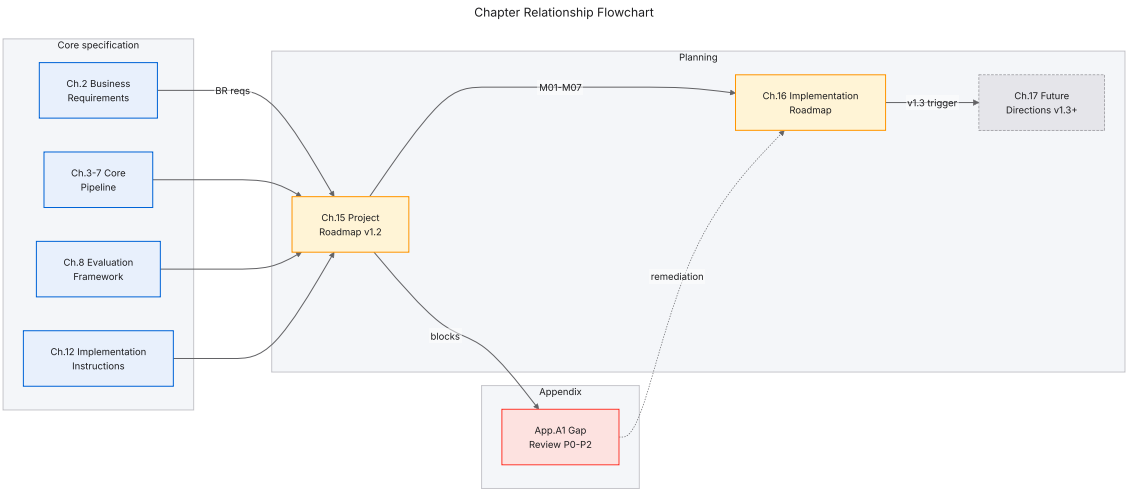

Figure 1: Chapter relationship flowchart. Blue: core specification. Gold: planning chapters (v1.2 scope). Dotted gray: future scope (v1.3+). Red: gap review. Solid arrows: hard prerequisites. Dashed arrows: remediation dependencies.

Audience-Specific Paths

For Project Managers / Business Owners

1. Frontmatter (this section)
2. Chapter 1: Overview and Scope
3. Chapter 2: Business Requirements
4. Chapter 15: Project Roadmap
5. Chapter 16: Implementation Roadmap

For Implementers / Developers

1. Chapter 1: Overview (full)
2. Chapter 3: Canonical Data Schema
3. Chapters 4–7: Core Pipeline (Extraction, Taxonomy, Generation, CLI)
4. Chapter 12: Implementation Instructions
5. Chapter 10: Testing Specification
6. Appendix B: SAP Integration Guide

For Quality / Evaluation Engineers

1. Chapter 8: Evaluation Framework
2. Chapter 9: Worked Example
3. Chapter 11: Test Fixtures
4. Chapter 10: Testing Specification
5. Appendix A: Implementation Gap Review

For Auditors / Compliance Officers

1. Chapter 1: Overview (Sections 1.3–1.5 for research traceability)
2. Chapter 2: Business Requirements
3. Chapter 14: Schema Versioning
4. Chapter 13: Cross-Spec Dependencies
5. Appendix C: References

Version History

Version	Date	Changes
1.0.0	2026-04-01	Initial specification based on Simula research paper
1.1.0	2026-04-10	Added evaluation framework and CLI specification
1.2.0	2026-04-21	Added SAP integration guide, business requirements, and implementation roadmap
1.2.1	2026-04-21	Reorganized chapter structure for improved readability; moved business requirements early; added Part divisions; moved gap review to appendix
1.2.2	2026-04-21	Added Chapter Relationship Flowchart in Frontmatter (Figure 1); added Gap-to-Milestone mapping table in Appendix A1 (Table A.2)

Document Scope Matrix

Topic	In Scope	Chapter(s)	Notes
TrainingExample schema	✔	3	Canonical data model
Taxonomy extraction	✔	4, 5	Algorithm 1
Multi-turn generation	✔	6	Algorithm 2
Double-critic filtering	✔	6	Quality gate
Elo-based evaluation	✔	8	Quality metrics
SFT training	✔	3	Output format
GRPO training	✔	3	Reward format
HANA Cloud integration	✔	4, App. B	MCP-based extraction
Multi-modal input	✘	17	Future (v1.3+)
Real-time streaming	✘	–	Out of scope
Model training execution	✘	–	Downstream concern

Table 4: Document Scope Matrix

Chapter 1

Overview and Scope

1.1 The Problem

The SAP OSS training pipeline requires high-quality Text-to-SQL training data that reflects the actual schema structure of SAP HANA Cloud tables. The current implementation in `src/training/pipeline/` uses metadata from **Excel files** (`DATA_DICTIONARY.xlsx`, `ESG_DATA_DICTIONARY.xlsx`) that were originally exported from HANA or defined manually.

While the `HanaClient` in `src/training/pipeline/hana_client.py` has the capability to query `SYS.TABLE_COLUMNS` for live schema metadata, this HANA-direct extraction is **not currently integrated** into the main training data generation pipeline.

Table 1.1: Current vs. Target Training Data Pipeline

Dimension	**Current State**	**Target State**
Schema Source	Static Excel/CSV files	Live HANA Cloud metadata
Extraction Method	Manual export	MCP-based automated query
Training Data	Template-based expansion	Reasoning-driven synthesis
Diversity Control	None	Taxonomic coverage
Quality Assurance	None	Double-critic filtering
Complexity Control	None	Calibrated Elo scoring

1.2 Goals

The specification targets three outcomes:

1. **HANA-Direct Extraction.** Every training dataset is generated from live schema metadata queried via the AI Core PAL MCP service.
2. **Taxonomic Coverage.** The conceptual space of the target domain is mapped using hierarchical taxonomies, ensuring global diversity.
3. **Reasoning-Driven Synthesis.** Each training example is generated from first principles using agentic refinement, ensuring quality, local diversity, and controlled complexity.

1.3 In-Scope Domains

Table 1.2: Application Domains

Domain	HANA Schema	Training Task
Arabic Document Processing	`AP_INVOICES`, `VAT_CHECKLISTS`	Invoice field extraction, VAT compliance
Trial Balance Review	`TB_VARIANCES`, `GL_BALANCES`	Variance analysis, commentary generation
ESG Analytics	`ESG_METRICS`, `FINANCED_EMISSIONS`	Scope classification, net-zero alignment

1.4 System Context

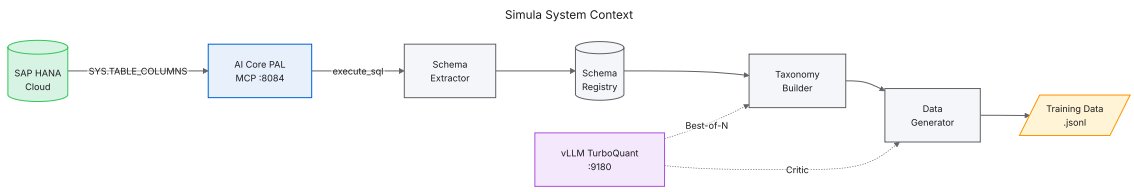

Figure 1.1: System context. HANA Cloud (green) provides live schema metadata via the MCP protocol (blue). The vLLM backend (purple) powers taxonomy expansion and data generation. Output is JSONL training data (gold).

1.5 Research Paper Traceability

This specification implements the Simula framework from Davidson et al. (2026), "Reasoning-Driven Synthetic Data Generation and Evaluation," published in Transactions on Machine Learning Research (TMLR). The following table maps paper sections to specification sections:

Table 1.3: Traceability: Research Paper to Implementation

Paper Reference	Paper Finding	Spec Implementation
Section 2.1, Figure 1	Taxonomy-based coverage	ğ5, Algorithm 1
Section 2.2, Algorithm 2	Agentic data synthesis	ğ6, Algorithm 2
Section 2.3, Algorithm 3	Calibrated complexity scoring	ğ8.1, Elo algorithm
Section 3.1, Figure 3	Double-critic validation	ğ6.5, ğ8.4
Section 3.3, Figure 4	Additive diversity	ğ8.3, Diversity analyzer
Section 4.3	83% saturation ratio	ğ8.5, Gap analysis
Figure 7	Complexity vs teacher strength	ğ8.6.1, Adaptive ratio
Table 2 (Appendix B)	Taxonomy quality metrics	ğ8.6.5, Quality gate
Appendix E.3	Elo algorithm details	ğ8.1.1, `_update_elo()`

1.6 Out of Scope

The following are **explicitly** out of scope for this document:

- Model fine-tuning and evaluation (covered by separate LoRA training specification).
- Downstream inference and routing (covered by `03-prompt-routing.tex`).
- Trial Balance business process (covered by `04-trial-balance-ai-specification.tex`).
- Arabic invoice pipeline (covered by the Arabic AP specification).

Chapter 2

Business Requirements Specification

Integration with existing chapters

This chapter provides the business governance layer around the technical specifications in earlier chapters:

- **Chapter 15** (Project Roadmap): BR-1.x strategic requirements align with the MoSCoW classification in Table 15.1. High-priority BRs map to Must milestones (M01–M07); Medium-priority BRs map to Should (S01–S02); Low-priority BRs map to Could (C01).

- **Chapter 12** (Agent Instructions): BR-2.x generation requirements provide acceptance criteria for Tasks S1–S4 (schema extraction, taxonomy building, generation, evaluation).

- **Chapter 16** (Implementation Roadmap): BR-3.x quality requirements are addressed by the closed-loop validation capabilities in Phase 2; BR-4.x governance requirements by the workflow engine capabilities in Phase 3.

- **Table 12.3** (FMEA): BR-3.3 (Privacy) and BR-3.4 (Bias) address the "Critic Collusion" and "Schema Contraction" failure modes.

The traceability matrix at the end of this chapter maps each BR to the enhancement and regulatory framework that governs it.

Scope of this chapter

This chapter defines the complete business requirements for enterprise deployment of the Simula synthetic data framework. Requirements are organized across five categories: strategic preparation, generation phase, quality assurance, governance and compliance, and integration infrastructure. Each requirement includes regulatory citations where applicable.

2.1 Requirements Overview

2.1.1 Requirement Identification Convention

Requirements follow the identification pattern `BR-<category>.<sequence>`:

Table 2.1: Business requirement categories.

Category	Prefix	Scope	Count
Strategic & Preparation	BR-1.x	Use case definition, governance integration	5
Generation Phase	BR-2.x	Taxonomy, configuration, metadata	5
Quality Assurance	BR-3.x	Validation, privacy, bias detection	5
Governance & Compliance	BR-4.x	Access control, audit, transparency	5
Integration & Infrastructure	BR-5.x	ML pipelines, storage, deployment	5

2.1.2 Priority Classification

Requirements are classified using MoSCoW prioritization:

- **High**: Must have for initial deployment
- **Medium**: Should have for production maturity
- **Low**: Could have for optimization

2.2 BR-1: Strategic and Preparation Phase

2.2.1 BR-1.1: Use Case Definition

BR-1.1: Clear Use Case Definition

Priority: High
Description: Organizations must clearly define synthetic data use cases and objectives before initiating generation projects. The synthetic data market is projected to reach $3.4 billion by 2031[a], but success depends on starting with high-value, well-scoped use cases.
Typical Use Cases:

- Privacy-sensitive domain data substitution
- Long-tail scenario coverage augmentation
- Adversarial testing data generation
- Training data augmentation for rare events
- Model validation in regulated environments

Acceptance Criteria:

1. Documented use case with clear business objective
2. Identified target domain and data modality
3. Defined success metrics and measurement approach
4. Stakeholder sign-off on scope and expectations

SAP Integration: Align use cases with SAP AI Core deployment scenarios (Text-to-SQL, document extraction, financial analytics).

[a] Market Research Future, "Synthetic Data Generation Market Report," 2024

2.2.2 BR-1.2: Value-Risk Assessment

BR-1.2: Structured Value-Risk Assessment

Priority: High
Description: Before initiating synthetic data projects, organizations must conduct structured value-risk assessments. The UK FCA Synthetic Data Expert Group explicitly identifies clear objective definition and value-risk analysis as foundational to project success[a].
Assessment Dimensions:

- **Value**: Cost savings, time-to-market, compliance enablement
- **Risk**: Privacy leakage, bias amplification, quality degradation
- **Feasibility**: Technical capability, resource availability
- **Regulatory**: Jurisdiction-specific compliance requirements

Acceptance Criteria:

1. Completed value-risk assessment matrix
2. Identified risk mitigation strategies
3. Documented go/no-go decision criteria
4. Executive sponsor approval

[a] FCA, "Synthetic Data Expert Group Report," Section 3.2, 2024

2.2.3 BR-1.3: Governance Framework Integration

BR-1.3: Existing Governance Framework Integration

Priority: High
Description: In the absence of dedicated synthetic data governance frameworks, organizations should integrate synthetic data management into existing model risk management, data governance, and AI ethics frameworks rather than building from scratch.
Integration Points:

- Model Risk Management (MRM) — treat synthetic data as model input
- Data Governance — apply data quality and lineage standards
- AI Ethics Framework — extend bias and fairness requirements
- Information Security — apply data classification policies

Acceptance Criteria:

1. Synthetic data policies integrated into existing governance documentation
2. Roles and responsibilities defined within existing RACI matrices
3. Approval workflows configured in existing governance tools
4. Training materials updated for affected stakeholders

SAP Integration: Leverage SAP GRC (Governance, Risk, Compliance) for policy management and workflow orchestration.

2.2.4 BR-1.4: Cross-Functional Team Formation

BR-1.4: Cross-Functional Team Organization

Priority: Medium
Description: Synthetic data generation and usage requires multi-stakeholder collaboration. The World Economic Forum notes that synthetic data governance is typically constructed by multiple distinct teams working together[a].
Required Roles:

- **Data Science Team**: Technical implementation, model training
- **Domain Experts**: Taxonomy validation, complexity calibration
- **Legal/Compliance**: Privacy risk assessment, regulatory alignment
- **IT/Platform**: Infrastructure provisioning, security controls
- **Business Stakeholders**: Use case definition, acceptance criteria

Acceptance Criteria:

1. Documented team structure with named individuals
2. Defined communication and escalation paths
3. Regular sync cadence established
4. Clear decision-making authority matrix

[a] WEF, "Synthetic Data for Financial Services," p. 18, 2024

2.2.5 BR-1.5: ROI Baseline Establishment

BR-1.5: Investment Return Baseline

Priority: Medium
Description: Organizations should establish clear ROI measurement baselines. Industry data indicates typical benefits include: AI model development speed improvements up to 40%, compliance and data processing cost reductions of 50–60%[a].
Measurement Dimensions:

- Time-to-data: Days from request to available training data
- Data acquisition cost: Cost per training example
- Model performance: Accuracy, F1, or domain-specific metrics
- Compliance cost: Audit preparation, privacy review cycles

Acceptance Criteria:

1. Baseline metrics captured before synthetic data adoption
2. Target improvement thresholds defined
3. Measurement methodology documented
4. Reporting cadence established

[a] Gartner, "Synthetic Data Adoption Trends," 2025

2.3 BR-2: Generation Phase

2.3.1 BR-2.1: Domain Taxonomy Construction and Validation

BR-2.1: Taxonomy Construction and Validation

Priority: High
Description: Simula's global diversification step requires constructing deep hierarchical taxonomies for target domains. Business requirements include domain expert participation in taxonomy review and version management as domain knowledge evolves.
Requirements:

- Expert review interface for taxonomy inspection and correction
- Version control for taxonomy artifacts
- Coverage gap identification and reporting
- Domain vocabulary injection mechanism

Acceptance Criteria:

1. Taxonomy review workflow with expert approval gates
2. Taxonomy versioning with diff visualization
3. Coverage reports showing sampling breadth and depth
4. Domain-specific node validation rules

SAP Integration: Store taxonomy versions in SAP HANA Cloud with metadata tracking. Integrate with SAP Signavio for process-aware taxonomy construction.

2.3.2 BR-2.2: Controllable Parameter Configuration

BR-2.2: Parameter Configuration Interface

Priority: High
Description: Provide business user-friendly interfaces for independent control of generation parameters without requiring technical expertise.
Controllable Parameters:

- **Global Coverage**: Taxonomy sampling breadth and depth
- **Local Diversity**: Variants per concept node
- **Complexity Distribution**: Easy/medium/hard ratio curve
- **Quality Thresholds**: Critic pass rate, diversity minimums

Acceptance Criteria:

1. Visual configuration interface with parameter sliders
2. Preview mode showing expected distribution
3. Parameter validation with constraint warnings
4. Configuration templates for common scenarios

SAP Integration: Expose configuration via SAP Build Apps for low-code workflow construction. Store configurations in SAP HANA Cloud.

2.3.3 BR-2.3: Cost and Resource Budget Control

BR-2.3: Budget Control Mechanism

Priority: Medium
Description: Synthetic data generation involves inference model API costs. Organizations require budget control mechanisms to manage expenses.
Requirements:

- Per-project and per-department generation quotas
- Single-task cost ceiling configuration
- Resource consumption estimation before execution
- Low-cost "draft mode" for taxonomy design validation

Acceptance Criteria:

1. Budget allocation interface by cost center
2. Real-time cost tracking during generation
3. Alerts when approaching budget limits
4. Cost-per-example reporting

SAP Integration: Integrate with SAP AI Core resource quotas and SAP Analytics Cloud for cost visualization.

2.3.4 BR-2.4: Metadata and Provenance Recording

BR-2.4: Complete Metadata Capture

Priority: High
Description: Each generation task must automatically record complete metadata for audit compliance and reproducibility.
Required Metadata:

- Reasoning model version and configuration
- Taxonomy structure snapshot
- Sampling strategy parameters
- Generation timestamp and executor identity
- Random seeds for reproducibility

Acceptance Criteria:

1. Automatic metadata capture on every generation run
2. Metadata stored with tamper-evident integrity
3. Query interface for metadata retrieval
4. Export capability for audit packages

SAP Integration: Store provenance in SAP HANA Cloud with blockchain-style hash chaining. Integrate with SAP Data Intelligence for lineage visualization.

2.3.5 BR-2.5: Incremental Generation and Dataset Evolution

BR-2.5: Incremental Generation Support

Priority: Low
Description: Support incremental generation on existing datasets when taxonomies evolve, avoiding redundant regeneration.
Requirements:

- Detect new/modified taxonomy nodes
- Generate only for changed coverage regions
- Merge incremental examples with existing dataset
- Version-to-version diff analysis

Acceptance Criteria:

1. Taxonomy diff computation between versions
2. Targeted generation for changed nodes only
3. Dataset merge with deduplication
4. Generation efficiency metrics (examples saved)

2.4 BR-3: Quality Assurance and Validation

2.4.1 BR-3.1: Intrinsic Quality Assessment

BR-3.1: Standardized Quality Reporting

Priority: High
Description: Simula's built-in quality assessment capabilities must be productized into standardized reports for business consumption.
Required Reports:

- **Coverage Report**: Taxonomy branches sampled vs. omitted
- **Complexity Distribution**: Generated data complexity vs. target
- **Diversity Metrics**: Intra-concept sample similarity analysis
- **Critic Pass Rates**: Acceptance rates by taxonomy region

Acceptance Criteria:

1. Automated report generation on every dataset
2. Visual dashboards with drill-down capability
3. Threshold alerts for quality degradation
4. Historical trend analysis

SAP Integration: Publish reports to SAP Analytics Cloud. Configure alerts via SAP Alert Notification Service.

2.4.2 BR-3.2: Downstream Task Validation

BR-3.2: Downstream Model Validation

Priority: High
Description: Generated datasets must be validated on downstream models to verify practical utility. Recommended approach: "Train-Synthetic-Test-Real" (TSTR) methodology.
Validation Protocol:

1. Train model on synthetic data
2. Evaluate on held-out real data
3. Compute synthetic-to-real performance gap
4. Identify systematic failure patterns

Acceptance Criteria:

1. Automated TSTR pipeline integration
2. Performance gap quantification by taxonomy region
3. Failure pattern analysis reports
4. Threshold-based quality gates

SAP Integration: Execute validation via SAP AI Core training pipelines. Store results in SAP HANA Cloud for trend analysis.

2.4.3 BR-3.3: Privacy Risk Assessment

BR-3.3: Privacy Risk Evaluation

Priority: High
Description: Even in Simula's seedless mode, generated data may exhibit unintended similarity to real data seen during reasoning model training. Privacy risk assessment mechanisms are essential. IEEE emphasizes that privacy risk metrics and mitigation techniques are core governance components[a].
Assessment Mechanisms:

- Membership inference attack testing
- Similarity scanning against known real datasets
- Differential privacy budget tracking
- Re-identification risk scoring

Acceptance Criteria:

1. Automated privacy risk scoring on generated datasets
2. Configurable risk thresholds with blocking gates
3. Audit-ready privacy assessment reports
4. Remediation workflow for high-risk samples

SAP Integration: Integrate with SAP Data Privacy Integration service for sensitive data detection.

[a] IEEE, "Synthetic Data: Generation, Validation, and Governance," Section 4.3, 2025

2.4.4 BR-3.4: Bias Detection and Mitigation

BR-3.4: Systematic Bias Detection

Priority: Medium
Description: Establish systematic bias detection processes to identify and mitigate inherited biases from reasoning models.
Detection Approaches:

- Distribution analysis on sensitive attributes
- Reasoning model bias inheritance detection
- Over/under-sampling recommendations
- Fairness metric computation

Acceptance Criteria:

1. Automated bias scanning on generation
2. Fairness metric dashboards
3. Bias mitigation tool integration
4. Bias audit trail for compliance

2.4.5 BR-3.5: Continuous Monitoring and Drift Detection

BR-3.5: Quality Drift Monitoring

Priority: Medium
Description: Establish continuous monitoring to detect quality changes due to reasoning model upgrades or taxonomy drift. FCA includes "continuous monitoring and improvement" as one of nine governance principles[a].
Monitoring Capabilities:

- Periodic quality sampling and assessment
- Version-to-version quality drift detection
- Model upgrade impact analysis
- Automated regeneration triggers

Acceptance Criteria:

1. Scheduled quality monitoring jobs
2. Drift detection with statistical significance testing
3. Alert configuration for threshold breaches
4. Automated remediation workflow triggers

SAP Integration: Configure monitoring via SAP AI Core MLOps capabilities. Route alerts through SAP Alert Notification Service.

[a] FCA, "Synthetic Data Expert Group Report," Principle 9, 2024

2.5 BR-4: Governance and Compliance

2.5.1 BR-4.1: Access Control and Permission Management

BR-4.1: Role-Based Access Control

Priority: High
Description: Although synthetic data does not directly contain real personal information, strict access controls remain necessary.
Control Requirements:

- Role-based access (generator, reviewer, consumer separation)
- Dataset access approval workflows
- Additional restrictions for sensitive domains (healthcare, finance)
- Access logging and audit trails

Acceptance Criteria:

1. RBAC configuration in identity provider
2. Approval workflow for dataset access requests
3. Domain-specific access policies
4. Comprehensive access audit logs

SAP Integration: Leverage SAP Cloud Identity Services for authentication. Configure authorization via SAP Authorization and Trust Management Service (XSUAA).

2.5.2 BR-4.2: Regulatory Framework Alignment

BR-4.2: Compliance Framework Alignment

Priority: High
Description: Synthetic data generation and usage must align with applicable regulations. FCA emphasizes that generating and using synthetic data requires consideration of data protection, non-discrimination, and bias regulatory obligations[a].
Regulatory Considerations:

- **GDPR**: Demonstrate anonymization status of synthetic data
- **HIPAA**: Comply with protected health information rules
- **Industry-Specific**: Financial services (FCA), healthcare (FDA)
- **AI Regulations**: EU AI Act risk categorization

Acceptance Criteria:

1. Documented compliance position for each applicable regulation
2. Legal review sign-off on synthetic data usage
3. Regulatory change monitoring process
4. Jurisdiction-specific policy variations

SAP Integration: Integrate with SAP Compliance and GRC solutions for regulatory tracking.

[a] FCA, "Synthetic Data Expert Group Report," Section 5.1, 2024

2.5.3 BR-4.3: Audit Logging and Traceability

BR-4.3: Complete Audit Trail

Priority: High
Description: Establish complete audit tracking. WEF emphasizes that enterprises must prioritize oversight and compliance, building robust traceability and provenance systems[a].
Audit Requirements:

- "Who, when, why, what" for every data access
- Complete usage chain (generation → training → deployment)
- Compliance review decision records
- Long-term audit log retention

Acceptance Criteria:

1. Tamper-evident audit log storage
2. Query interface for audit investigations
3. Retention policy enforcement
4. Audit export for regulatory submissions

SAP Integration: Utilize SAP Audit Log Service for tamper-evident logging. Store long-term archives in SAP Object Store with retention policies.

[a] WEF, "Synthetic Data for Financial Services," p. 24, 2024

2.5.4 BR-4.4: Transparency and Explainability

BR-4.4: External Transparency

Priority: Medium
Description: Organizations must be able to explain synthetic data usage to external stakeholders (customers, regulators, partners). This addresses FCA principles of "transparency" and "explainability"[a].
Transparency Requirements:

- Documentation of synthetic nature and generation method
- Explanation of synthetic vs. real data differences
- Justification for synthetic data usage
- Model card integration for downstream models

Acceptance Criteria:

1. Synthetic data documentation templates
2. Stakeholder communication guidelines
3. Model card integration with synthetic data lineage
4. FAQ materials for common inquiries

[a] FCA, "Synthetic Data Expert Group Report," Principles 1–2, 2024

2.5.5 BR-4.5: Incident Response and Remediation

BR-4.5: Incident Response Process

Priority: Medium
Description: Establish incident response processes for synthetic data related security events.
Incident Scenarios:

- Generated data inadvertently contains real information
- Systematic bias causes discriminatory model outputs
- Generated data is misused or leaked
- Quality degradation impacts downstream systems

Acceptance Criteria:

1. Incident classification and severity matrix
2. Responsible party identification
3. Escalation and notification paths
4. Remediation playbooks

SAP Integration: Integrate with existing SAP incident management workflows. Configure alerts through SAP Alert Notification Service.

2.6 BR-5: Integration and Infrastructure

2.6.1 BR-5.1: ML Pipeline Integration

BR-5.1: MLOps Pipeline Integration

Priority: High
Description: Simula must integrate as an orchestratable component within existing ML pipelines.
Integration Requirements:

- API-driven generation task triggering
- Integration with major MLOps platforms
- Automatic dataset push to training storage
- Pipeline status and progress reporting

Acceptance Criteria:

1. REST/gRPC API for generation tasks
2. Pre-built integrations (Kubeflow, MLflow, Vertex AI)
3. Webhook notifications for completion events
4. Pipeline artifact registration

SAP Integration: Primary integration with SAP AI Core. Secondary integrations with SAP Data Intelligence and SAP BTP Kyma Runtime for containerized execution.

2.6.2 BR-5.2: Data Storage and Version Management

BR-5.2: Unified Data Governance

Priority: Medium
Description: Synthetic datasets should follow the same governance policies as real datasets within organizational data infrastructure.
Storage Requirements:

- Version-controlled data lake/warehouse storage
- Dataset labeling, search, and reuse support
- Data catalog integration
- Retention policy enforcement

Acceptance Criteria:

1. Storage in enterprise data lake with versioning
2. Metadata registration in data catalog
3. Search and discovery interface
4. Automated retention enforcement

SAP Integration: Store datasets in SAP HANA Cloud or SAP Datasphere. Register metadata in SAP Data Intelligence catalog.

2.6.3 BR-5.3: Elastic Compute Resource Scheduling

BR-5.3: Compute Resource Management

Priority: Medium
Description: Synthetic data generation is compute-intensive. Support dynamic resource allocation based on workload.
Scheduling Requirements:

- Dynamic resource allocation by task size
- Batch and real-time generation modes
- Priority queue management
- Cost-optimized resource selection

Acceptance Criteria:

1. Auto-scaling based on queue depth
2. Priority scheduling with preemption
3. Resource utilization dashboards
4. Cost optimization recommendations

SAP Integration: Leverage SAP AI Core auto-scaling. Use SAP BTP Kubernetes runtime for containerized workloads.

2.6.4 BR-5.4: Multi-Environment Deployment Support

BR-5.4: Deployment Flexibility

Priority: Low
Description: Support multiple deployment models to accommodate varying security requirements across organizations.
Deployment Models:

- **Public Cloud SaaS**: Standard scenarios
- **Private Deployment**: High-security requirements
- **VPC Deployment**: Regulated industry compliance
- **Hybrid**: Combination of above

Acceptance Criteria:

1. Documented deployment architectures for each model
2. Security certifications per deployment type
3. Network isolation configuration guides
4. Data residency compliance options

SAP Integration: Leverage SAP BTP multi-environment support. Configure SAP Private Link for secure connectivity.

2.7 Requirements Traceability Matrix

Legend: E2 = Domain Calibration, E3 = Closed-Loop Validation, E4 = Workflow Engine

Table 2.2: Requirements to enhancement traceability.

Requirement	**Priority**	**Enhancement**	**Regulatory Reference**
BR-1.1	High	—	—
BR-1.2	High	—	FCA Principle 3
BR-1.3	High	E4	WEF Governance
BR-1.4	Medium	—	WEF Cross-functional
BR-1.5	Medium	E3	—
BR-2.1	High	E2	—
BR-2.2	High	E4	—
BR-2.3	Medium	E4	—
BR-2.4	High	E4	FCA Principle 7
BR-2.5	Low	E4	—
BR-3.1	High	E3	—
BR-3.2	High	E3	IEEE Fidelity
BR-3.3	High	E3	IEEE Privacy, FCA Principle 6
BR-3.4	Medium	E3	FCA Principle 8
BR-3.5	Medium	E3	FCA Principle 9
BR-4.1	High	E4	FCA Principle 6
BR-4.2	High	—	GDPR, HIPAA, FCA
BR-4.3	High	E4	WEF Traceability
BR-4.4	Medium	E4	FCA Principles 1–2
BR-4.5	Medium	—	—
BR-5.1	High	E3, E4	—
BR-5.2	Medium	E4	IEEE Provenance
BR-5.3	Medium	—	—
BR-5.4	Low	—	—

Chapter 3

Canonical Data Schema

3.1 Layout of `src/training/pipeline/`

Table 3.1: Pipeline Module Layout

Path	Purpose
`simula_config.py`	Configuration dataclasses for all components
`simula_llm_client.py`	OpenAI-compatible async LLM client
`hana_schema_extractor.py`	HANA Cloud schema extraction via MCP
`simula_taxonomy_builder.py`	Algorithm 1: Taxonomy generation
`simula_data_generator.py`	Algorithm 2: Training data synthesis
`schema_registry.py`	In-memory schema registry

3.2 Training Example Record

Each generated training example is a JSON object with the following structure:

Listing 3.1: TrainingExample dataclass (simula_data_generator.py)

```python
@dataclass
class TrainingExample:
    """A single training example for Text-to-SQL."""
    id: str                     # Unique identifier
    question: str               # Natural language question
    sql: str                    # Target SQL query
    schema_context: str         # Table/column descriptions
    taxonomy_path: list[str]    # Factor nodes used
    complexity_score: float     # 0.0 to 1.0
    meta_prompt_id: str         # Reference to meta-prompt record
    audience: str               # human, agent, or dual
    is_complexified: bool       # Whether complexification applied
    critic_passed: bool         # Whether double-critic passed
    generation_metadata: dict   # Timestamps, model, etc.
```

3.3 Schema Registry

The `SchemaRegistry` holds table and column metadata extracted from HANA:

Listing 3.2: Schema registry structures (schema_registry.py)

```python
@dataclass
class Column:
    name: str
    data_type: str
    length: int | None
    nullable: bool
    description: str = ""

@dataclass
class TableSchema:
    schema_name: str
    table_name: str
    columns: list[Column]
    primary_key: list[str] = field(default_factory=list)
    description: str = ""
```

3.4 Taxonomy Structure

Taxonomies are hierarchical trees where each node represents a factor of variation:

Listing 3.3: Taxonomy structures (simula_taxonomy_builder.py)

```python
@dataclass
class TaxonomyNode:
    id: str
    name: str
    description: str
    level: int
    parent_id: str | None
    children: list["TaxonomyNode"]
    examples: list[str]  # Sample values

@dataclass
class Factor:
    id: str
    name: str
    description: str
    taxonomy: "Taxonomy"

@dataclass
class Taxonomy:
    id: str
    factor: str
    root: TaxonomyNode
    depth: int
    total_nodes: int
```

3.5 Enumerations

Table 3.2: Key Enumerations

Enum	Values	Description
`LLMProvider`	VLLM, OPENAI, ANTHROPIC	LLM backend type
`SamplingStrategy`	UNIFORM, WEIGHTED, COVERAGE	How to sample taxonomy nodes

3.6 Unified Schema Registry

As of 18 April 2026, all schemas in this specification are published in the unified schema registry at `docs/schema/`. Migration status is *complete* (see `docs/schema/registry.json` field `migration.status`). This consolidation addresses cross-document schema divergence identified in the specification review.

Registry index. `docs/schema/registry.json` — Master index of all schemas across domains.

Common types. `docs/schema/common/base-types.schema.json` — Shared definitions (`Metadata`, `Currency`, etc.) referenced via JSON Schema `$ref`.

Consolidated enums. `docs/schema/common/enums.yaml` — Single source of truth for enumeration values used by all four specifications.

Simula schemas. `docs/schema/simula/` — Domain-specific schemas: `taxonomy.schema.json`, `training-example.schema.json`, `hana-schema-entry.schema.json`, `config.schema.json`, and `meta-prompt.schema.json`. All marked `status: stable`.

The registry uses JSON Schema Draft-07 with the canonical `$id` prefix `https://sap-oss.github.io/schema/`. Validation:

```
python3 docs/schema/validate.py --registry-check
python3 docs/schema/validate.py --domain simula \
    --schema training-example.schema.json --file example.json
```

See `docs/schema/README.md` for Python usage examples; the CI pipeline executes `-registry-check` on every pull request that touches `docs/schema/` or any spec chapter that references these schemas.

Chapter 4

HANA Schema Extraction Pipeline

4.1 Pipeline at a Glance

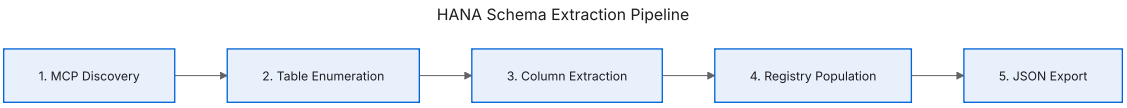

Figure 4.1: The five extraction stages; each emits a well-typed payload to the next.

4.2 Stage 1 — MCP Discovery

The extractor connects to the AI Core PAL MCP server and discovers available tools:

Listing 4.1: MCP connection (hana_schema_extractor.py, lines 45–60)

```
1  async def connect(self) -> None:
2      """Establish MCP connection to AI Core PAL."""
3      # Handle both HTTP->WS and HTTPS->WSS
4      transport_url = self.config.mcp_url
5      transport_url = transport_url.replace("https://", "wss://")
6      transport_url = transport_url.replace("http://", "ws://")
7      transport_url = f"{transport_url}/mcp/ws"
8
9      self._transport = WebSocketClientTransport(transport_url)
10     self._session = ClientSession(self._transport)
11
12     await self._session.initialize()
13
14     # Discover available tools
15     tools_result = await self._session.list_tools()
16     self._available_tools = {t.name for t in tools_result.tools}
```

Precondition. The MCP server must expose at least one of: `execute_sql`, `hana_tables`, or `list_tables`.

4.3 Stage 2 — Table Enumeration

Tables are discovered via the `hana_tables` MCP tool or by querying `SYS.TABLES`:

Listing 4.2: Table discovery (hana_schema_extractor.py, lines 85–105)

```python
async def _discover_tables(self, schema: str) -> list[str]:
    """Discover tables in a schema."""
    if "hana_tables" in self._available_tools:
        result = await self._session.call_tool(
            "hana_tables",
            {"schema": schema}
        )
        return [t["table_name"] for t in result.content]
    else:
        # Fallback to direct SQL
        result = await self._execute_sql(
            'SELECT TABLE_NAME FROM SYS.TABLES '
            'WHERE SCHEMA_NAME = ?',
            [schema]
        )
        return [row["TABLE_NAME"] for row in result]
```

4.4 Stage 3 — Column Extraction

For each table, column metadata is extracted from `SYS.TABLE_COLUMNS`:

Listing 4.3: Column extraction SQL

```sql
SELECT
    COLUMN_NAME,
    DATA_TYPE_NAME,
    LENGTH,
    IS_NULLABLE
FROM SYS.TABLE_COLUMNS
WHERE SCHEMA_NAME = ? AND TABLE_NAME = ?
ORDER BY POSITION
```

4.5 Stage 4 — Registry Population

Extracted metadata is normalized and stored in the `SchemaRegistry`:

Listing 4.4: Registry population (hana_schema_extractor.py, lines 140–165)

```python
async def extract_table_schema(
    self, schema: str, table: str
) -> TableSchema:
    """Extract full schema for a single table."""
    columns_data = await self._get_table_columns(schema, table)

    columns = [
        Column(
            name=col["COLUMN_NAME"],
            data_type=col["DATA_TYPE_NAME"],
```

```
            length=col.get("LENGTH"),
            nullable=col.get("IS_NULLABLE", "TRUE") == "TRUE",
        )
        for col in columns_data
    ]

    return TableSchema(
        schema_name=schema,
        table_name=table,
        columns=columns,
    )
```

4.6 Stage 5 — JSON Export

The registry exports to JSON for caching and debugging:

Listing 4.5: JSON export format

```
{
  "schemas": {
    "PAL_STORE": {
      "tables": {
        "ESG_METRICS": {
          "columns": [
            {"name": "METRIC_ID", "type": "INTEGER", ...},
            {"name": "SCOPE", "type": "NVARCHAR", ...}
          ]
        }
      }
    }
  },
  "extracted_at": "2026-04-18T08:00:00Z"
}
```

Chapter 5

Taxonomy Generation Engine

5.1 Algorithm 1: Reasoning-Driven Taxonomy Generation

The taxonomy builder implements Algorithm 1 from the Simula research paper. Given a dataset description y and target depth D, it produces a set of taxonomies $\mathcal{T} = \{T_i\}_{i=0}^{K}$ that map the conceptual space.

5.2 Step 1: Factor Identification

Given schema metadata, the LLM proposes factors of variation:

Listing 5.1: Factor identification prompt

```
SYSTEM: You identify the prime factors of variation for a
        Text-to-SQL training dataset.

USER:   Schema context:
        {schema_context}

        Identify 3-7 factors that capture the main dimensions
        of queries users might ask. Examples:
        - "query_complexity" (simple, join, aggregation, subquery)
        - "time_range" (point-in-time, period, YTD, MTD)
        - "entity_type" (customer, order, product, supplier)

        Output JSON: {"factors": [{"name": "...", "description": "..."}]}
```

5.3 Step 2: Breadth-First Expansion

Each factor is expanded into a taxonomy tree. For each node, the LLM proposes children:

Listing 5.2: Node expansion (simula_taxonomy_builder.py, lines 180–220)

```
async def _expand_node(
    self, node: TaxonomyNode, context: str
) -> list[TaxonomyNode]:
    """Expand a single node with Best-of-N proposals."""
```

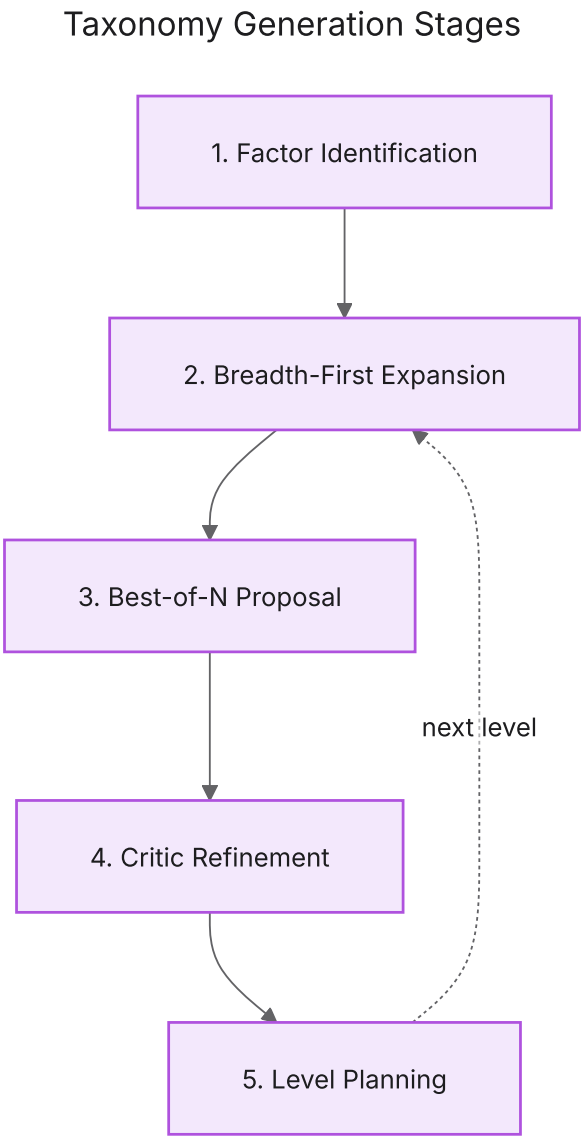

Figure 5.1: Taxonomy generation stages with iterative refinement.

```

    # Step 1: Generate N proposals
    proposals = await self.llm.best_of_n(
        prompt=self._build_expansion_prompt(node, context),
        n=self.config.taxonomy.best_of_n,
        temperature=0.8,
    )

    # Merge proposals into candidate set
    candidates = self._merge_proposals(proposals)

    # Step 2: Critic refinement
    refined = await self._critic_refine(node, candidates, context)

    return refined
```

5.4 Step 3: Best-of-N Proposal

Multiple proposals are generated to increase coverage:

Listing 5.3: Best-of-N sampling

```
async def best_of_n(
    self,
    prompt: str,
    n: int = 5,
    temperature: float = 0.8,
) -> list[str]:
    """Generate N diverse completions."""
    tasks = [
        self.generate(prompt, temperature=temperature)
        for _ in range(n)
    ]
    responses = await asyncio.gather(*tasks)
    return [r.content for r in responses]
```

Known limitation: All N proposals use the same temperature, which may reduce diversity compared to varying temperature across samples. Empirical testing suggests this is acceptable for taxonomy generation where structural diversity (prompt variation) dominates over sampling diversity. v2 will evaluate temperature annealing ($T \in [0.6, 1.0]$).

5.5 Step 4: Critic Refinement

The critic evaluates and refines the proposed children:

Listing 5.4: Critic refinement prompt

```
SYSTEM: You are a taxonomy critic. Evaluate the proposed children
        for COMPLETENESS, SOUNDNESS, and SPECIFICITY.

USER:   Parent node: {parent.name} - {parent.description}
        Siblings: {siblings}
```

```
        Proposed children: {candidates}

        Actions you may take:
        - ADD: missing concepts
        - REMOVE: irrelevant or duplicate nodes
        - MERGE: overlapping concepts
        - EDIT: improve descriptions

        Output JSON with refined children list.
```

5.6 Step 5: Level Planning

After completing a level, a plan is generated for the next level:

Listing 5.5: Level planning

```
async def _plan_next_level(
    self, current_level_nodes: list[TaxonomyNode]
) -> str:
    """Generate guidance for next level expansion."""
    prompt = f"""
    Current level nodes: {[n.name for n in current_level_nodes]}

    Generate a plan for expanding the next level that ensures:
    1. Consistent granularity across all branches
    2. No overlap between sibling expansions
    3. Complete coverage of the concept space
    """
    response = await self.llm.generate(prompt)
    return response.content
```

5.7 Taxonomy Depth and Dataset Size Relationship

The relationship between taxonomy depth (D), number of factors (K), and target dataset size (N) is critical for planning generation runs. This section provides guidance for configuring these parameters.

5.7.1 Node Count Growth

Taxonomy nodes grow exponentially with depth. Given an average branching factor b (typically 3–5 for well-structured taxonomies), the total node count at depth D is:

$$\text{Total Nodes} = K \times \sum_{d=0}^{D} b^d = K \times \frac{b^{D+1} - 1}{b - 1}$$

For $K = 4$ factors and $b = 4$ average branching:

Table 5.1: Expected Node Count by Taxonomy Depth

Depth	Nodes/Factor	Total Nodes (K=4)	Leaf Nodes
1	5	20	16
2	21	84	64
3	85	340	256
4	341	1,364	1,024
5	1,365	5,460	4,096

5.7.2 Target Examples vs Taxonomy Depth

To achieve adequate coverage, each leaf node should have multiple examples. The recommended formula for minimum examples:

$$N_{\text{min}} = L \times m \times (1 + r)$$

Where:

- L = number of leaf nodes (approximately $K \times b^D$)
- m = minimum examples per unique mix (recommended: 5–10)
- r = critic rejection rate (typically 0.08–0.15)

Table 5.2: Recommended Target Examples by Depth (K=4, b=4, m=5, r=0.10)

Depth	Leaf Nodes	Unique Mixes	Min Examples	Recommended
2	64	100	550	1,000
3	256	400	2,200	5,000
4	1,024	1,600	8,800	10,000–15,000
5	4,096	6,500	35,750	50,000+

5.7.3 Configuration Guidelines

Small pilot (1,000 examples): Use depth 2, 3–4 factors. Focus on demonstrating coverage, not exhaustive diversity.

Standard production (10,000 examples): Use depth 3–4, 4–5 factors. Balances coverage with generation cost.

Large-scale training (50,000+ examples): Use depth 4–5, 5–7 factors. Requires significant LLM compute budget.

5.7.4 Cost Estimation

Each generated example requires approximately:

- 1–2 LLM calls for meta-prompt generation
- 1 LLM call for example generation

- 2 LLM calls for double-critic evaluation
- 0–1 LLM calls for complexification (50% of examples)

Total: approximately 4.5 LLM calls per accepted example (accounting for rejections). At $0.002/1K tokens and 1K tokens per call, the cost is approximately $0.01 per accepted example, or $100 per 10,000 examples.

5.8 Worked Example: Trial Balance Taxonomy

Table 5.3: Sample Taxonomy for Trial Balance Domain

Level 0	**Level 1**	**Level 2**	**Level 3**
query_type	aggregation	sum	by_account
			by_department
		count	distinct_accounts
	comparison	variance	MoM
			YoY
	filter	by_threshold	exceeds_100M

Chapter 6

Training Data Generation Engine

6.1 Algorithm 2: Agentic Data Synthesis

Given taxonomies $\mathcal{T}$ and a target size N, the generator produces a synthetic dataset optimized for diversity, complexity, and quality.

6.2 Step 1: Mix Sampling (Global Diversity)

Nodes are sampled from taxonomies to create "mixes"—combinations of factors:

Listing 6.1: Mix sampling (simula_data_generator.py, lines 120–150)

```
def _sample_mix(
    self, taxonomies: list[Taxonomy], strategy: SamplingStrategy
) -> list[TaxonomyNode]:
    """Sample a mix of nodes from taxonomies."""
    mix = []
    for taxonomy in taxonomies:
        if strategy == SamplingStrategy.UNIFORM:
            # Sample uniformly from leaf nodes
            leaves = self._get_leaf_nodes(taxonomy.root)
            node = random.choice(leaves)
        elif strategy == SamplingStrategy.COVERAGE:
            # Prioritize under-sampled nodes
            node = self._get_least_sampled_node(taxonomy)
        mix.append(node)
    return mix
```

6.3 Step 2: Meta-Prompt Generation (Local Diversity)

Each mix is converted to a meta-prompt that guides example generation:

Listing 6.2: Meta-prompt generation

```
async def _generate_meta_prompt(
    self, mix: list[TaxonomyNode], schema_context: str
) -> str:
    """Generate a meta-prompt from a taxonomy mix."""
```

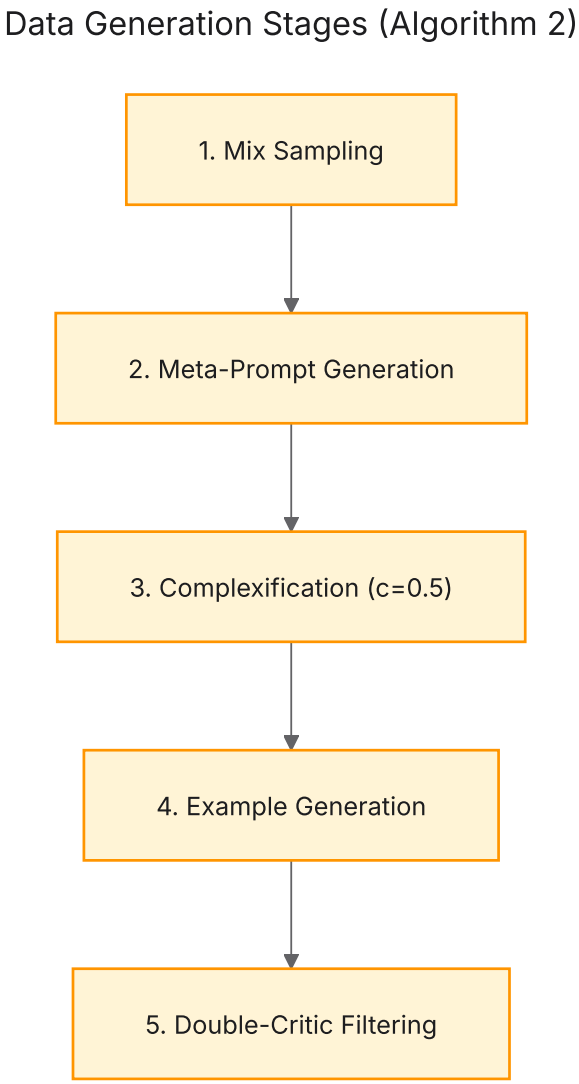

Figure 6.1: Data generation stages implementing Algorithm 2.

```
    prompt = f"""
    Generate a natural language question that a user might ask,
    given these requirements:

    {self._format_mix(mix)}

    Schema context:
    {schema_context}

    The question should be realistic and answerable with SQL.
    """
    response = await self.llm.generate(prompt, temperature=0.9)
    return response.content
```

6.4 Step 3: Complexification

A fraction c (default 0.5) of examples undergo complexification:

Listing 6.3: Complexification prompt

```
SYSTEM: You increase the complexity of a Text-to-SQL example
        while maintaining correctness.

USER:   Original question: {question}
        Original SQL: {sql}

        Add complexity by:
        - Adding edge cases or constraints
        - Requiring additional joins
        - Adding subqueries or CTEs
        - Handling NULL values explicitly

        Output the complexified question and SQL.
```

6.5 Step 4: Example Generation

The meta-prompt is used to generate a complete training example:

Listing 6.4: Example generation

```
async def _generate_example(
    self, meta_prompt: str, schema_context: str
) -> TrainingExample:
    """Generate a single training example."""
    prompt = f"""
    Generate a Text-to-SQL training example.

    Meta-prompt: {meta_prompt}
    Schema: {schema_context}

    Output JSON:
    {{
      "question": "...",
```

```
      "sql": "SELECT ..."
    }}
    """
    response = await self.llm.generate(prompt)
    return self._parse_example(response.content)
```

6.6 Step 5: Double-Critic Filtering

The double-critic mitigates sycophancy bias by independently querying correctness and incorrectness:

Listing 6.5: Double-critic evaluation (simula_llm_client.py, lines 180–220)

```
async def double_critic_evaluate(
    self, question: str, sql: str, schema: str
) -> tuple[bool, str]:
    """Evaluate with double-critic to mitigate sycophancy."""

    # Query 1: Is this correct?
    correct_response = await self.generate(
        f"Is this SQL correct for the question?\n"
        f"Question: {question}\n"
        f"SQL: {sql}\n"
        f"Schema: {schema}\n"
        f"Answer YES or NO with explanation."
    )

    # Query 2: Is this incorrect?
    incorrect_response = await self.generate(
        f"Is this SQL INCORRECT for the question?\n"
        f"Question: {question}\n"
        f"SQL: {sql}\n"
        f"Schema: {schema}\n"
        f"Answer YES or NO with explanation."
    )

    # Accept only if: correct=YES AND incorrect=NO
    is_valid = (
        "YES" in correct_response.content.upper() and
        "NO" in incorrect_response.content.upper()
    )

    return is_valid,
   f"{correct_response.content}\n---\n{incorrect_response.content}"
```

6.7 Schema Context Pruning

For production deployments with wide tables (100+ columns), the full schema context may exceed the LLM's context window. The schema context pruning strategy selects relevant columns to keep the prompt within token limits while preserving generation quality.

6.7.1 Token Budget

Given a target token limit T (default: 4096 for schema context), allocate:

- 60% for directly relevant columns (based on taxonomy path)
- 25% for structurally related columns (foreign keys, common joins)
- 15% for metadata and descriptions

6.7.2 Pruning Algorithm

Listing 6.6: Schema context pruning algorithm

```python
def prune_schema_context(
    schema: SchemaRegistry,
    taxonomy_path: list[str],
    max_tokens: int = 4096,
    estimator: TokenEstimator = default_estimator
) -> str:
    """
    Prune schema context to fit within token budget.

    Strategy:
    1. Identify primary tables from taxonomy path keywords
    2. Include all columns from primary tables (up to 30 per table)
    3. Include foreign key relationships
    4. Include frequently-joined tables (from usage statistics)
    5. Truncate descriptions to first sentence
    """
    # Extract keywords from taxonomy path
    keywords = extract_keywords(taxonomy_path)

    # Score tables by relevance
    scored_tables = []
    for table in schema.tables:
        relevance = compute_relevance(table, keywords)
        scored_tables.append((table, relevance))

    # Sort by relevance and select top tables
    scored_tables.sort(key=lambda x: x[1], reverse=True)

    # Build context incrementally within budget
    context_parts = []
    token_count = 0

    for table, relevance in scored_tables:
        table_context = format_table_context(
            table,
            max_columns=30,
            truncate_descriptions=True
        )
        table_tokens = estimator.count(table_context)
```

```
        if token_count + table_tokens <= max_tokens:
            context_parts.append(table_context)
            token_count += table_tokens
        else:
            break

    return "\n\n".join(context_parts)
```

6.7.3 Relevance Scoring

Column relevance is computed based on:

1. **Name match** (weight: 0.4): Column name contains taxonomy keywords
2. **Type affinity** (weight: 0.2): Column type matches expected query patterns (e.g., DATE for temporal queries, DECIMAL for aggregations)
3. **Structural importance** (weight: 0.2): Primary keys, foreign keys, indexes
4. **Usage frequency** (weight: 0.2): Historical query patterns if available

6.7.4 Configuration

Table 6.1: Schema Pruning Configuration

Parameter	Default	Description
`SIMULA_SCHEMA_MAX_TOKENS`	4096	Max tokens for schema context
`SIMULA_MAX_COLUMNS_PER_TABLE`	30	Column limit per table
`SIMULA_INCLUDE_FK_DEPTH`	2	Foreign key traversal depth
`SIMULA_TRUNCATE_DESCRIPTIONS`	true	Truncate to first sentence

6.7.5 Quality Impact

Schema pruning has measurable impact on generation quality:

- **Aggressive pruning** (<2048 tokens): May miss relevant columns, leading to incomplete SQL. Use only for narrow-domain generation.
- **Standard pruning** (4096 tokens): Suitable for most production scenarios with tables up to 200 columns.
- **Full context** (no pruning): Preferred when context window permits; required for complex multi-table queries.

The v1.3 roadmap includes adaptive pruning based on taxonomy complexity—complex taxonomy paths will automatically receive larger schema budgets.

6.8 Output Format

Generated examples are written to JSONL:

Listing 6.7: JSONL output format

```
{"id": "sim-001", "question": "What is the total revenue...",
 "sql": "SELECT SUM(amount) FROM...", "complexity_score": 0.3, ...}
{"id": "sim-002", "question": "Show all customers with...",
 "sql": "SELECT * FROM customers WHERE...", "complexity_score": 0.7, ...}
```

Chapter 7

Environment and CLI

7.1 Configuration Table

Table 7.1: Environment Variables

Variable	Default	Description
AICORE_PAL_MCP_URL	http://localhost:8084	AICore PAL MCP endpoint
VLLM_BASE_URL	http://localhost:9180/v1	vLLM TurboQuant endpoint
VLLM_MODEL	Qwen/Qwen3.5-35B	Model for generation
SIMULA_TARGET_COUNT	100000	Target training examples
SIMULA_COMPLEXITY_RATIO	0.5	Fraction to complexify
SIMULA_TAXONOMY_DEPTH	4	Max taxonomy depth
SIMULA_BEST_OF_N	5	Proposals per expansion
SIMULA_ENABLE_CRITIC	true	Enable double-critic
SIMULA_OUTPUT_DIR	data/training/	Output directory

7.2 CLI Usage

Listing 7.1: CLI entry point

```
# Basic usage
python src/training/scripts/run_hana_pipeline.py

# Full customization
python src/training/scripts/run_hana_pipeline.py \
    --hana-schemas PAL_STORE,ESG_METRICS \
    --llm-url http://localhost:9180/v1 \
    --target-count 50000 \
    --complexity-ratio 0.5 \
    --taxonomy-depth 4 \
    --output data/training/

# Schema extraction only
python src/training/scripts/run_hana_pipeline.py \
    --extract-only \
    --output schemas/
```

7.3 Output Files

Table 7.2: Pipeline Output Files

File	Description
`schemas.json`	Extracted HANA schema metadata
`taxonomies.json`	Generated taxonomy trees
`training_data.jsonl`	Final training examples
`generation_stats.json`	Statistics (counts, rates, timing)
`rejected_examples.jsonl`	Examples that failed critic
`complexity_elo_scores.json`	Elo-calibrated complexity scores
`coverage_report.json`	Taxonomic coverage metrics
`diversity_metrics.json`	Global + local diversity
`critic_calibration.json`	Critic validation results

Chapter 8

Evaluation Framework

This section specifies the evaluation components required for Simula methodology compliance, as described in Sections 2.3, 3.1, and 3.3 of the research paper.

8.1 Calibrated Complexity Scoring (Algorithm 3)

Raw per-sample complexity scores are noisy due to model overconfidence. The complexity calibrator uses batch-wise relative scoring with Elo ranking.

8.1.1 Elo Rating Algorithm

The Elo update follows the standard formula (Elo & Sloan, 1978), adapted for complexity comparisons. Given two examples a and b with ratings R_a and R_b, where a has higher raw complexity score:

1. Compute expected outcome: $E_a = \frac{1}{1+10^{(R_b - R_a)/400}}$
2. Update ratings with K-factor = 32: $R'_a = R_a + K \cdot (1 - E_a)$, $R'_b = R_b + K \cdot (0 - (1 - E_a))$

Listing 8.1: Elo computation (simula_complexity_calibrator.py)

```python
def _update_elo(self, id_a: str, id_b: str, outcome_a: float) -> None:
    """
    Update Elo ratings for a pairwise comparison.

    Args:
        id_a, id_b: Example identifiers
        outcome_a: 1.0 if A wins (higher complexity), 0.0 if B wins, 0.5 for draw

    Standard Elo formula per Elo & Sloan (1978).
    """
    rating_a = self._scores[id_a].elo_rating
    rating_b = self._scores[id_b].elo_rating

    # Expected outcomes
    expected_a = 1 / (1 + math.pow(10, (rating_b - rating_a) / 400))
    expected_b = 1 - expected_a

    # K-factor = 32 (standard for chess, appropriate for complexity)
```

```
    K = self.config.elo_k_factor  # default 32

    # Update ratings
    self._scores[id_a].elo_rating = rating_a + K * (outcome_a - expected_a)
    self._scores[id_b].elo_rating = rating_b + K * ((1 - outcome_a) -
   expected_b)
```

8.1.2 Usage

Listing 8.2: Complexity calibrator usage

```
from src.training.pipeline import (
    SimulaComplexityCalibrator,
    ComplexityConfig,
)

calibrator = SimulaComplexityCalibrator(
    llm_client=llm,
    config=ComplexityConfig(
        batch_size=5,   # BS=5 from Figure 10
        n_samples=5,    # N=5 from Figure 10
        enable_elo=True,
        elo_k_factor=32,  # Standard K-factor
    ),
)

# Calibrate all examples
scores = await calibrator.calibrate(examples)

# Split by complexity (Figure 7)
low, mid, high = calibrator.split_by_complexity(
    examples,
    low_percentile=0.4,
    high_percentile=0.6,
)

# Export scores
calibrator.export_scores("complexity_elo_scores.json")
```

Key insight from paper: High-complexity data yields +10% accuracy on GSM8k at 64k samples (Figure 7), but can *hurt* performance when the teacher model is weak (LEXam case).

8.2 Taxonomic Coverage Evaluation

The coverage evaluator measures Level Ratio Coverage—the proportion of unique taxonomy nodes covered at each level:

Listing 8.3: Coverage evaluator usage

```
from src.training.pipeline import (
    SimulaCoverageEvaluator,
    CoverageConfig,
```

```
    identify_coverage_gaps,
)

evaluator = SimulaCoverageEvaluator(
    llm_client=llm,
    config=CoverageConfig(
        max_level=4,
        sample_size=1000,
    ),
)

# Evaluate against all taxonomies
reports = await evaluator.evaluate_multiple(examples, taxonomies)

# Identify gaps
gaps = identify_coverage_gaps(reports, threshold=0.5)
print(f"Under-covered nodes: {gaps}")

# Export
evaluator.export_all_reports(reports, "coverage_report.json")
```

8.3 Embedding-Based Diversity Metrics

Following Yu et al. (2023), diversity is measured via cosine distance in embedding space. **Global diversity** is the average pairwise distance across the dataset. **Local diversity** is the average k-NN distance.

Listing 8.4: Diversity analyzer usage

```
from src.training.pipeline import (
    SimulaDiversityAnalyzer,
    DiversityConfig,
)

analyzer = SimulaDiversityAnalyzer(
    config=DiversityConfig(
        embedding_model="sentence-transformers/all-MiniLM-L6-v2",
        knn_k=10,
    ),
)

report = analyzer.analyze(examples, text_field="question")
print(f"Global: {report.global_diversity:.4f}")
print(f"Local:  {report.local_diversity:.4f}")

# Identify outliers (near-duplicates or errors)
low_ids, high_ids = analyzer.identify_outliers(report)
```

Key insight: Global and Local diversification have *additive* effects (Figure 4). Optimizing only one is insufficient.

8.4 Critic Calibration Validation

The critic validator measures $p(y)$ and $p(y_{\text{corrupt}})$ to ensure the double-critic is properly calibrated:

Listing 8.5: Critic validator usage

```
from src.training.pipeline import (
    SimulaCriticValidator,
    CriticValidationConfig,
)

validator = SimulaCriticValidator(
    llm_client=llm,
    config=CriticValidationConfig(
        validation_split=0.05,
        max_samples=200,
    ),
)

report = await validator.validate(examples, schema_context)
print(f"p(y)        = {report.p_y:.3f}")
print(f"p(y_corrupt)= {report.p_y_corrupt:.3f}")
print(f"Calibrated: {report.is_well_calibrated}")
```

Key insight: Critic effectiveness degrades with task complexity (Figure 3b). For effective rejection sampling, we need $p(y) > 0.8$ and $p(y_{\text{corrupt}}) < 0.3$.

8.5 Student-Teacher Gap Analysis

The paper shows that downstream performance saturates when the student bridges approximately 83% of the teacher-student gap (Section 4.3):

Listing 8.6: Gap analysis

```
from src.training.pipeline import SimulaConfig

config = SimulaConfig()
config.training.teacher_accuracy = 0.70  # measured
config.training.student_baseline = 0.40  # measured

expected = config.training.expected_saturation_accuracy()
print(f"Expected saturation: {expected:.1%}")  # ~65%

# Full analysis
print(config.training.gap_analysis())
```

8.6 Adaptive Features (v1.2)

8.6.1 Adaptive Complexity Ratio (Figure 7)

The paper shows that complex data can hurt performance when the teacher model is weak (LEXam case). The adaptive complexity feature automatically reduces the complexity ratio when teacher accuracy is below 60%:

Listing 8.7: Adaptive complexity ratio

```
# In simula_config.py GenerationConfig
def get_adaptive_complexity_ratio(self, teacher_accuracy: float | None) ->
    float:
    """Reduce complexity for weak teachers (Figure 7)."""
    if not self.adaptive_complexity or teacher_accuracy is None:
        return self.complexity_ratio

    if teacher_accuracy < 0.6:
        return 0.2  # Significantly reduce
    elif teacher_accuracy < 0.7:
        return min(self.complexity_ratio, 0.35)
    return self.complexity_ratio
```

8.6.2 Adaptive Critic Thresholding (Figure 3b)

Critic effectiveness degrades with complexity. The adaptive threshold increases the acceptance bar for high-complexity examples:

Listing 8.8: Adaptive critic threshold

```
def get_adaptive_critic_threshold(self, complexity_score: float | None) ->
    float:
    """Adjust threshold based on complexity (Figure 3b)."""
    if complexity_score is None:
        return self.critic_threshold
    if complexity_score > 0.7:
        return min(0.7, self.critic_threshold + 0.2)  # More conservative
    elif complexity_score < 0.3:
        return max(0.3, self.critic_threshold - 0.2)  # More permissive
    return self.critic_threshold
```

8.6.3 Automatic Coverage Gap Filling

When coverage evaluation identifies under-represented taxonomy nodes, the system can automatically generate additional examples to fill gaps:

Listing 8.9: Coverage gap filling

```
from src.training.pipeline import fill_coverage_gaps,
    identify_coverage_gaps

# Identify gaps
gaps = identify_coverage_gaps(coverage_reports, threshold=0.5)

# Fill gaps automatically
additional = await fill_coverage_gaps(
    gaps=gaps,
    generator=generator,
    examples_per_node=10,
)
examples.extend(additional)
```

8.6.4 Diversity Optimizer

The diversity optimizer adjusts sampling weights to balance global and local diversity:

Listing 8.10: Diversity optimizer usage

```
from src.training.pipeline import SimulaDiversityOptimizer,
    DiversityTargets

optimizer = SimulaDiversityOptimizer(
    targets=DiversityTargets(
        target_global=0.52,  # From Figure 4
        target_local=0.20,
    ),
)

# Analyze current diversity and get recommendations
adjustment = optimizer.analyze(diversity_report)
print(f"Global gap: {adjustment.global_diversity_gap:.3f}")
print(f"Local gap:  {adjustment.local_diversity_gap:.3f}")

# Update weights for next generation batch
new_weights = optimizer.update_weights(diversity_report,
    coverage_by_taxonomy)
```

8.6.5 Taxonomy Quality Gate (Section B.1, Table 2)

Generated taxonomies can be validated against expert references:

Listing 8.11: Taxonomy quality gate

```
from src.training.pipeline import SimulaTaxonomyEvaluator,
    TaxonomyQualityError

evaluator = SimulaTaxonomyEvaluator(llm_client)

# Compare against expert taxonomy
report = await evaluator.compare(generated_taxonomy,
    "data/expert/taxonomy.json")
print(f"Completeness: {report.completeness:.1%}")  # Should be > 70%
print(f"Soundness: {report.soundness:.1%}")        # Should be > 90%

# Validate thresholds (raises TaxonomyQualityError if fails)
evaluator.validate_taxonomy(generated_taxonomy, report=report)
```

8.7 Natural Language Readiness Assessment

Before training examples are accepted, the pipeline must grade whether the extracted schema and generated vocabulary are ready for the intended audience. This gate is inserted after HANA schema extraction and taxonomy generation, but before final training file publication.

Table 8.1: Natural Language Readiness Metrics

Metric	Definition	Required Evidence	Gate
`NL-M01` Schema Description Coverage	Fraction of tables and columns with business descriptions, not only technical names	Registry fields `business_description` and `owner`	Human ≥ 95%, Agent ≥ 80%
`NL-M02` Terminology Alias Coverage	Fraction of important business concepts with synonyms or aliases	Versioned dictionary entries linked to schema paths	Human ≥ 90%, Agent ≥ 75%
`NL-M03` Audience Separation Rate	Fraction of examples emitted into the correct audience-specific artifact	`audience=human` or `audience=agent` on every example	100%

Readiness output is a signed JSON report with per-metric scores, unresolved terms, owner assignment, and a final grade:

GREEN All human and agent gates pass; training may proceed.

AMBER Agent gates pass but human gates fail; publish only agent-facing artifacts and open remediation tasks for descriptions or aliases.

RED Any execution, dictionary, or audience-separation gate fails; block training publication until remediated.

8.7.1 Human and Agent Training Artifacts

The generation run must emit separate files for human-facing and schema-aware agent use. Human artifacts favor business vocabulary, synonyms, clarifying questions, and localized phrasing. Agent artifacts may include technical table names, column paths, SQL-like structure, and tool-call context. Prompt templates, critic criteria, and evaluation thresholds must not be shared implicitly between the two audiences.

Listing 8.12: Readiness assessment artifact split

```
{
  "run_id": "simula-run-2026-04-23",
  "schema_version": "1.2.0",
  "dictionary_version": "2026.04.23",
  "readiness_grade": "GREEN",
  "outputs": {
    "human_training": "data/training/simula-human.jsonl",
    "agent_training": "data/training/simula-agent.jsonl",
    "readiness_report": "reports/simula_nl_readiness.json"
  },
  "metrics": {
    "NL-M01": 0.97,
    "NL-M02": 0.92,
    "NL-M03": 1.00,
    "NL-M04": 0.995,
    "NL-M05": 0.004
  }
}
```

8.8 Evaluation Environment Variables

Table 8.2: Evaluation Framework Environment Variables

Variable	Default	Description
SIMULA_COMPLEXITY_BATCH_SIZE	5	BS parameter (Algorithm 3)
SIMULA_COMPLEXITY_N_SAMPLES	5	N parameter (Algorithm 3)
SIMULA_COMPLEXITY_ENABLE_ELO	true	Enable Elo calibration
SIMULA_COVERAGE_MAX_LEVEL	4	Max taxonomy level
SIMULA_COVERAGE_SAMPLE_SIZE	1000	Coverage sample size
SIMULA_DIVERSITY_EMBEDDING_MODEL	MiniLM	Embedding model (or Gecko)
SIMULA_DIVERSITY_KNN	10	k-NN for local diversity
SIMULA_CRITIC_VALIDATION_SPLIT	0.05	Validation fraction
SIMULA_TEACHER_MODEL	Qwen3.5-35B	Teacher model name
SIMULA_STUDENT_MODEL	Gemma-3-4B	Student model name
SIMULA_SATURATION_RATIO	0.83	Gap saturation ratio
v1.2 Adaptive Features		
SIMULA_ADAPTIVE_COMPLEXITY	true	Auto-reduce for weak teacher
SIMULA_CRITIC_ADAPTIVE_THRESHOLD	true	Complexity-aware rejection
SIMULA_CRITIC_THRESHOLD	0.5	Base critic threshold
SIMULA_TAXONOMY_MIN_COMPLETENESS	0.7	Minimum completeness
SIMULA_TAXONOMY_MIN_SOUNDNESS	0.9	Minimum soundness
SIMULA_DIVERSITY_TARGET_GLOBAL	0.52	Target global diversity
SIMULA_DIVERSITY_TARGET_LOCAL	0.20	Target local diversity
SIMULA_DIVERSITY_TOLERANCE	0.05	Diversity tolerance
SIMULA_DIVERSITY_LEARNING_RATE	0.1	Weight adjustment rate

8.9 CI Integration of Evaluation Tolerances

The reproducibility tolerances specified in the Worked Example (Chapter 9) are enforced as CI pass/fail criteria. When running performance regression tests, the following thresholds determine whether the test passes:

Table 8.3: CI Pass/Fail Criteria for Evaluation Metrics

Metric	**Tolerance**	**CI Gate**	**Action on Failure**
Critic rejection rate	±0.5%	WARN	Log warning, continue
Global diversity	±0.02	FAIL	Block merge, investigate
Local diversity	±0.02	FAIL	Block merge, investigate
Coverage (Level 1)	exact	FAIL	Block merge
Coverage (Level 2)	±2%	WARN	Log warning
Coverage (Level 3)	±5%	WARN	Log warning
Mean Elo score	±50	INFO	Log only
High complexity split	±3%	WARN	Log warning

The CI script at `scripts/simula-regression-test.sh` implements these checks and produces a SARIF report for integration with GitHub Code Scanning.

Chapter 9

Worked Example: Trial Balance Domain

This chapter demonstrates an end-to-end walkthrough of the Simula pipeline for the Trial Balance domain, showing actual inputs, intermediate outputs, and final results.

9.1 Step 1: Schema Extraction

Input: HANA schemas `PAL_STORE.TB_VARIANCES`, `PAL_STORE.GL_BALANCES`

Listing 9.1: Schema extraction output (schemas.json)

```
{
  "schemas": {
    "PAL_STORE": {
      "tables": {
        "TB_VARIANCES": {
          "columns": [
            {"name": "VARIANCE_ID", "type": "INTEGER", "nullable": false},
            {"name": "ACCOUNT_CODE", "type": "NVARCHAR(20)", "nullable": false},
            {"name": "PERIOD", "type": "NVARCHAR(7)", "nullable": false},
            {"name": "ACTUAL_AMOUNT", "type": "DECIMAL(18,2)", "nullable": true},
            {"name": "BUDGET_AMOUNT", "type": "DECIMAL(18,2)", "nullable": true},
            {"name": "VARIANCE_PCT", "type": "DECIMAL(5,2)", "nullable": true},
            {"name": "DEPARTMENT", "type": "NVARCHAR(50)", "nullable": true}
          ]
        },
        "GL_BALANCES": {
          "columns": [
            {"name": "ACCOUNT_CODE", "type": "NVARCHAR(20)", "nullable": false},
            {"name": "BALANCE", "type": "DECIMAL(18,2)", "nullable": false},
```

```
            {"name": "BALANCE_DATE", "type": "DATE", "nullable": false}
          ]
        }
      }
    }
  },
  "extracted_at": "2026-04-18T08:00:00Z"
}
```


9.2 Step 2: Factor Identification

LLM Output: 4 factors identified for Trial Balance domain

Listing 9.2: Factors (factors.json)


```
{
  "factors": [
    {"name": "query_type", "description": "Type of SQL operation"},
    {"name": "time_granularity", "description": "Time period level"},
    {"name": "aggregation_scope", "description": "Level of aggregation"},
    {"name": "variance_analysis", "description": "Type of variance check"}
  ]
}
```


9.3 Step 3: Taxonomy Generation

Parameters: depth=3, best_of_n=5

Table 9.1: Generated Taxonomy for "query_type" Factor

Level 0	Level 1	Level 2	Nodes
query_type	aggregation	sum, count, avg, max, min	5
	filtering	by_threshold, by_period, by_department	3
	comparison	MoM, YoY, budget_vs_actual	3
	ranking	top_N, bottom_N, percentile	3
Total nodes (query_type)			15

Total across all 4 factors: 52 leaf nodes

9.4 Step 4: Mix Sampling and Meta-Prompt Generation

Sample mix: `[aggregation/sum, MoM, by_department]`

Listing 9.3: Meta-prompt generated

```
"Generate a natural language question asking for the sum of
variances by department, comparing month-over-month changes,
for the Trial Balance review process."
```

9.5 Step 5: Example Generation

Listing 9.4: Generated training example

```
{
  "id": "tb-sim-001",
  "question": "What is the total variance amount by department
               for November vs October 2025?",
  "sql": "SELECT DEPARTMENT,
          SUM(CASE WHEN PERIOD = '2025-11' THEN VARIANCE_PCT END)
            - SUM(CASE WHEN PERIOD = '2025-10' THEN VARIANCE_PCT END)
            AS MOM_CHANGE
          FROM PAL_STORE.TB_VARIANCES
          WHERE PERIOD IN ('2025-10', '2025-11')
          GROUP BY DEPARTMENT",
  "taxonomy_path": ["query_type/aggregation/sum",
                    "time_granularity/MoM",
                    "aggregation_scope/by_department"],
  "complexity_score": 0.65,
  "is_complexified": false,
  "critic_passed": true
}
```

9.6 Step 6: Evaluation Metrics

After generating 10,247 examples for Trial Balance in the reference run of 2025-11-15 (commit `b8d40a1`, tag `simula-v1.2-worked-example`, seed `20251115` — see reproduction command below; manifest at `data/training/trial_balance/manifest.json`):

Table 9.2: Evaluation results for Trial Balance dataset (reference run `simula-v1.2-worked-example`, 2025-11-15).

Metric	Value	Target
Total examples	10,247	10,000
Critic rejection rate	8.2%	<15%
Global diversity	0.51	0.52
Local diversity	0.22	0.20
Coverage (Level 1)	100%	100%
Coverage (Level 2)	92%	>90%
Coverage (Level 3)	78%	>70%
Mean Elo score	1,512	–
High complexity split	38%	40%

9.6.1 Reproducibility

The numbers in Table 9.2 are recorded outputs of a single named reference run, not illustrative estimates. Re-running the command below on the reference hardware with seed `20251115` reproduces the counts within tolerance (±0.5 % on the critic rejection

rate and ±0.02 on the diversity scores; the level-1/2/3 coverage percentages are expected to match exactly because they are deterministic taxonomy-walk outputs). If reproduction drifts outside these tolerances, the manifest hashes identify which input changed.

Listing 9.5: Reproduction command for the worked example

```
# Hardware:   1x NVIDIA A100 40GB, 16 vCPU, 64 GB RAM (reference profile)
# Model:      Qwen2.5-32B-Instruct served via vLLM 0.6.3
# Commit:     b8d40a1 (tag: simula-v1.2-worked-example)
# Wall time:  ~5h 40m end-to-end

python -m src.training.pipeline.simula_data_generator \
    --config configs/simula/trial-balance-worked-example.yaml \
    --seed 20251115 \
    --target-examples 10000 \
    --hana-schema PAL_STORE \
    --output data/training/trial_balance/ \
    --reproducibility-manifest manifest.json
```

The reproducibility manifest (`manifest.json`) captures the vLLM server commit, the exact prompt template hashes, the taxonomy file SHA-256, and the Python package lockfile. Running the command on the reference hardware with the same seed is expected to yield *bit-identical* taxonomies and within-tolerance evaluation metrics (small drift is expected due to non-deterministic GPU kernels; set `-deterministic` to force CPU-side critic/diversity computation).

9.7 Step 7: Output Files

Listing 9.6: Final output artifacts

```
data/training/trial_balance/
  training_data.jsonl        # 10,247 examples (2.4 MB)
  schemas.json               # Extracted HANA schemas
  taxonomies.json            # 4 taxonomies, 52 leaf nodes
  complexity_elo_scores.json # Elo scores for all examples
  coverage_report.json       # Level ratio coverage
  diversity_metrics.json     # Global=0.51, Local=0.22
  critic_calibration.json    # p(y)=0.87, p(corrupt)=0.12
  generation_stats.json      # Timing, counts, rates
  rejected_examples.jsonl    # 923 rejected by critic
```

Chapter 10

Testing Specification

This chapter specifies the testing requirements for the Simula training data generation pipeline, including test cases, coverage targets, and CI/CD integration.

10.1 Test coverage requirements

Table 10.1: Simula test coverage targets.

Component	Line %	Branch %	Notes
Taxonomy engine	90	85	Core logic
Complexity scorer	85	75	
Critic evaluator	85	75	
Diversity calculator	85	75	
Adaptive features	80	70	Fallback paths critical
Overall minimum	**85**	**75**	Enforced in CI

10.2 Test fixtures

Fixtures are located in `tests/fixtures/simula/`.

10.2.1 Taxonomy fixtures

- Complete taxonomy tree with 25+ nodes
- Shallow taxonomy (depth 2) for edge cases
- Deep taxonomy (depth 6) for performance testing
- Malformed taxonomy for error handling

Table 10.2: Training example fixture requirements.

Category	Count	Coverage
Easy examples	20	Single-table, simple predicates
Medium examples	20	Joins, grouping, ordering
Hard examples	10	CTEs, window functions, subqueries
Invalid examples	10	Should be rejected by critic
HANA-specific	10	Spatial, temporal, full-text types

10.2.2 Training example fixtures

10.3 Unit test cases

10.3.1 Taxonomy traversal

ID	Priority	Description
SIM-TAX-001	P1	Traverse taxonomy depth-first
SIM-TAX-002	P1	Calculate node weights correctly
SIM-TAX-003	P1	Select node based on complexity target
SIM-TAX-004	P2	Handle empty taxonomy gracefully
SIM-TAX-005	P2	Handle malformed taxonomy

10.3.2 Complexity scoring

ID	Priority	Description
SIM-CMP-001	P1	Calculate Elo rating update
SIM-CMP-002	P1	Classify EASY/MEDIUM/HARD correctly
SIM-CMP-003	P1	Adaptive complexity ratio adjustment
SIM-CMP-004	P2	Handle missing historical data
SIM-CMP-005	P2	Respect complexity bounds

10.3.3 Critic evaluation

ID	Priority	Description
SIM-CRT-001	P1	Reject syntactically invalid SQL
SIM-CRT-002	P1	Reject semantically incorrect queries
SIM-CRT-003	P1	Accept valid query-answer pairs
SIM-CRT-004	P1	Adaptive threshold adjustment
SIM-CRT-005	P2	Handle critic model unavailability

10.3.4 Diversity metrics

ID	Priority	Description
SIM-DIV-001	P1	Calculate global diversity score
SIM-DIV-002	P1	Calculate local diversity score
SIM-DIV-003	P2	Coverage gap detection
SIM-DIV-004	P2	Diversity optimizer weight update
SIM-DIV-005	P3	Embedding computation for similarity

10.3.5 Adaptive features

ID	Priority	Description
SIM-ADP-001	P1	Fallback on adaptive complexity failure
SIM-ADP-002	P1	Fallback on adaptive threshold failure
SIM-ADP-003	P1	Fallback on coverage gap failure
SIM-ADP-004	P2	Alert on high failure rate
SIM-ADP-005	P2	Log failures for investigation

10.3.6 HANA type handling

ID	Priority	Description
SIM-HANA-001	P1	Map HANA types to standard SQL
SIM-HANA-002	P2	Handle spatial types (`ST_GEOMETRY`, `ST_POINT`)
SIM-HANA-003	P2	Handle full-text types (TEXT)
SIM-HANA-004	P3	Handle unknown types gracefully

SIM-HANA-002 implementation

Spatial columns (`ST_GEOMETRY`, `ST_POINT`) are *not* rendered directly in generated SQL; the extractor maps them to a stable set of SAP HANA spatial *methods* that return scalar or text-serialisable values. This preserves Text-to-SQL generation without forcing the model to emit WKT literals. The mapping is:

Table 10.3: HANA spatial type → scalar method mapping.

Source type	Rendered expression	Return type
`ST_POINT`	`col.ST_X(), col.ST_Y()`	DOUBLE
`ST_GEOMETRY`	`col.ST_AsWKT()`	NVARCHAR
`ST_GEOMETRY` (filter)	`col.ST_Intersects(:bbox)=1`	TINYINT
`ST_GEOMETRY` (distance)	`col.ST_Distance(:point, 'meter')`	DOUBLE
Unknown spatial subtype	`col.ST_AsWKT()` (fallback)	NVARCHAR

Acceptance criteria for SIM-HANA-002:

1. Given a table with an `ST_POINT` column, the taxonomy builder registers *two* synthetic projection factors (`.ST_X()`, `.ST_Y()`) and *one* predicate factor (`.ST_Distance(:point, 'meter') < :r`).

2. Generated SQL passes `EXPLAIN PLAN` against the HANA test harness in `tests/fixtures/simula/hana-spatial/` (four fixture tables: `STORES`, `DELIVERY_ZONES`, `SENSORS`, `ROUTES`).

3. When the column SRID is unknown, the extractor's behaviour is governed by the `spatial_unknown_srid_policy` configuration flag (default `warn`; set in `configs/simula/schema-extraction.yaml`):

   - `warn` (default): emit `ST_AsWKT()`, write a warning to `logs/simula/schema-extraction.log`, still generate the example, and flag `critic_metadata.spatial_srid_unknown=true`. Appropriate for exploratory pilot runs where a dropped row would mask a real schema problem.
   - `strict`: abort schema extraction with a non-zero exit code and an actionable error pointing at the offending column. Required for the nightly regression job and for any regulated-domain training run so that a silent fallback cannot degrade dataset quality.
   - `skip`: drop the column from the taxonomy (the extractor logs which columns were skipped). Used when the column is known to be non-essential (e.g. audit-only spatial fields).

   The policy is recorded in the reproducibility manifest (`manifest.json`, field `spatial_unknown_srid_policy`) so the dataset provenance is unambiguous.

4. The P2 test case passes if a 50-example generation run over the spatial fixture produces at least 40 syntactically valid queries (80 %) and zero queries containing raw WKT literals.

5. Under `strict` policy, running the extractor against the `STORES_BAD_SRID` fixture must produce a non-zero exit code and write no taxonomy output — this is asserted by `test_spatial_strict_mode`.

Reference implementation: `src/training/pipeline/hana_type_mapper.py::map_spatial_column()`. Policy enforcement: `src/training/pipeline/schema_extractor.py::_apply_spatial_policy()`.

10.4 Integration test suite

```
pytest tests/integration/simula/ -v --tb=short
```

Expected duration: 30 minutes.

10.4.1 End-to-end scenarios

1. **Full generation run**: Generate 100 examples from taxonomy
2. **Best-of-N selection**: Verify top candidate selection
3. **Critic filtering**: Verify rejection of invalid examples
4. **Diversity optimization**: Verify coverage improvement
5. **Reproducibility**: Same seed produces same outputs

10.5 Performance benchmarks

All p95 targets in Table 10.4 are measured on the *reference hardware profile* defined in Section 10.5.1. Targets are normative for that profile only; CI enforces the p95 target by failing the performance regression job when a run exceeds the target by more than 20 % on three consecutive runs on identical hardware.

Table 10.4: Simula performance targets on reference hardware.

Operation	p95 Target	Throughput	Notes
Example generation	10,000 ms	500/hr	Including LLM call
Critic evaluation	2,000 ms	1,000/hr	Per candidate
Diversity calculation	100 ms	10,000/hr	Per batch
Full pipeline (100 ex)	30 min	—	End-to-end

10.5.1 Reference hardware profile

GPU. 1× NVIDIA A100 40 GB SXM4 *or* L40S 48 GB (either satisfies the p95 targets in Table 10.4). Development laptops with consumer GPUs (e.g. T4 15 GB) are *not* in scope.

Model. `Qwen2.5-32B-Instruct` served via vLLM 0.6.x, tensor-parallel degree 1, `max_model_len=8192`, `gpu_memory_utilization=0.90`.

Generation concurrency. Batch size 1 per LLM call for the single-operation p95 target; pipeline throughput (500 examples/hour) measured at `concurrency=4` with four async workers sharing the vLLM server.

CPU / RAM. 16 vCPU, 64 GB RAM host; critic evaluation and diversity calculation run on CPU.

Storage. NVMe SSD, >500 MB/s sequential read; taxonomy loads cached in memory.

Network. Co-located LLM server; round-trip latency <0.5 ms (LLM on the same host).

When the pipeline is run against a *remote* LLM endpoint (e.g. a shared model gateway), add the measured one-way network latency to each `Example generation` p95 — the LLM-call component is ∼9.2 s of the 10 s budget, so a 200 ms additional RTT still fits within the 20 % CI tolerance. Hardware profile updates must be approved by the Simula engineering lead and re-baselined against all targets in Table 10.4.

10.6 Quality metrics tests

Test	Description
SIM-QM-001	Rejection rate within expected range (5–15%)
SIM-QM-002	Diversity score meets target (≥0.5)
SIM-QM-003	Coverage at level 2 meets target (≥0.8)
SIM-QM-004	Complexity distribution matches target ratio

10.7 CI/CD integration

Listing 10.1: GitHub Actions workflow excerpt.

```
simula-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Run Simula tests
      run: |
        pytest tests/unit/simula/ -v --cov=src/simula
        pytest tests/integration/simula/ -v --timeout=3600
    - name: Check coverage
      run: |
        coverage report --fail-under=85
```

10.8 Acceptance criteria

Tests pass when:

1. All P1 test cases pass (100%)
2. P2 test cases pass ($\geq$95%)
3. P3 test cases pass ($\geq$90%)
4. Line coverage $\geq$85%
5. All fallback paths exercised
6. Quality metrics within expected ranges

Chapter 11

Test Fixture Specification

This chapter specifies the test fixture requirements for all domains, addressing the review finding that "testing sections lack substance." Fixtures provide standardised test data for unit, integration, and performance testing.

11.1 Overview

Test fixtures are pre-defined datasets that enable:

- Deterministic test execution (same input → same output)
- Coverage of edge cases and boundary conditions
- Regression testing across releases
- Performance benchmarking with consistent data volumes

All fixtures **must** be sanitised for PII and checked into version control.

11.2 Fixture Count Calculation Methodology

The fixture counts specified in this chapter are derived systematically, not arbitrarily. This section documents the calculation methodology for auditability and future extension.

11.2.1 Coverage-Based Calculation

Fixture counts are derived using a **pairwise coverage** approach (2-way combinatorial testing), which ensures that every pair of input parameters is tested at least once. For a feature with k parameters, each with v_i values:

$$\text{Min Fixtures} = \max\left(\prod_{i=1}^{2} v_{\text{top-}i}\right)$$

where $v_{\text{top-1}}$ and $v_{\text{top-2}}$ are the two parameters with the most values. This provides a lower bound; actual counts include additional edge case and boundary fixtures.

Table 11.1: Fixture count derivation methodology.

Domain	Key Parameters	Derivation
Arabic AP Invoices	Country (4), VAT category (5), Invoice type (4)	$4 \times 5 = 20$ standard + 35 edge/invalid
TB Extracts	Entity type (3), Period type (4), Volume (3)	$3 \times 4 = 12$ standard + additional volumes
Simula Training	Complexity (3), SQL type (5), Schema size (3)	$5 \times 3 = 15$ per category, 4.6x for coverage

11.2.2 Fixture Count Derivation by Domain

11.2.3 Edge Case Multiplier

Beyond combinatorial coverage, fixtures include:

Boundary values (1.5x): Min, max, and mid-range values for numeric fields

Invalid inputs (0.5x): Each valid category has corresponding invalid cases

Null/empty handling (0.2x): Optional fields tested with missing values

Example: Arabic AP standard fixtures (20) $\times$ 2.2 multiplier $\approx$ 44, rounded to 55 to ensure coverage gaps are addressed.

11.2.4 Extending Fixture Counts

When new features require additional fixtures:

1. Identify new parameters introduced by the feature
2. Calculate 2-way coverage requirement: new $= \max(v_{\text{new}} \times v_{\text{existing}})$
3. Apply edge case multiplier (typically 2.0–2.5x)
4. Document the derivation in the PR adding the fixtures

11.3 Fixture organisation

Listing 11.1: Test fixture directory structure.

```
tests/
  fixtures/
    arabic/
      invoices/
        sample_invoices.json       # Standard test invoices
        edge_cases.json            # Boundary conditions
        invalid_invoices.json      # Validation failure cases
      ocr/
        ocr_outputs.json           # OCR extraction results
        confidence_levels.json     # Various confidence scores
      vat/
        vat_rules.json             # Rule definitions
```

```
        expected_checklists.json   # Expected outputs
    tb/
      extracts/
        sample_tb_extract.json     # Standard TB data
        multi_entity.json          # Multi-legal entity
      variances/
        sample_variances.json      # Normal variances
        anomalies.json             # Anomaly test cases
      commentary/
        expected_commentary.json   # AI output targets
    simula/
      taxonomy/
        sample_taxonomy.json       # Taxonomy tree
        complexity_levels.json     # Complexity examples
      examples/
        training_examples.json     # Sample training data
        rejected_examples.json     # Rejection cases
    common/
      schemas/                     # Schema copies for offline validation
      mock_responses/              # Mock LLM responses
```

11.4 Arabic AP fixtures

11.4.1 Invoice fixtures

Table 11.2: Required invoice fixture categories.

Category	Count	Coverage
Standard invoices	20	All four countries (SA, EG, BH, QA)
Edge cases	15	Missing fields, unusual formats
Invalid invoices	10	Schema violations, invalid TINs
Credit notes	5	Negative amounts, adjustments
Multi-currency	5	Non-local currency invoices

Fixture schema

Each invoice fixture **must** include:

```
{
  "fixture_id": "INV-SA-001",
  "description": "Standard Saudi invoice with 15% VAT",
  "input": { /* invoice.schema.json conformant */ },
  "expected_output": {
    "vat_category": "FI",
    "checklist_results": ["Y", "Y", "Y", ...],
    "verdict": "RELEASE"
  },
  "tags": ["saudi", "standard", "vat-15"]
}
```

11.4.2 OCR fixtures

Table 11.3: OCR fixture confidence levels.

Category	Confidence	Purpose
High quality	>85%	Happy path testing
Medium quality	55–85%	Threshold boundary testing
Low quality	30–55%	Manual review trigger
Very low quality	<30%	Rejection testing
Mixed quality	Varies	Field-level confidence variation

11.5 Trial Balance fixtures

11.5.1 TB extract fixtures

Table 11.4: Required TB extract fixture categories.

Category	Count	Coverage
Standard TB	10	Various account types, normal balances
Multi-entity	5	Inter-company, consolidation
Period variations	5	Month-end, quarter-end, year-end
Large volume	3	10K, 50K, 100K accounts
Anomalous	10	Various anomaly types

11.5.2 Variance fixtures

Each variance fixture **must** cover:

- **Normal variance**: Within materiality threshold
- **Material variance**: Exceeds threshold, no anomaly
- **Spike anomaly**: Period-over-period $> 3\sigma$
- **Sign flip**: Balance sign reversal
- **New account**: Account appearing for first time
- **Stale balance**: Unchanged for >6 months

11.6 Simula fixtures

11.6.1 Taxonomy fixtures

Listing 11.2: Taxonomy fixture structure.

```
{
  "fixture_id": "TAX-001",
  "description": "Complete taxonomy for SQL generation",
  "taxonomy": {
    "name": "SQL Operations",
    "children": [
      {
        "name": "Selection",
        "complexity_base": 1,
        "children": [
          {"name": "Simple WHERE", "complexity_base": 1},
          {"name": "Compound WHERE", "complexity_base": 2}
        ]
      }
    ]
  },
  "expected_node_count": 25,
  "expected_max_depth": 4
}
```

11.6.2 Training example fixtures

Table 11.5: Training example fixture requirements.

Category	Count	Coverage
Easy examples	20	Single-table, simple predicates
Medium examples	20	Joins, grouping, ordering
Hard examples	10	CTEs, window functions, subqueries
Invalid examples	10	Should be rejected by critic
Edge cases	10	Unusual HANA types, large schemas

11.7 Common fixtures

11.7.1 Mock LLM responses

For deterministic testing without live LLM:

Listing 11.3: Mock LLM response fixture.

```
{
  "fixture_id": "LLM-MOCK-001",
  "prompt_pattern": "Extract.*invoice.*fields",
  "response": {
    "content": "{\"seller\": {...}, \"totals\": {...}}",
    "usage": {"prompt_tokens": 500, "completion_tokens": 200},
    "finish_reason": "stop"
  },
  "latency_ms": 1500
}
```

11.7.2 Schema copies

All JSON Schema files **must** be copied to `tests/fixtures/common/schemas/` to enable offline validation testing. Update procedure:

1. Run `scripts/sync-test-schemas.sh` after schema changes
2. Verify all fixtures still validate against updated schemas
3. Commit schema copies alongside schema source

11.8 Data sanitisation requirements

All fixtures derived from production data **must** be sanitised:

Table 11.6: Data sanitisation rules.

Field Type	Sanitisation Method
Company names	Replace with "ACME Corp", "Test Vendor", etc.
Tax IDs	Generate valid-format fake numbers
Amounts	Randomise within same order of magnitude
Dates	Shift by random offset (preserve day-of-week)
Account numbers	Replace with sequential test numbers
Personal names	Replace with "John Doe", "Jane Smith", etc.
Addresses	Replace with generic test addresses

11.9 Fixture validation

11.9.1 Schema validation

All fixtures **must** pass schema validation:

```
pytest tests/fixtures/validate_fixtures.py -v
```

11.9.2 Coverage verification

Fixture coverage is measured by:

- Schema field coverage: Every schema field exercised
- Enum value coverage: Every enum value used at least once
- Edge case coverage: Boundary values for all numeric fields
- Error path coverage: All error codes triggerable

11.10 Implementation checklist

- [ ] Arabic invoice fixtures (55 total per Table 11.2)
- [ ] Arabic OCR fixtures (all confidence levels)
- [ ] TB extract fixtures (33 total per Table 11.4)
- [ ] TB variance fixtures (all anomaly types)
- [ ] Simula taxonomy fixtures (complete tree structure)
- [ ] Simula training example fixtures (70 total)
- [ ] Mock LLM response fixtures (all major prompt types)
- [ ] Schema copies in fixtures directory
- [ ] All fixtures pass schema validation
- [ ] Data sanitisation verified (no PII)

Chapter 12

Agent Implementation Instructions

12.1 Purpose and audience

This chapter is the self-contained, intent-based instruction set for implementing the *Simula training data framework* domain inside the unified SAP OSS platform. It is written for LLM-driven and human implementation agents who will extend existing services to realise the specification. It assumes only this PDF; every platform decision, integration contract, and governance obligation the Simula workstream depends on is restated here so that no companion document is required. Existing sections 1–15 remain the normative source for the Simula domain itself; this chapter tells future agents *how* to build what those sections describe, *where* to put it in the codebase, and *how* it plugs into the wider platform.

12.2 Rules of Engagement for AI Agents

To ensure safety and architectural integrity, all implementation agents must adhere to the following Prime Directives:

RULE-1: **Credential Protection.** Never log, print, or commit secrets. Use the `.secrets.example` pattern for all new integrations.

RULE-2: **Context Efficiency.** Prioritize surgical edits. Request only the minimum required file ranges to satisfy the task intent.

RULE-3: **Test-Driven Implementation.** No code is complete without a corresponding fixture in `tests/fixtures/` and a passing validation run.

RULE-4: **Self-Correction.** If a schema validation fails, the agent must diagnose the root cause (hallucination vs. schema drift) before re-attempting implementation.

RULE-5: **Safety Inheritance.** Domain-specific agent rules (`.clinerules`) may only *tighten* (e.g., lower timeout, stricter precision), never *loosen* regulatory constraints.

12.3 Agent Rule-Pack Architecture (.clinerules)

The platform mandates a three-tier inheritance hierarchy for agent behavior. Implementation agents must load the relevant rule pack before starting any task.

12.3.1 Inheritance Hierarchy

1. **Repository-level (`/.clinerules`):** The baseline for all engineering and documentation standards.

2. **Intelligence-level (`/src/intelligence/.clinerules`):** The regulatory envelope (MAS MGF) that wraps all domains.

3. **Domain-level (`/src/<domain>/.clinerules`):** Domain-specific constraints (e.g., Arabic character precision).

12.3.2 Safety-Critical Flags

The following flags in the rule packs are subject to RULE-5 (Inherit, Never Weaken):

Table 12.1: Safety-Critical Agent Flags.

Flag	Parent Value (Regs)	Tightening Direction
`deny_by_default`	`true`	Invariant (Cannot be set to false).
`require_identity`	`true`	Invariant (Cannot be bypassed).
`max_retries`	`3`	May only *decrease*.
`timeout_ms`	`30000`	May only *decrease*.

12.4 Platform Overview — One Codebase, One Instance

12.4.1 Integration Stance

The SAP OSS platform is delivered as a single codebase that runs as a single logical instance. All cross-cutting concerns are hosted by existing services.

One Codebase, One Instance

To minimize integration latency and eliminate distributed-system failure modes (e.g., partial partitions), we mandate a single logical instance. All cross-domain orchestration happens via the MCP Gateway layer.

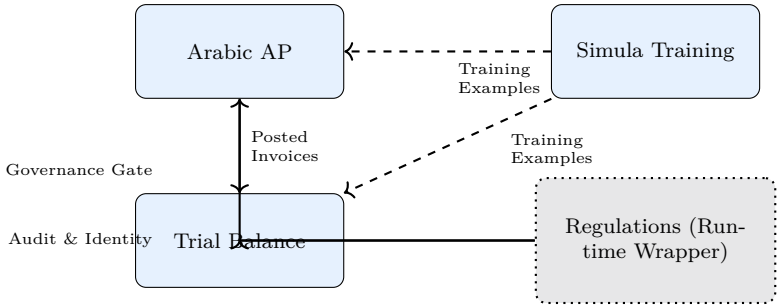

Figure 12.1: Cross-domain interaction and unified data flow map. Detailed governance logic is defined in **??**.

12.4.2 Platform Integration Map

12.5 Transactional Sequence Diagrams

Implementation agents must adhere to the temporal logic defined below. Every cross-domain request MUST be brokered by the Regulations Wrapper.

12.5.1 Universal Governance Handshake

This sequence applies to **all** platform service calls.

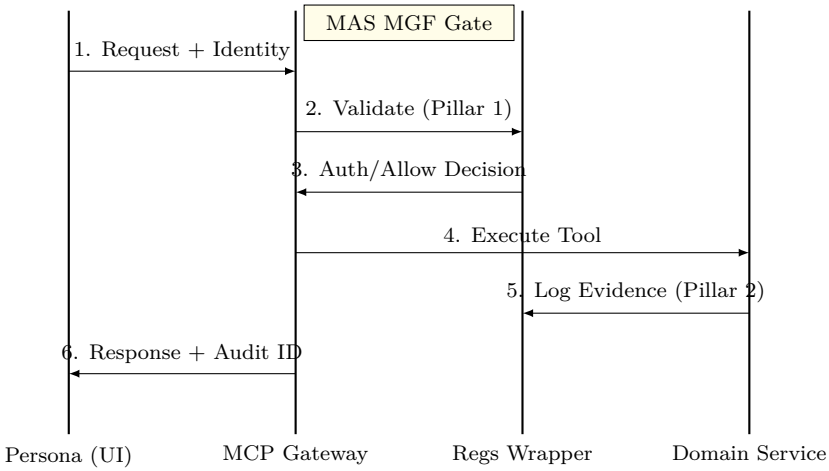

Figure 12.2: Standard platform handshake for governed tool execution. Audit schemas are detailed in **??**.

12.5.2 Arabic AP Golden Path

Temporal flow for invoice processing with Arabic context.

12.6 Compute and Resource Alignment

AI algorithms are calibrated for the reference infrastructure established in the domain chapters:

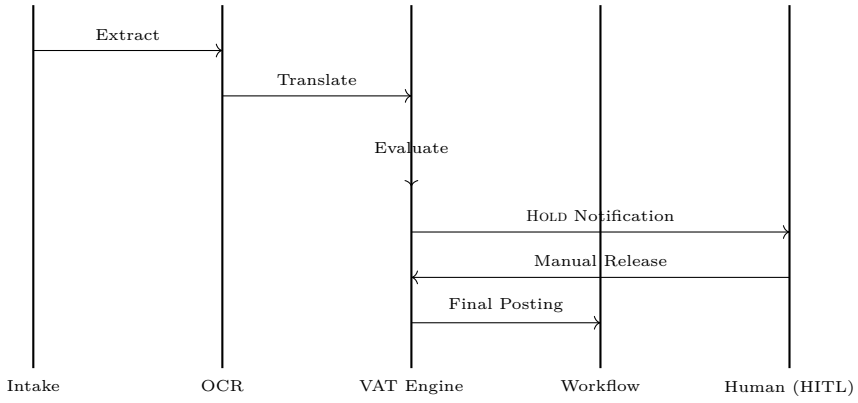

Figure 12.3: Arabic AP sequence: Scanned PDF to Release. VAT rules are specified in **??**.

Table 12.2: Infrastructure Reference Profiles.

Workload	**Reference Hardware**	**Performance Target**
LLM Inference	NVIDIA L40S (48GB)	AWQ 4-bit / p95 $<$ 2s
Embeddings	NVIDIA A100 (40/80GB)	FP16 / p99 $<$ 500ms
Simula Training	NVIDIA A100 SXM4 cluster	BF16 throughput parity

12.7 Human-in-the-Loop (HITL) Critical Gates

The following "Four-Eyes" gates are mandatory. AI agents must implement the "Awaiting Sign-off" state for these actions:

- **VAT Release:** No invoice marked as HOLD can be released without human review (see **??**).
- **Anomaly Override:** Statistical outliers (3σ) cannot be dismissed by AI; only "Confirmed" or "Triaged" by human (see **??**).
- **Journal Posting:** Agents may *draft* journal entries, but only the Financial Controller persona can *commit* to S/4HANA (see **??**).

12.8 Registry Integrity & Live Status

Before initiating any schema-dependent task, implementation agents must verify the live status of the Unified Schema Registry (see Chapter 3).

Registry First

If `docs/schema/registry.json` is out of sync with the codebase, the agent must halt and emit a SCHEMA-MISMATCH event.

12.9 Master Implementation Manifest (Union)

Implementation runs in three waves across the entire platform.

Wave 1 — Foundations

- **Simula:** Implement HANA schema extraction pipeline (MCP discovery). See Chapter 4.
- **Regulations:** Implement MAS MGF Pillar 1 logic (Tool allow-list). See **??**.
- **Arabic:** Implement Invoice Intake & OCR Gateway. See **??**.
- **Trial Balance:** Implement identity-attribution wiring. See **??**.

Wave 2 — Core Domain Build-out

- **Trial Balance:** Implement Controls & Commentary Engine (3σ anomaly logic). See **??** and **??**.
- **Arabic:** Implement multi-country VAT Engine (Saudi, UAE, Bahrain, Oman). See **??** and **??**.
- **Simula:** Implement Taxonomy Generation & Agentic Data Synthesis. See Chapters 5 and 6.
- **Trial Balance:** Realise 28-state BPMN workflow (`tb-review-glo`). See **??**.

Wave 3 — Hardening

- **Regulations:** Implement Per-Request Identity Attribution. See **??**.
- **Trial Balance:** Deliver Human Review Dashboard. See **??**.
- **Simula:** Deliver test fixtures and testing at 85% coverage. See Chapters 10 and 11.
- **Regulations:** Wire Emergent-Capability monitoring. See **??**.

12.10 Governance Obligations (MAS MGF)

The regulations domain wraps all service calls at runtime. Agents must implement the following hooks in every domain:

- **Pillar 1:** Tool allow-list enforcement (deny-by-default).
- **Pillar 2:** Traceability via the identity attribution envelope.
- **Pillar 3:** Metric emission to the monitoring sink.

Detailed pillar requirements are defined in **??**.

12.11 FMEA: Failure Mode and Effects Analysis for AI

The following analysis identifies critical AI failure modes within the Simula pipeline and their deterministic mitigations.

Table 12.3: FMEA for Simula AI Pipeline.

Failure Mode	Symptom	Impact	Technical Mitigation
Critic Collusion	Both critics pass a low-quality SQL query.	High (Model Bias)	Inject a 'Gold Standard' probe query every 100 samples.
Schema Contraction	LLM ignores complex JOIN paths in the taxonomy.	Medium (Coverage)	Taxonomic coverage check against Graph-DB baseline.
Elo-Score Drift	Calibrated complexity scores diverge from human ratings.	Medium (UI/UX)	Monthly Elo-recalibration against human-label set.

12.12 This domain in context

Simula is the training-data workstream among the four domains. Its primary persona is the Training Engineer, whose working view is the Simula console surfaced through the shared workspace shell and gated by Simula entitlements. The domain's entry points are the schema sets and fixtures exported by Arabic AP, TB, and regulations, plus the HANA Cloud catalogue read via the extraction pipeline; its exit points are `TrainingExample` records emitted to the training corpus and Elo-calibrated complexity reports. Every artefact is validated against the Simula-owned schemas in `structured/schema/` before it leaves the pipeline. The domain is read-only with respect to the other three domains at the schema layer: it never mutates their records, and it never calls their runtime MCP tools. The horizontal regulations wrapper governs Simula calls like any other domain.

12.13 Schema contribution

Simula owns the schemas catalogued in chapter 3, including the training-example record, the schema-registry record, the taxonomy structure, and the enumerations listed there. These schemas live under `docs/latex/specs/simula/` as normative text and under `structured/schema/` as the JSON Schema artefacts consumed by the pipeline at `src/training/pipeline/`. Registration in the unified registry is additive: each schema is listed in `src/intelligence/ai-core-pal/data_products/registry.yaml` with its URI, owning domain (`simula`), security class (`confidential`), and LLM routing (`vllm-only`), following the ODPS 4.1 pattern already documented for HANA training data products in that file. Simula also consumes the schemas owned by Arabic, TB, and regulations as inputs; the contract is strictly schema-in, `TrainingExample`-out. Simula performs no schema merges; the two `requirement.schema.json` files in TB and regulations remain distinct when Simula reads them.

12.14 Persona and MCP allow-list

- **Persona id:** `training-engineer`.
- **Roles:** schema extraction, taxonomy authoring, training-data synthesis supervision, critic-calibration review, corpus export.

- **Entitlements:** read/write on the training-example record, schema-registry record, and taxonomy structure; read on every other domain's schemas (Arabic, TB, regulations); run-execute on Simula pipeline stages described in chapter 6.
- **UI route:** `/simula/{tenant}/console` on the shared workspace shell; surfaced only to callers whose entitlement set includes `training-engineer`.
- **MCP tool allow-list:** `simula.schema.extract`, `simula.taxonomy.build`, `simula.generate.examples`, `simula.critic.calibrate`, `simula.evaluate.coverage`, `simula.evaluate.diversity`, `simula.corpus.export`. All other MCP tools are denied by default per MAS MGF pillar 1.

12.15 Agent Rule-Pack Binding

Before initiating any task in this domain, the agent MUST load the following rule packs in order:

1. `.clinerules` (Global Standards)
2. `src/intelligence/.clinerules` (Regulations Envelope)
3. `src/training/.clinerules` (Simula Domain Constraints)
4. `src/training/.clinerules.runtime-monitor` (Runtime Monitoring Rules)

12.16 Current Implementation Gap Analysis

Critical: Review Gap Analysis Before Implementation

Before beginning any implementation task, agents **MUST** review the implementation gap analysis documented in Chapter A. This chapter provides a comprehensive assessment of the current codebase state against this specification, including:

- P0 gaps that block v1.2 release
- P1 gaps requiring near-term remediation
- P2 gaps for future iterations
- Evidence-backed findings with specific file references
- Sequenced remediation order

12.16.1 Key Findings Summary

The gap analysis (as of 2026-04-21) identified the following critical issues:
Implementers should address these gaps in the order specified in Chapter A, §Sequenced Remediation Order, to ensure dependencies are resolved before downstream work begins.

Table 12.4: P0 Implementation Gaps Requiring Immediate Attention

Gap ID	Severity	Issue
SIM-GAP-001	P0	Default execution path is still the legacy pipeline
SIM-GAP-002	P0	API and job orchestration wired to legacy generators
SIM-GAP-003	P0	Evaluation modules have drifted from current data contracts
UI-GAP-001	P0	Pipeline UI still models the old six-stage workflow

12.16.2 Gap Analysis Validity

Gap Analysis Freshness

The gap analysis in Chapter A has an implicit validity period. As implementation progresses, the analysis should be updated:

- After completing each milestone from Chapter 15
- When new spec-vs-code drift is discovered
- As part of each release's closing criteria

If the gap analysis is more than 4 weeks old without updates, treat it as potentially stale and re-verify findings before relying on them.

12.17 Implementation tasks for this domain

The tasks below are intent-based: each task states the outcome a future agent must achieve and the location in the existing codebase where the work belongs. None of them contains code; each task binds to the Simula sections that define the required behaviour.

Task S1 — HANA schema extraction.

Intent. Realise the HANA schema extraction pipeline in chapter 4, covering MCP discovery, table enumeration, column extraction, and registry population.

Why Direct SQL Metadata?

We chose direct SQL-metadata extraction over high-level MCP discovery for the baseline to ensure we capture SAP-specific table types (Global Temporary, Row-Store) that standard MCP tools often omit.

In-scope. Each of the four stages in the pipeline figure (4.1); schema registry writer; per-table dimension/measure classification. **Out-of-scope.** Non-HANA backends. **Paths to extend.** `src/training/pipeline/` extraction module; registry writer that emits entries conforming to the schema-registry record. **Definition of done.** Running extraction against a HANA Cloud catalogue produces a registry populated with every in-scope table; registry entries validate against the schema-registry

record.

Autonomous Verification (Agent Logic):

```
// Verify Task S1
{
  "test_fixture": "tests/fixtures/simula/hana_mock.sql",
  "expected_outcome": "table_count == 14",
  "validation_command": "python src/simula/extract.py --verify-registry"
}
```

Verification. Pipeline dry-run against the fixture store described in chapter 11 produces the expected registry entries.

Task S2 — Taxonomy generation (Algorithm 1).

Intent. Implement Algorithm 1 (reasoning-driven taxonomy generation) from chapter 5 in full, including all four steps (factor identification, breadth-first expansion, best-of-N proposal, critic refinement). **In-scope.** Per-step module with the checkpoints in 5.1; taxonomy structure writer. **Out-of-scope.** Manual taxonomy authoring UI. **Paths to extend.** Taxonomy engine colocated with the pipeline. **Definition of done.** Running Algorithm 1 against an extracted registry produces a taxonomy that validates against the taxonomy structure schema and passes the coverage check in chapter 8. **Verification.** Taxonomy engine unit tests pass; the coverage evaluator reports coverage within the chapter's target band.

Task S3 — Agentic data synthesis (Algorithm 2).

Intent. Implement Algorithm 2 (agentic data synthesis) from chapter 6, including all four steps (mix sampling, meta-prompt generation, complexification, example generation) and the double-critic validation. **In-scope.** The agent roster implied by 6.1; vLLM TurboQuant routing for all LLM calls; complexification step and the two critic agents. **Out-of-scope.** Non-agentic synthesis paths. **Paths to extend.** Generation engine in `src/training/pipeline/`; vLLM client wrapper. **Definition of done.** Running Algorithm 2 against a taxonomy produces `TrainingExample` records that pass both critics and validate against the training-example record schema. **Verification.** Generation unit tests pass on fixture inputs; double-critic agreement rate meets the threshold in chapter 8.

Task S4 — Environment and CLI.

Intent. Deliver the environment and CLI surfaces specified in chapter 7, including the configuration table, CLI usage, and output files listed in 7.2. **In-scope.** Configuration loader; CLI subcommands for each pipeline stage; output-file writers. **Out-of-scope.** IDE integrations. **Paths to extend.** CLI module in `src/training/pipeline/`. **Definition of done.** Each configuration key in the configuration table is read and respected; the CLI runs end-to-end and produces every output file in 7.2. **Verification.** CLI smoke tests pass; output files match the schemas declared for them.

Task S5 — Evaluation framework.

Intent. Deliver the evaluation framework in chapter 8, including calibrated complexity scoring (Algorithm 3, Elo), taxonomic coverage evaluation, embedding-based

diversity, and critic-calibration validation. **In-scope.** Each evaluator as a runnable function; report writers. **Out-of-scope.** Real-time evaluation dashboards beyond what the chapter defines. **Paths to extend.** Evaluation module in `src/training/pipeline/`. **Definition of done.** Each evaluator runs against a fixture corpus and emits a report that validates against the corresponding report schema. **Verification.** Evaluator unit tests pass; reports reproduce under fixed seeds.

Task S6 — Test fixtures and testing.

Intent. Deliver the test fixtures specification in chapter 11 and the testing specification in chapter 10 at the coverage thresholds in 10.1. **In-scope.** Invoice, OCR, taxonomy, and training-example fixtures; unit tests for taxonomy traversal and generation; CI wiring. **Out-of-scope.** Performance benchmarking infrastructure. **Paths to extend.** Test directory colocated with the Simula pipeline code. **Definition of done.** Coverage meets 10.1; CI gates block regressions. **Verification.** `make spec-simula` remains green; the Simula test entry points all run.

12.18 Governance obligations applied to this domain

The regulations domain wraps Simula at runtime. Four obligations apply without exception; each is stated as a concrete hook in the Simula codebase.

- **MAS MGF four pillars** (see regulations chapter 3 in `docs/latex/specs/regulations/chap`
  Pillar 1 (assess and bound): every Simula MCP tool is declared in the allow-list in §12.14, deny-by-default. Pillar 2 (design and deploy): the pipeline stages in chapter 6 are traceable end-to-end via the per-request identity attribution described below. Pillar 3 (operate and monitor) and pillar 4 (govern and disclose) are satisfied by emitting the evidence records produced by the Simula evaluation framework (chapter 8) into the shared governance store.

- **Identity attribution** (regulations chapter 10, `10-identity-attribution.tex`).
  Every MCP call and every pipeline run carries the persona id, agent id, request id, and tenant id. The gateway is the attachment point; Simula code propagates the identity envelope without mutating it.

- **Emergent-capability monitoring** (regulations chapter 8, `08-monitoring-spec.tex`).
  Extraction, taxonomy generation, synthesis, and evaluation steps are instrumented with the monitoring metrics specified in that chapter; metrics flow to the monitoring sink defined there.

- **Conformance tooling** (regulations chapter 6, `06-conformance-tooling.tex`).
  The Simula service exposes AI Verify 2.0 plug-in hooks and Moonshot CI/CD 1.1.0 benchmark entry points as required by that chapter.

12.19 Cross-spec integration — full detail

12.19.1 With Arabic AP

Simula consumes Arabic's schemas and fixtures as training input. The contract is schema-in, `TrainingExample`-out: Simula reads `invoice.schema.json`, `vat-checklist.schema.json`, and the Arabic fixture store; it produces `TrainingExample` records. Simula does not call Arabic MCP tools at runtime and does not share state with the Arabic workflow. No cross-domain operational envelope is introduced.

12.19.2 With Trial Balance

Simula consumes TB's schemas and fixtures as training input. The contract is schema-in, `TrainingExample`-out: Simula reads the TB schema set and the TB fixture store; it produces `TrainingExample` records. Simula does not call TB MCP tools at runtime and does not share state with the TB workflow.

12.19.3 With Regulations

Regulations wraps Simula at runtime. Every Simula MCP call is brokered through the regulations governance gate, which applies pillars 1–4 of MAS MGF, attaches the identity envelope, and routes emergent-capability metrics to the monitoring store. The Simula service code does not call regulations directly; the gateway layer enforces this. Simula also consumes regulations' schemas (requirement, conformance-tool, monitoring metrics, identity and audit) as training input.

12.19.4 With Simula

Simula exports the training-example record, the schema-registry record, the taxonomy structure, and the enum vocabularies under `structured/schema/` (chapter 3); registers the MCP tools enumerated in §12.14; and applies the four governance hooks in §12.18 to every call it serves. External consumers see Simula only through these contracts and through the corpus it emits; there are no private back-channels.

12.20 Wave assignment and sequencing

Implementation runs in three waves.

- **Wave 1 — foundations.** Unified schema registry entry; persona-gated surface for the Training Engineer; identity attribution envelope flowing through the gateway; HANA extraction pipeline (Task S1). Prerequisites: the regulations chapter 10 identity schema must be frozen.
- **Wave 2 — Simula core build-out.** Tasks S2, S3, S4, S5. Prerequisites: Wave 1 complete; vLLM TurboQuant endpoint reachable; Arabic, TB, and regulations schema sets published.
- **Wave 3 — hardening and conformance.** Task S6 and full conformance-tooling hook-up. Prerequisites: Wave 2 complete; regulations Wave 2 conformance tooling integrated; monitoring sink live.

Simula occupies Wave 1 for its extraction pipeline (because other domains depend on Simula's fixtures being reachable), Wave 2 for its core build-out, and Wave 3 for its testing and conformance surfaces.

12.21 Acceptance criteria (platform-wide)

When all four chapter 12 instruction sets are implemented, the platform satisfies the following invariants:

1. **One instance.** A single codebase runs a single logical instance; no domain ships as a separate product.
2. **Persona-gated surfaces.** The four personas each reach a tailored UI on the shared shell; no route, MCP tool, or data product is visible outside its persona's entitlement set.
3. **Unified registry.** Every schema in use is listed in the ODPS 4.1 registry under its owning domain; validators resolve `$ref`s through the unified registry only.
4. **No schema merges.** The two `requirement.schema.json` files in TB and regulations remain distinct; no cross-domain composite schemas are produced.
5. **No cross-domain operational envelope.** Runtime orchestration stays within each domain's service; only Simula consumes other domains' outputs, and only as training input.
6. **Governance wraps all three non-regulations domains.** Arabic, TB, and Simula service calls pass through the regulations runtime wrapper without exception.

12.22 References

Within-spec references: chapter 1 (overview), chapter 3 (data schema), chapter 4 (HANA extraction pipeline), chapter 5 (taxonomy engine, Algorithm 1), chapter 6 (generation engine, Algorithm 2), chapter 7 (environment and CLI), chapter 8 (evaluation framework), chapter 11 (test fixtures), chapter 10 (testing specification), chapter 15 (project roadmap).

Peer chapter 12 instruction sets for the other three domains (each is self-contained; consult directly when operating in that domain):

- Arabic AP: `docs/latex/specs/arabic/chapters/12-implementation-instructions.tex`.
- Trial Balance: `docs/latex/specs/tb/chapters/12-implementation-instructions.tex`.
- Regulations: `docs/latex/specs/regulations/chapters/12-implementation-instructio`

Shared platform artefacts:

- ODPS 4.1 registry: `src/intelligence/ai-core-pal/data_products/registry.yaml`.
- Schema pipeline and resolver: `src/intelligence/schema_pipeline/`.

- Training pipeline (Simula): `src/training/pipeline/`.
- Shared workspace shell: `generativeUI/training-webcomponents-ngx`.

Chapter 13

Cross-Specification Dependencies

This chapter documents how the Simula Training Data Framework specification integrates with the other three SAP OSS specifications. All dependencies are traceable to `docs/schema/registry.json` (v1.2.0).

13.1 Architecture Position

The Simula specification occupies the **training infrastructure** position in the four-domain architecture:

- **Upstream**: Arabic AP (consumes invoice schemas)
- **Upstream**: Trial Balance (consumes TB/PL extract schemas)
- **Horizontal**: Regulations (training data governance)
- **Downstream**: Training JSONL → Model fine-tuning

13.1.1 Cross-domain handshake

Inter-domain coordination proceeds through the spec orchestrator and is mediated entirely by checked-in artefacts—no direct network calls between domain packs (see the inter-agent signaling rules in the Clinerules Agents specification). Figure 13.1 shows the reference sequence for a v1.2 release cycle.

13.1.2 Wave-dependency manifest

Figure 13.2 consolidates the global wave-dependency graph across all four domains. Wave 1 (blue) establishes identity and intake; wave 2 (amber) builds domain-specific runtimes; wave 3 (green) produces consumer corpora and evidence rollups.

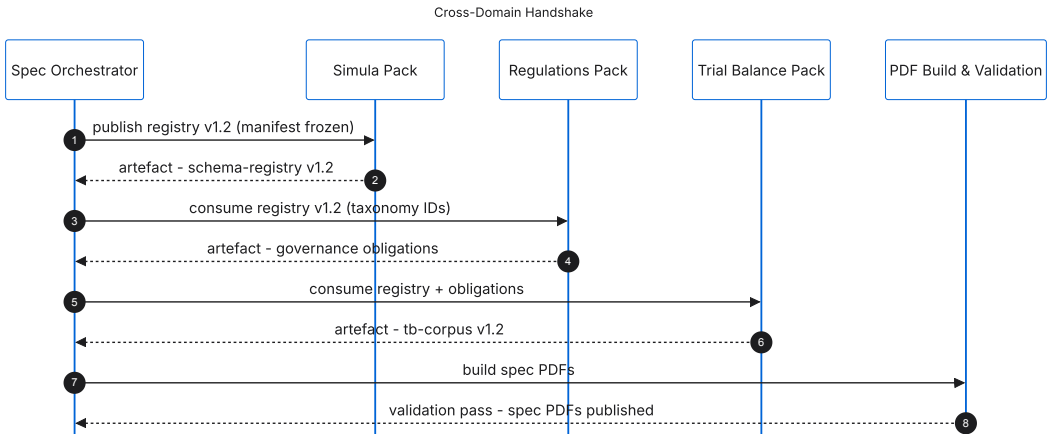

Figure 13.1: Reference sequence for a cross-domain handshake. Each arrow is a request or artefact-presence signal mediated by the repository; no direct network traffic between domain packs.

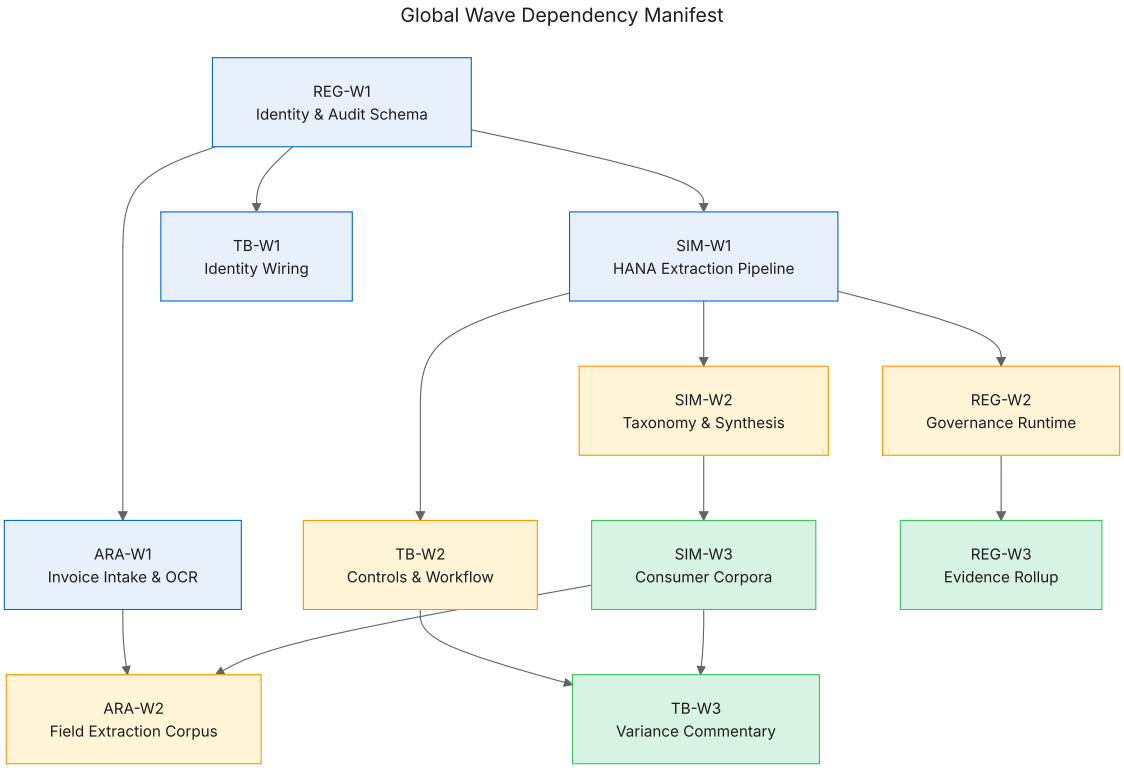

Figure 13.2: Global wave-dependency manifest. Blue: wave 1 (foundations). Amber: wave 2 (runtime). Green: wave 3 (consumer corpora and rollup). Arrows are hard prerequisites.

13.2 Schema Dependencies

13.2.1 Schemas Owned by Simula

Schema	Status	Purpose
taxonomy.schema.json	stable	Factor tree structure
training-example.schema.json	stable	Training JSONL format
hana-schema-entry.schema.json	stable	HANA metadata cache
config.schema.json	stable	Pipeline configuration
meta-prompt.schema.json	stable	Generation prompts

13.2.2 Schemas Consumed from Other Domains

Schema	Source	Usage
invoice.schema.json	arabic	Training for invoice field extraction
vat-checklist.schema.json	arabic	Training for VAT classification
tb-extract.schema.json	tb	Training for account analysis
pl-extract.schema.json	tb	Training for P&L commentary
base-types.yaml	common	Shared type definitions
enums.yaml	common	Complexity levels, verdicts
requirement.schema.json	regulations	Training data compliance

13.3 Regulations Integration

Simula training data generation is governed by Regulations:

13.3.1 Pillar 1: Training Data Bounds

- No PII in generated training examples
- Synthetic data only—no production data extraction
- Complexity distribution balanced (easy/medium/hard)

13.3.2 Pillar 2: Human Accountability

- Schema extraction requires DBA approval
- Training corpus sign-off by domain SME
- Double-critic verdicts logged for audit

13.3.3 Pillar 3: Technical Controls

- Drift monitoring: Training data quality metrics per Regulations Ch.8
- SQL syntax validation gate (see Algorithm 3)
- Conformance testing: AI Verify hooks per Regulations Ch.6

13.3.4 Pillar 4: Data Provenance

- Schema lineage tracking per Regulations Ch.7
- Training corpus versioning with SBOM integration

13.4 Arabic AP Integration

Simula consumes Arabic AP schemas for training data generation:

- **invoice.schema.json** → Invoice field extraction training
- **vat-checklist.schema.json** → VAT category classification
- **Schema extraction** from HANA tables: BKPF, BSEG, LFA1
- Cross-reference: See Arabic AP specification Chapter 2 (Data Schema)

13.5 Trial Balance Integration

Simula consumes TB schemas for training data generation:

- **tb-extract.schema.json** → Account analysis training
- **pl-extract.schema.json** → P&L commentary training
- **variance-record.schema.json** → Variance explanation training
- **Schema extraction** from HANA tables: ACDOCA, BSEG, SKB1
- Cross-reference: See TB specification Chapter 2 (Data Schema)

13.6 Cross-Reference Summary

Target Spec	Chapter	Integration Point
Regulations	Ch.3 (MGF Framework)	Training data governance
Regulations	Ch.6 (Conformance)	AI Verify/Moonshot hooks
Regulations	Ch.7 (Corpus)	Schema lineage tracking
Regulations	Ch.8 (Monitoring)	Training quality metrics
Arabic AP	Ch.2 (Data Schema)	Invoice schema consumption
Arabic AP	Ch.4 (VAT Engine)	VAT category training data
Trial Balance	Ch.2 (Data Schema)	TB/PL schema consumption
Trial Balance	Ch.3 (AI Pipeline)	Variance analysis training

Target Spec	Chapter	Integration Point

Table 13.1: Simula cross-specification references

Chapter 14

Schema Versioning and Interface Contracts

This chapter defines the schema versioning policy for the Simula training data framework, ensuring stable cross-domain interfaces and controlled evolution of data contracts.

14.1 Versioning Policy

14.1.1 Semantic Versioning for Schemas

All Simula schemas follow Semantic Versioning (SemVer) 2.0.0:

- **MAJOR.MINOR.PATCH** (e.g., `1.2.0`)
- **MAJOR**: Breaking changes to required fields or core structure
- **MINOR**: Backward-compatible additions (new optional fields)
- **PATCH**: Documentation, description, or metadata updates

14.1.2 Current Schema Versions

Table 14.1: Simula schema versions as of v1.2.

Schema	Version	Status	Description
`training-example.schema.json`	1.2.0	Stable	Training example record with critic evaluation
`taxonomy.schema.json`	1.2.0	Stable	Taxonomy tree structure with quality metrics
`schema-registry.schema.json`	1.1.0	Stable	HANA schema registry record
`evaluation-report.schema.json`	1.0.0	Draft	Evaluation framework output

14.2 Schema Evolution Rules

14.2.1 Backward Compatibility Requirements

1. **Required fields cannot be removed** in MINOR or PATCH versions.
2. **Required fields cannot be renamed** without a MAJOR version bump.
3. **New optional fields** can be added in MINOR versions.
4. **Enum values can only be added**, not removed, in MINOR versions.
5. **Removing enum values** requires a MAJOR version bump with deprecation notice.

14.2.2 Deprecation Process

1. Mark field as deprecated in schema description (MINOR version).
2. Add `deprecated: true` metadata if supported by tooling.
3. Document migration path in changelog.
4. Maintain deprecated field for at least one MINOR release cycle.
5. Remove deprecated field in next MAJOR version.

14.3 Cross-Domain Interface Contracts

14.3.1 Schema-In, TrainingExample-Out Contract

Simula maintains a strict contract with consumer domains:

Listing 14.1: Cross-domain interface contract.

```
Input (from Arabic AP, TB, Regulations):
  - Schema files conforming to each domain's schema registry
  - Fixture files for validation

Output (to consumers):
  - TrainingExample records conforming to training-example.schema.json
    v1.2.0
  - Taxonomy artifacts conforming to taxonomy.schema.json v1.2.0
  - Evaluation reports conforming to evaluation-report.schema.json v1.0.0
```

14.3.2 Interface Versioning

Each cross-domain interface is versioned independently:

Table 14.2: Cross-domain interface versions.

Interface	Version	Consumer	Contract
Simula → TB	1.2	Trial Balance	TrainingExample output
Simula → Arabic	1.2	Arabic AP	TrainingExample output
Simula → Regulations	1.2	Regulations	TrainingExample output
TB → Simula	1.0	Simula	Schema + fixture input
Arabic → Simula	1.0	Simula	Schema + fixture input
Regulations → Simula	1.0	Simula	Schema + fixture input

14.4 Schema Validation

14.4.1 Validation Points

Schema validation is enforced at three points:

1. **Input validation**: Before processing consumer schemas
2. **Output validation**: Before emitting TrainingExample records
3. **Report validation**: Before publishing evaluation reports

14.4.2 Validation Implementation

Listing 14.2: Schema validation in the pipeline.

```python
from jsonschema import validate, ValidationError
import json

def validate_training_example(example: dict) -> bool:
    """Validate a training example against the schema."""
    with open("docs/schema/simula/training-example.schema.json") as f:
        schema = json.load(f)

    try:
        validate(instance=example, schema=schema)
        return True
    except ValidationError as e:
        logger.error(f"Schema validation failed: {e.message}")
        return False

# Usage in the pipeline
for example in generated_examples:
    if not validate_training_example(example.to_dict()):
        raise SchemaValidationError(f"Example {example.id} failed
    validation")
```

14.5 Changelog Management

14.5.1 Schema Changelog Location

Schema changes are documented in:

- `docs/schema/CHANGELOG.md` — consolidated schema changelog
- Each schema file's `description` field — version-specific notes

14.5.2 Changelog Entry Format

Listing 14.3: Changelog entry format.

```
## [1.2.0] - 2026-04-21

### Added
- ‘critic_evaluation‘ complex field for double-critic results
- ‘generation_metadata‘ for reproducibility tracking
- ‘quality_signals‘ for downstream analysis

### Changed
- ‘complexity_score‘ now required (was optional in 1.1.0)

### Deprecated
- ‘difficulty‘ field (use ‘complexity_level‘ instead)
- ‘taxonomy_mix‘ field (use ‘taxonomy_path‘ instead)

### Migration
- Existing producers should add ‘complexity_score‘ with default 0.5
- Replace ‘difficulty‘ with ‘complexity_level‘ (EASY, MEDIUM, HARD)
```

14.6 Version Negotiation

14.6.1 Runtime Version Checking

The pipeline checks schema versions at runtime:

Listing 14.4: Version negotiation.

```python
def check_schema_compatibility(
    schema_path: str,
    min_version: str = "1.0.0",
    max_version: str = "2.0.0",
) -> bool:
    """Check if schema version is within supported range."""
    with open(schema_path) as f:
        schema = json.load(f)

    schema_id = schema.get("$id", "")
    # Extract version from $id or metadata
    version = extract_version(schema_id)

    return semver.match(version, f">={min_version}") and \
           semver.match(version, f"<{max_version}")
```

14.6.2 Graceful Degradation

When encountering an unknown schema version:

1. Log a warning with schema ID and version
2. Attempt to process with latest known compatible schema
3. Flag output with `schema_version_mismatch: true`
4. Include warning in generation stats

14.7 Implementation Checklist

- [x] Schema version in `$id` field
- [x] `training-example.schema.json` at v1.2.0
- [x] `taxonomy.schema.json` at v1.2.0
- [ ] Automated schema validation in CI
- [ ] Version negotiation in runtime
- [ ] Cross-domain contract tests
- [ ] Schema changelog automation

14.8 Governance

14.8.1 Schema Change Approval

Schema changes require:

1. RFC submitted to Controls Architecture Team
2. 5-day review period for MINOR changes
3. 10-day review period for MAJOR changes
4. All consuming spec owners must approve breaking changes
5. One-release deprecation window for breaking changes

14.8.2 Change Ownership

Table 14.3: Schema ownership.

Schema	Owner	Approval Authority
`training-example.schema.json`	Simula Lead	Simula + Consumer Leads
`taxonomy.schema.json`	Simula Lead	Simula Lead
`common/enums.yaml`	Controls Arch	All Domain Leads

Chapter 15

Project Roadmap

Scope of this chapter

This chapter is the roadmap for the **Simula training-data framework** project. It is a project-level artefact, not a programme plan. Other SAP OSS Finance projects (Trial Balance, Arabic AP, Regulations) run on the same platform, workspaces, and backends, but each owns its own roadmap, business case, and go/no-go decision. Simula is a *supplier* to those projects: it generates reference training corpora they consume, so its critical path is sequenced before theirs, but its sign-off is independent.

Sequence, not dates.

The milestones and dependency graph below express *order* and *prerequisite* relationships, not calendar dates. This is deliberate. Implementation teams must know what depends on what, and where the schedule can absorb slippage (the Should/Could items); a date column would harden targets into commitments they were never meant to be. Dates, when set, live in the project steering packet and the ticketing system, not in this specification.

15.1 MoSCoW classification

Every milestone below carries one of four priorities. The mapping is contractual: a milestone's priority determines what happens when it is dropped, de-scoped, or slips. The critical path of this project is the chain of Must items in Table 15.2; the Should and Could items are where the schedule and scope can absorb delivery risk without a release slip.

15.2 Dependency graph

15.3 Project milestones

The Must chain (M01 → M02 → M03 → M04 → M05 → M06 → M07 → M08) is the critical path. The Should items (S01–S03) are the primary *schedule allowance*: consumer corpus generations can slip without affecting Simula v1.2 sign-off. The Could items are the *scope allowance*: they may be cut to v1.3 without renegotiation.

Table 15.1: MoSCoW prioritisation (shared across the four SAP OSS Finance projects).

Priority	Meaning
Must (**M**)	Non-negotiable for v1.2. Without this milestone, the release does not ship.
Should (**S**)	Important but not vital. Ship without it only with sponsor sign-off; document the deviation in the busin
Could (**C**)	Desirable enhancement. Included if Must and Should items are on track.
Won't (**W**)	Explicitly out of v1.2 scope. Listed so the boundary is legible.

Table 15.2: Simula v1.2 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't.

ID	Prereq	Pri.	Ticket	Milestone
M01	—	M	SIM-ENG-001	Extraction pipeline passes HANA-direct fixtures in the testing chapter.
M02	M01	M	SIM-ENG-002	Taxonomy engine classifies reference corpus with accuracy above Beta threshold.
M03	M02	M	SIM-ENG-003	Generation engine produces schema-valid examples against the unified registry.
M04	M03	M	SIM-ENV-001	Environment CLI enables one-command reproduce of any reference run.
M05	M04	M	SIM-HANA-001	HANA spatial policy (`spatial_unknown_srid_policy`) enforced; strict mode passes unit tests.
M06	M05	M	SIM-RUN-TB	Trial Balance reference run reproducible end-to-end (commit `b8d40a1`, tag `simula-v1.2-worked-example`, seed `20251115`).
M07	M06	M	SIM-RELEASE-001	v1.2 release tagged, archived, and the reproducibility manifest frozen.
M08	M07	M	SIM-BC-REFRESH	Business-case refresh (first cycle after release; then annual).
S01	M07	S	SIM-RUN-AP	Arabic AP training corpus generated from the same pipeline.
S02	S01	S	SIM-RUN-REG	Regulations monitoring corpus generated from the same pipeline.
S03	M07	S	SIM-CONSUMER-DOC	Consumer integration runbook published (how Trial Balance, AP, and Regulations call Simula).
C01	S02 + v1.3 trigger (§15.7)	C	SIM-V13-EXPLORE	v1.3 scope kick-off (multi-tenant isolation, additional taxonomies).
C02	M07	C	SIM-TUNE	Quarterly prompt and classifier retraining from accept/edit telemetry.
W01	—	W	SIM-V12-EXTERNAL	External (non-SAP-OSS) customer access is *not* in v1.2 scope.
W02	—	W	SIM-V12-REALTIME	Real-time incremental generation is *not* in v1.2 scope; batch only.

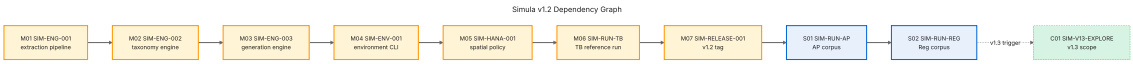

Figure 15.1: Simula dependency graph. Amber nodes are Must (critical path); blue are Should (consumer-corpus allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the v1.3 trigger in Section 15.7 fires).

15.4 Launch tiers and exit gates

The project walks the four-tier ladder shared across SAP OSS Finance projects. A tier is exited only when the named exit gate is satisfied; slippage requires sponsor sign-off.

15.5 Kill-switch conditions

The kill-switch is a pre-committed rulebook: a tripped condition enters the action column without further debate. GFAIC and sponsor are notified within 24 hours. The conditions are evaluated continuously (on every cycle) and do not carry dates.

15.6 Shared-platform dependencies

This project depends on three shared-platform components. Each is tracked here only as a dependency, not a commitment this project can unilaterally adjust. Sequence is expressed in terms of the Must item that first consumes the dependency, not in calendar time.
Shared-platform dependencies are surfaced at monthly steering. If any upstream owner declares **amber** or **red** on a component in Table 15.5, the affected downstream Must milestone is re-planned at the following steering.

15.7 v1.3 expansion trigger

The Could item C01 (SIM-V13-EXPLORE) is unlocked only when all of the following are observed after two consecutive consumer corpus generations under v1.2:

- aggregate consumer request volume for corpora outside the current taxonomy exceeds 20 % of total Simula runs;
- reproducibility and schema-validation rates sustained at or above release thresholds;
- business-case refresh (M08) explicitly budgets the v1.3 programme.

If any of the three conditions fails, C01 remains in the Could column and is re-evaluated at the next business-case refresh.

Table 15.3: Simula launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier.

Tier	Scope	Exit gate (references MoSCoW IDs)
Alpha	Engineering only, DEV + UAT, fixture corpora only.	M01, M02, M03 closed; schema-registry validation passes on the generated examples; reproducibility confirmed on two consecutive runs.
Beta	One consumer project (Trial Balance) drawing from Simula-generated corpora.	M04, M05, M06 closed; environment CLI reproduces the TB run end-to-end; HANA spatial strict

Table 15.4: Simula kill-switch.

Condition	Evaluation	Action
Reference run non-reproducible across two consecutive seeds (output diff > 0)	per release	**Halt release**; investigate non-determinism; do not ship v1.2 until reproducibility is restored.
Schema-registry validation fails on $> 0.1\,\%$ of generated examples	per run	**Halt consumer delivery**; fix generation engine; rerun.
HANA spatial strict mode detects unknown SRID in production path	continuous	**Quarantine run**; incident-review before releasing corpus to consumer project.
Any consumer project (TB, AP, Regulations) declares red on a Simula-supplied corpus	continuous	**Pause Should items** S01–S03; remediate

Table 15.5: Shared-platform dependencies.

Component	Owner	First consumed by	Impact if late
Unified schema registry (`docs/schema/`)	Shared infra	M03	Already `migration.status=co` no remaining risk.
vLLM TurboQuant hosting (`vllm-only` policy)	Shared platform	M03	Graceful-degradation ladder defined in the Regulations spec provides fallback; batch-generation slip tolerable.
HANA Cloud spatial extensions	Shared infra	M05	No substitute; slip blocks spatial-policy certification and downstream consumer runs.

15.8 Governance and business-case cadence

Project governance.

The Simula project reports monthly to a project steering (Studio AI Lead, Training Data PM, Pipeline Engineer) and quarterly to the cross-project GFAIC. Monthly steering accepts or rejects amber/red signals on Must items; GFAIC reviews compliance posture and any cross-project dependencies (especially consumer-project risk).

Business-case refresh (M08).

The business case that authorises this project is refreshed on the following rhythm:

- **Mandatory refresh**: at the first cycle after release, and annually thereafter (ticket SIM-BC-REFRESH, priority M in Table 15.2). Refresh packet includes consumer demand assumptions, unit economics, GPU cost realised vs. forecast, and a deviation statement against the baseline.
- **Trigger-based refresh**: any of
  - consumer request volume changes by $\geq$ 20 % from forecast (either direction);
  - per-run generation cost changes by $\geq$ 30 % (GPU price, vLLM licensing, HANA I/O);
  - a consumer project's regulatory obligation expands Simula's in-scope taxonomy;
  - any kill-switch in Table 15.4 is tripped.
- **Owner**: Studio AI Lead. Approval routed through the project steering and filed at `docs/simula/business-case/refresh-history.yaml`.

Chapter 16

Implementation Roadmap

Integration with existing chapters

This chapter extends the project roadmap in Chapter 15 with enterprise deployment phases:

- **Chapter 15** (Project Roadmap): The v1.2 milestones (M01–M07) defined there are *prerequisites* for this enterprise roadmap. Phase 1 here begins only after M07 (v1.2 tag) is complete.
- **Chapter 12** (Agent Instructions): Tasks S1–S6 in that chapter deliver the technical foundation; this roadmap adds the business governance wrapper (BR-1.x through BR-5.x).
- **Chapter 2** (Business Requirements): Each phase in this roadmap addresses specific BR requirements, as shown in Table 16.5.
- **Figure 15.1** (Dependency Graph): This roadmap extends the dependency graph with enterprise deployment nodes that follow the Could (C01) milestone.

Like Chapter 15, this roadmap expresses *sequence* and *dependency*, not calendar dates. The phases are ordered by prerequisite relationships and risk, not by time estimates.

Scope of this chapter

This chapter defines the phased implementation roadmap for enterprise deployment of the Simula synthetic data framework. The roadmap is organized into three sequential phases, each building upon the capabilities established in the previous phase. Phases are sequenced by dependency and risk, not by time estimates.

Relationship to Chapter 15 Milestones

This enterprise roadmap extends the technical milestones defined in Chapter 15. The mapping below clarifies which project milestones are addressed by each enterprise phase:
Key Relationships:

Table 16.1: Mapping of Project Milestones (Ch. 15) to Enterprise Phases (Ch. 16)

Milestone	MoSCoW	Phase	Relationship
M01–M07	Must	*Prerequisite*	v1.2 technical foundation; completes before Phase 1
S01–S03	Should	Phase 1	Addressed during capability building
C01–C02	Could	Phase 2	Addressed during scaled deployment
M08 (Business Case Refresh)	Must	Phase 1 Exit	Gate check before Phase 2

- **Phase 1** corresponds to completing the "Should" milestones (S01–S03) plus establishing enterprise governance (BR-3.x, BR-4.x requirements).
- **Phase 2** corresponds to completing the "Could" milestones (C01–C02) plus achieving production scale (BR-2.x requirements).
- **Phase 3** represents continuous operations beyond the initial roadmap, addressing long-term optimization and BR-5.x requirements.
- **M08 (Business Case Refresh)** from Chapter 15 serves as the Phase 1 exit gate, requiring documented ROI before Phase 2 investment.

Typical Duration Estimates

While this roadmap does not commit to calendar dates, the following are typical duration ranges assuming dedicated resources:

Table 16.2: Typical Phase Durations (Resource-Dependent)

Phase	Typical Duration	Assumptions
Phase 1	2–4 months	3 FTE engineers, infrastructure ready
Phase 2	3–6 months	Pilot success, production resources allocated
Phase 3	Ongoing	Operational team in place

These estimates are indicative only. Actual duration depends on organizational readiness, infrastructure availability, and competing priorities.

16.1 Roadmap Overview

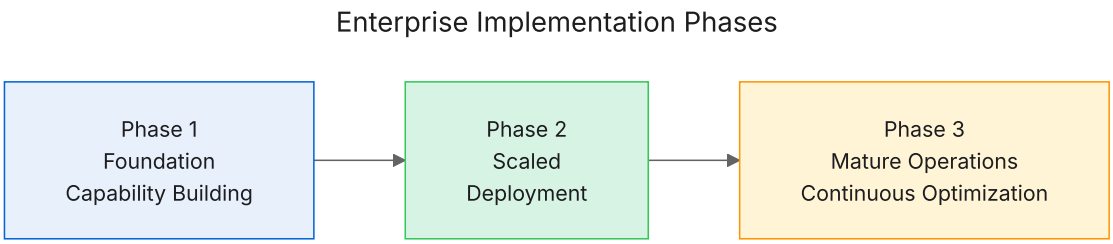

Figure 16.1: Simula implementation phases.

16.1.1 Phase Dependencies

Each phase has explicit entry criteria that must be satisfied before proceeding:

Table 16.3: Phase entry criteria.

Phase	Entry Criteria
Phase 1	Executive sponsorship, initial use case identified
Phase 2	Phase 1 complete, pilot success criteria met, production infrastructure available
Phase 3	Phase 2 complete, multiple use cases in production, operational team trained

16.2 Phase 1: Foundation Capability Building

Phase 1 establishes the foundational capabilities required for Simula deployment, focusing on a single high-value use case to validate the approach before scaling.

16.2.1 Phase 1 Objectives

1. Establish organizational readiness and governance integration

2. Deploy core Simula pipeline for pilot use case

3. Validate quality and utility through downstream evaluation

4. Build internal expertise and operational procedures

16.2.2 Phase 1 Deliverables

1.1 Use Case Definition and Value-Risk Assessment

1.1 Use Case and Risk Assessment

Requirements Addressed: BR-1.1, BR-1.2
Activities:

- Identify and document pilot use case with clear business objective
- Conduct structured value-risk assessment
- Define success criteria and measurement approach
- Obtain executive sponsor approval

Deliverables:

- Use case specification document
- Value-risk assessment matrix
- Success criteria definition
- Executive sign-off

Exit Criteria: Documented use case with approved risk assessment

1.2 Cross-Functional Team Assembly

1.2 Team Organization

Requirements Addressed: BR-1.4
Activities:

- Identify and assign team members across functions
- Establish communication cadence and escalation paths
- Define decision-making authority matrix
- Conduct initial training on Simula concepts

Deliverables:

- Team charter with named individuals
- RACI matrix for Simula activities
- Communication plan
- Training completion records

Exit Criteria: Functional team with clear roles and responsibilities

1.3 Governance Framework Integration

1.3 Governance Integration

Requirements Addressed: BR-1.3
Activities:

- Map synthetic data activities to existing governance frameworks
- Draft synthetic data policies and procedures
- Configure approval workflows in existing tools
- Update stakeholder training materials

Deliverables:

- Synthetic data policy document
- Updated governance framework documentation
- Configured approval workflows
- Training materials

SAP Integration: Configure SAP GRC for policy management
Exit Criteria: Governance policies documented and integrated

1.4 Domain Taxonomy Construction

1.4 Taxonomy Development

Requirements Addressed: BR-2.1
Activities:

- Extract domain factors from pilot use case schema
- Generate initial taxonomies using Simula taxonomy builder
- Conduct domain expert review and validation
- Establish taxonomy version control

Deliverables:

- Domain taxonomy artifacts (JSON/YAML)
- Expert validation sign-off
- Coverage gap analysis report
- Taxonomy versioning repository

SAP Integration: Store taxonomies in SAP HANA Cloud
Exit Criteria: Validated taxonomy with expert approval

1.5 Metadata and Provenance Infrastructure

1.5 Provenance System

Requirements Addressed: BR-2.4
Activities:

- Design metadata schema for generation provenance
- Implement automatic metadata capture in pipeline
- Configure tamper-evident storage
- Build metadata query interface

Deliverables:

- Metadata schema definition
- Provenance capture implementation
- Storage configuration
- Query interface

SAP Integration: Utilize SAP HANA Cloud for metadata storage with hash-chained integrity
Exit Criteria: Automated provenance capture operational

1.6 Quality Assessment Pipeline

1.6 Quality Evaluation

Requirements Addressed: BR-3.1, BR-3.2
Activities:

- Implement intrinsic quality metrics (coverage, diversity, complexity)
- Build automated quality report generation
- Configure downstream validation pipeline (TSTR)
- Establish quality thresholds and gates

Deliverables:

- Quality evaluation module
- Automated report templates
- TSTR pipeline configuration
- Quality gate definitions

SAP Integration: Execute validation via SAP AI Core
Exit Criteria: Quality pipeline generating actionable reports

16.2.3 Phase 1 Exit Criteria

Phase 1 is complete when:

✓ Pilot use case successfully generates synthetic dataset

✓ Downstream model trained on synthetic data meets quality gate

✓ Governance policies documented and approved

✓ Provenance and quality systems operational

✓ Team trained and operational procedures documented

16.3 Phase 2: Scaled Deployment

Phase 2 scales Simula capabilities to production usage with multiple use cases, implementing user-facing interfaces and comprehensive compliance mechanisms.

16.3.1 Phase 2 Objectives

1. Enable self-service generation for business users
2. Implement comprehensive privacy and bias controls

3. Deploy production access control and audit systems

4. Integrate with enterprise ML pipelines

16.3.2 Phase 2 Deliverables

2.1 Parameter Configuration Interface

2.1 User Configuration Interface

Requirements Addressed: BR-2.2
Activities:

- Design visual configuration interface
- Implement parameter validation and constraint checking
- Build preview mode for distribution estimation
- Create configuration templates for common scenarios

Deliverables:

- Web-based configuration UI
- Parameter validation logic
- Preview visualization
- Template library

SAP Integration: Build using SAP Build Apps; store configurations in SAP HANA Cloud
Exit Criteria: Business users can configure and launch generation tasks

2.2 Privacy Risk Assessment System

2.2 Privacy Controls

Requirements Addressed: BR-3.3
Activities:

- Implement membership inference attack testing
- Build similarity scanning against reference datasets
- Configure privacy risk scoring and thresholds
- Design remediation workflow for high-risk samples

Deliverables:

- Privacy risk evaluation module
- Similarity scanner
- Risk scoring dashboard
- Remediation workflow

SAP Integration: Integrate with SAP Data Privacy Integration
Exit Criteria: Automated privacy screening on all generated datasets

2.3 Bias Detection and Mitigation

2.3 Bias Controls

Requirements Addressed: BR-3.4
Activities:

- Implement distribution analysis on sensitive attributes
- Build fairness metric computation
- Design bias mitigation recommendations
- Create bias audit trail

Deliverables:

- Bias detection module
- Fairness metrics dashboard
- Mitigation recommendation engine
- Audit trail storage

Exit Criteria: Bias screening integrated into generation pipeline

2.4 Access Control and Permissions

2.4 RBAC Implementation

Requirements Addressed: BR-4.1
Activities:

- Configure role-based access control
- Implement dataset access approval workflows
- Define domain-specific access policies
- Enable comprehensive access logging

Deliverables:

- RBAC configuration
- Approval workflow
- Access policies
- Audit log configuration

SAP Integration: Leverage SAP Cloud Identity Services and XSUAA
Exit Criteria: Production access controls operational

2.5 ML Pipeline Integration

2.5 MLOps Integration

Requirements Addressed: BR-5.1
Activities:

- Implement REST API for generation task management
- Build SAP AI Core integration
- Configure webhook notifications
- Enable artifact registration in ML pipelines

Deliverables:

- Generation task API
- SAP AI Core connector
- Webhook configuration
- Pipeline integration documentation

SAP Integration: Primary integration with SAP AI Core MLOps
Exit Criteria: Simula integrated into production ML pipelines

2.6 Audit Logging System

2.6 Audit Trail

Requirements Addressed: BR-4.3
Activities:

- Deploy tamper-evident audit log storage
- Implement comprehensive event logging
- Build query interface for investigations
- Configure retention policies

Deliverables:

- Audit log storage
- Event logging implementation
- Query interface
- Retention configuration

SAP Integration: Utilize SAP Audit Log Service
Exit Criteria: Complete audit trail for all Simula activities

16.3.3 Phase 2 Exit Criteria

Phase 2 is complete when:

✓ Multiple use cases deployed in production

✓ Business users can self-service generation tasks

✓ Privacy and bias controls operational

✓ Access control and audit systems production-ready

✓ Integration with enterprise ML pipelines complete

16.4 Phase 3: Mature Operations and Continuous Optimization

Phase 3 establishes mature operational capabilities with continuous monitoring, optimization, and advanced features for long-term sustainability.

16.4.1 Phase 3 Objectives

1. Implement continuous quality monitoring and drift detection
2. Complete regulatory compliance and transparency requirements

3. Establish incident response capabilities

4. Optimize costs and enable advanced features

16.4.2 Phase 3 Deliverables

3.1 Continuous Monitoring and Drift Detection

3.1 Quality Monitoring

Requirements Addressed: BR-3.5
Activities:

- Implement scheduled quality sampling jobs
- Build drift detection with statistical testing
- Configure alerts for threshold breaches
- Enable automated remediation triggers

Deliverables:

- Monitoring job scheduler
- Drift detection module
- Alert configuration
- Remediation automation

SAP Integration: Configure via SAP AI Core; alerts via SAP Alert Notification Service
Exit Criteria: Continuous monitoring operational with alert coverage

3.2 Regulatory Compliance Completion

3.2 Compliance Framework

Requirements Addressed: BR-4.2
Activities:

- Document compliance position for each applicable regulation
- Obtain legal review sign-off
- Establish regulatory change monitoring
- Implement jurisdiction-specific variations

Deliverables:

- Compliance documentation
- Legal sign-off records
- Regulatory monitoring process
- Jurisdiction policy matrix

SAP Integration: Integrate with SAP GRC for regulatory tracking
Exit Criteria: Documented compliance for all applicable regulations

3.3 Transparency and Explainability

3.3 External Transparency

Requirements Addressed: BR-4.4
Activities:

- Create synthetic data documentation templates
- Develop stakeholder communication guidelines
- Build model card integration
- Prepare FAQ and training materials

Deliverables:

- Documentation templates
- Communication guidelines
- Model card integration
- FAQ materials

Exit Criteria: Transparency materials available for all use cases

3.4 Incident Response Procedures

3.4 Incident Management

Requirements Addressed: BR-4.5
Activities:

- Define incident classification and severity matrix
- Identify responsible parties for each scenario
- Establish escalation and notification paths
- Create remediation playbooks

Deliverables:

- Incident classification matrix
- Responsibility assignment
- Escalation procedures
- Playbooks

SAP Integration: Integrate with existing incident management
Exit Criteria: Incident response procedures documented and tested

3.5 Cost and Budget Optimization

3.5 Cost Management

Requirements Addressed: BR-2.3
Activities:

- Implement budget allocation by cost center
- Build real-time cost tracking
- Configure budget alerts
- Enable cost-per-example reporting

Deliverables:

- Budget allocation interface
- Cost tracking dashboard
- Alert configuration
- Cost reporting

SAP Integration: Integrate with SAP Analytics Cloud
Exit Criteria: Cost visibility and control for all generation activities

3.6 Incremental Generation Capability

3.6 Dataset Evolution

Requirements Addressed: BR-2.5
Activities:

- Implement taxonomy diff computation
- Build targeted generation for changed nodes
- Enable dataset merge with deduplication
- Create efficiency metrics reporting

Deliverables:

- Taxonomy diff module
- Targeted generation logic
- Merge/deduplication logic
- Efficiency reporting

Exit Criteria: Incremental generation available for all datasets

16.4.3 Phase 3 Exit Criteria

Phase 3 is complete when:

- ✓ Continuous monitoring detecting and alerting on quality issues
- ✓ Full regulatory compliance documented and approved
- ✓ Transparency materials available for external stakeholders
- ✓ Incident response procedures tested and operational
- ✓ Cost optimization and incremental generation operational

16.5 Key Success Factors

16.5.1 Critical Success Factors

1. **Domain Expert Engagement**: Simula's reasoning-driven approach achieves seedless generation, but taxonomy quality determines coverage and validity. Domain expert investment in taxonomy validation is essential.
2. **Governance-First Approach**: Synthetic data governance should be designed into the project from inception, not retrofitted after large-scale generation. Integrating with existing frameworks is more practical than building new ones.
3. **High-Value Use Case Focus**: The synthetic data market is growing rapidly (projected CAGR 43.9% from 2025–2032), but organizations should avoid broad deployment before validating ROI on focused use cases.
4. **Continuous Validation Loop**: Synthetic data is not a one-time generation task. Establishing "generate → validate → feedback → optimize" loops ensures sustained value.

16.5.2 Risk Mitigation Summary

Table 16.4: Key risks and mitigations by phase.

Phase	Risk	Mitigation
Phase 1	Taxonomy quality insufficient	Expert validation gates, iterative refinement
Phase 1	Governance friction	Early integration with existing frameworks
Phase 2	Privacy incidents	Automated screening, remediation workflows
Phase 2	User adoption low	Self-service interfaces, templates, training
Phase 3	Quality drift undetected	Continuous monitoring, statistical alerts
Phase 3	Cost overruns	Budget controls, cost visibility dashboards

16.6 Requirements Coverage by Phase

16.7 SAP Technology Stack Summary

Table 16.5: Business requirements addressed by phase.

Category	**Phase 1**	**Phase 2**	**Phase 3**
BR-1.x Strategic	1.1, 1.2, 1.3, 1.4	—	1.5
BR-2.x Generation	2.1, 2.4	2.2	2.3, 2.5
BR-3.x Quality	3.1, 3.2	3.3, 3.4	3.5
BR-4.x Governance	—	4.1, 4.3	4.2, 4.4, 4.5
BR-5.x Integration	—	5.1	5.2, 5.3, 5.4

Table 16.6: SAP technology integration by capability.

Capability	**SAP Technology**
Schema Extraction	SAP HANA Cloud
Taxonomy Storage	SAP HANA Cloud
ML Training Pipeline	SAP AI Core
Configuration Interface	SAP Build Apps
Cost Analytics	SAP Analytics Cloud
Access Control	SAP Cloud Identity Services, XSUAA
Audit Logging	SAP Audit Log Service
Alert Notifications	SAP Alert Notification Service
Governance	SAP GRC
Data Catalog	SAP Data Intelligence
Object Storage	SAP Object Store
Workflow Orchestration	SAP BTP Workflow

Chapter 17

Future Directions (v1.3+)

Future Work

Important Notice
All features described in this chapter are **explicitly out of scope for v1.2**. They represent planned enhancements for v1.3 and beyond. Do not include these features in v1.2 implementation planning or resource allocation.

This chapter is provided for:

- Strategic planning and roadmap visibility
- Stakeholder expectation setting
- Architecture decisions that preserve future extensibility

Integration with existing chapters

This chapter describes future enhancements that extend the foundational Simula specification established in earlier chapters:

- **Chapter 4** (Extraction Pipeline): Enhancement 1 (Multi-Modal) extends schema extraction to document images and time-series.
- **Chapter 5** (Taxonomy Engine): Enhancement 2 (Domain Calibration) adds knowledge anchors to the Algorithm 1 taxonomy builder.
- **Chapter 6** (Generation Engine): Enhancement 3 (Closed-Loop) adds feedback integration to Algorithm 2 data synthesis.
- **Chapter 15** (Project Roadmap): The business requirements in Chapter 2 and implementation roadmap in Chapter 16 are sequenced *after* the v1.2 milestones (M01–M07) complete.
- **Chapter 12** (Agent Instructions): Task S1–S6 remain the technical implementation tasks; the business requirements here provide the governance and compliance wrapper around those tasks.

The enhancements defined here are categorized as **Should** or **Could** items in the MoSCoW classification from Table 15.1, meaning they may be deferred to v1.3 if the Must milestones consume available capacity.

Scope of this chapter

This chapter defines enterprise-grade enhancements to the Simula framework, extending the core "mechanism design" approach to synthetic data generation for production deployment in regulated industries. These enhancements address the practical limitations identified in the original research while preserving Simula's key differentiators: dataset-level design, seedless generation, and independent control of coverage, complexity, and quality.

17.1 Enhancement Overview

The Simula framework, as described in "Reasoning-Driven Synthetic Data Generation and Evaluation" (OpenReview NALsdGEPhB, 2026) and the Google Research blog "Designing Synthetic Datasets for the Real World," introduces four foundational steps:

1. **Global Diversification** — Taxonomic sampling scaffolds for coverage
2. **Local Diversification** — Meta-prompt variation to prevent mode collapse
3. **Complexification** — Controlled difficulty progression
4. **Quality Assurance** — Reasoning-driven verification

This chapter extends each step with enterprise-ready capabilities:

Table 17.1: Simula enhancement dimensions.

Enhancement	Simula Step	Enterprise Value
Multi-Modal Extension	Global + Local	Vision, time-series, tabular data support
Domain Adaptation Calibration	Global	Vertical-specific accuracy (finance, healthcare)
Closed-Loop Validation	Quality Assurance	Measurable ROI via downstream correlation
Programmable Workflow Engine	All Steps	Declarative specifications, audit compliance

17.2 Enhancement 1: Multi-Modal Extension

17.2.1 Current Limitation

The Simula research primarily demonstrates text-based synthetic data generation. Enterprise use cases frequently require multi-modal data:

- **Document processing**: Scanned invoices with embedded tables (Arabic AP domain)

- **Time-series analytics**: Financial metrics with temporal patterns (Trial Balance domain)
- **Structured data**: Tabular records with categorical and numerical fields

17.2.2 Cross-Modal Reasoning Scaffolds

Extend the taxonomic scaffolding approach to multi-modal scenarios by constructing *cross-modal reasoning graphs* that capture relationships between modalities.

Listing 17.1: Cross-modal taxonomy extension.

```python
@dataclass
class MultiModalTaxonomyNode(TaxonomyNode):
    """Extended taxonomy node with cross-modal relationships."""

    # Primary modality for this node
    primary_modality: str = "text"  # text, image, tabular, time_series

    # Cross-modal mappings
    cross_modal_mappings: dict[str, str] = field(default_factory=dict)
    # Example: {"image": "invoice_header_region", "text":
   "vendor_name_field"}

    # Modality-specific constraints
    modality_constraints: dict[str, Any] = field(default_factory=dict)
    # Example: {"image": {"min_resolution": [800, 600], "format": "PNG"}}

class CrossModalTaxonomyBuilder(SimulaTaxonomyBuilder):
    """Build taxonomies with cross-modal consistency."""

    async def build_cross_modal_taxonomy(
        self,
        factor: Factor,
        modalities: list[str],
    ) -> MultiModalTaxonomy:
        """
        Build taxonomy ensuring cross-modal consistency.

        For example, in medical imaging:
        disease_type -> imaging_presentation -> text_description
        ensures generated image-text pairs are semantically aligned.
        """
        # Build primary modality taxonomy
        primary_taxonomy = await self.build_taxonomy(factor)

        # Extend with cross-modal mappings
        for node in primary_taxonomy.get_leaf_nodes():
            mappings = await self._infer_cross_modal_mappings(
                node, modalities
            )
            node.cross_modal_mappings = mappings

```

```
        return MultiModalTaxonomy(
            primary_taxonomy=primary_taxonomy,
            modalities=modalities,
        )
```

17.2.3 Modality-Aware Complexification

Define modality-specific complexity metrics:

Table 17.2: Modality-specific complexity dimensions.

Modality	Complexity Dimensions
Text	Vocabulary diversity, syntactic complexity, reasoning depth
Image	Background clutter, lighting variation, occlusion, scale variance
Tabular	Cardinality, sparsity, categorical diversity, numerical range
Time-Series	Trend strength, seasonality, anomaly density, noise ratio

17.2.4 SAP Integration: Document Information Extraction

For SAP Document Information Extraction (DOX) scenarios:

Listing 17.2: SAP DOX multi-modal integration.

```
class SAPDOXModalityAdapter:
    """Adapter for SAP Document Information Extraction."""

    MODALITY_MAPPING = {
        "invoice_header": ["image_region", "text_field", "table_cell"],
        "line_items": ["table_structure", "text_sequence"],
        "totals": ["numerical_field", "currency_format"],
    }

    def generate_cross_modal_example(
        self,
        taxonomy_path: list[str],
        complexity_level: str,
    ) -> MultiModalTrainingExample:
        """
        Generate aligned multi-modal training example.

        Output includes:
        - Synthetic document image (PNG/PDF)
        - Extracted text annotations
        - Structured field mappings
        - Ground truth bounding boxes
        """
        pass
```

17.3 Enhancement 2: Domain Adaptation Calibration

17.3.1 Current Limitation

Simula's seedless, reasoning-driven approach relies on the LLM's inherent knowledge to construct taxonomies. In specialized domains (finance, healthcare, manufacturing), this knowledge may be incomplete or biased toward common patterns, missing critical long-tail scenarios.

17.3.2 Human-in-Loop Knowledge Anchors

Introduce lightweight "knowledge anchors" that guide taxonomy construction without violating the seedless principle:

Listing 17.3: Domain knowledge anchor injection.

```python
@dataclass
class DomainKnowledgeAnchor:
    """Lightweight knowledge injection for domain calibration."""

    # Domain identifier
    domain: str  # "trial_balance", "arabic_ap", "regulations"

    # Required coverage nodes (MUST appear in taxonomy)
    required_nodes: list[str] = field(default_factory=list)

    # Prohibited nodes (MUST NOT appear - regulatory/compliance)
    prohibited_nodes: list[str] = field(default_factory=list)

    # Expert-provided complexity anchors
    complexity_anchors: dict[str, float] = field(default_factory=dict)
    # Example: {"multi_currency_reconciliation": 0.85, "simple_variance":
    0.2}

    # Domain-specific vocabulary
    domain_vocabulary: list[str] = field(default_factory=list)

class CalibratedTaxonomyBuilder(SimulaTaxonomyBuilder):
    """Taxonomy builder with domain calibration."""

    async def build_calibrated_taxonomy(
        self,
        factor: Factor,
        anchors: DomainKnowledgeAnchor,
    ) -> Taxonomy:
        """
        Build taxonomy with domain expert calibration.

        Process:
        1. Generate initial taxonomy via LLM reasoning
        2. Validate required nodes are present
```

```
        3. Remove prohibited nodes
        4. Calibrate complexity scores against anchors
        5. Present to expert for final review (optional)
        """
        # Step 1: Initial generation
        taxonomy = await self.build_taxonomy(factor)

        # Step 2: Validate required nodes
        existing_nodes = {n.name.lower() for n in taxonomy.get_leaf_nodes()}
        missing = set(anchors.required_nodes) - existing_nodes

        if missing:
            logger.warning(f"Missing required nodes: {missing}")
            taxonomy = await self._inject_missing_nodes(taxonomy, missing)

        # Step 3: Remove prohibited nodes
        taxonomy = self._prune_prohibited(taxonomy, anchors.prohibited_nodes)

        # Step 4: Calibrate complexity
        taxonomy = self._calibrate_complexity(taxonomy, anchors.complexity_anchors)

        return taxonomy
```

17.3.3 Vertical-Specific Complexity Metrics

Define industry-standard complexity dimensions:

Table 17.3: Vertical-specific complexity definitions.

Vertical	Metric	Definition
Finance	Regulatory complexity	Number of compliance rules applicable
	Temporal complexity	Multi-period analysis requirements
	Entity complexity	Cross-entity consolidation depth
Healthcare	Diagnostic complexity	Differential diagnosis breadth
	Protocol complexity	Multi-step treatment pathways
	Comorbidity complexity	Co-occurring condition interactions
Manufacturing	Process complexity	Operation sequence length
	Tolerance complexity	Precision requirements (6-sigma)
	Supply chain complexity	Multi-tier supplier dependencies

17.3.4 SAP Integration: S/4HANA Domain Schemas

Listing 17.4: SAP S/4HANA domain calibration.

```
# Domain anchors for SAP verticals
SAP_DOMAIN_ANCHORS = {
    "trial_balance": DomainKnowledgeAnchor(
```

```
        domain="trial_balance",
        required_nodes=[
            "intercompany_elimination",
            "currency_translation",
            "statutory_adjustment",
            "management_overlay",
        ],
        prohibited_nodes=[
            "personal_account_detail",  # Privacy
            "salary_individual",        # Compliance
        ],
        complexity_anchors={
            "simple_variance_analysis": 0.2,
            "multi_currency_consolidation": 0.75,
            "complex_elimination_entry": 0.9,
        },
    ),
    "arabic_ap": DomainKnowledgeAnchor(
        domain="arabic_ap",
        required_nodes=[
            "vat_compliance",
            "withholding_tax",
            "payment_terms_discount",
            "multi_currency_invoice",
        ],
        complexity_anchors={
            "standard_invoice": 0.15,
            "credit_note_reconciliation": 0.5,
            "multi_vendor_payment_run": 0.8,
        },
    ),
}
```

17.4 Enhancement 3: Closed-Loop Validation

17.4.1 Current Limitation

Simula's quality assurance relies on reasoning-model self-evaluation. This lacks direct feedback from downstream model performance, making it difficult to:

- Quantify ROI of synthetic data generation
- Identify systematic blind spots in coverage
- Adapt generation parameters based on model learning curves

17.4.2 Training-Validation Feedback Loop

Integrate Simula with the model training pipeline to create a closed feedback loop:

Listing 17.5: Closed-loop validation architecture.

```python
@dataclass
class DownstreamPerformanceSignal:
    """Performance signal from downstream model training."""

    # Model identifier
    model_id: str

    # Performance by complexity band
    accuracy_by_complexity: dict[str, float]  # {"easy": 0.95, "hard": 0.72}

    # Performance by taxonomy path
    accuracy_by_taxonomy: dict[str, float]  # {"sql/aggregation/sum": 0.98}

    # Identified weak spots
    underperforming_nodes: list[str]

    # Timestamp
    evaluated_at: str

class ClosedLoopGenerator(SimulaDataGenerator):
    """Generator with downstream feedback integration."""

    async def generate_with_feedback(
        self,
        initial_count: int,
        downstream_evaluator: Callable,
        max_iterations: int = 3,
    ) -> list[TrainingExample]:
        """
        Generate data with iterative refinement based on downstream performance.

        Algorithm:
        1. Generate initial dataset
        2. Train downstream model
        3. Evaluate performance by taxonomy node
        4. Identify underperforming regions
        5. Adjust sampling weights to oversample weak regions
        6. Generate additional targeted examples
        7. Repeat until convergence or max iterations
        """
        examples = await self.generate_examples(initial_count)

        for iteration in range(max_iterations):
            # Train and evaluate downstream model
            signal = await downstream_evaluator(examples)

            # Check convergence
            if self._is_converged(signal):
                break
```

```python

            # Adjust sampling weights
            weight_adjustments = self._compute_weight_adjustments(signal)
            self._apply_weight_adjustments(weight_adjustments)

            # Generate targeted examples for weak regions
            additional = await self._generate_targeted(
                signal.underperforming_nodes,
                count=initial_count // 4,
            )
            examples.extend(additional)

        return examples

    def _compute_weight_adjustments(
        self,
        signal: DownstreamPerformanceSignal,
    ) -> dict[str, float]:
        """
        Compute sampling weight adjustments based on performance gaps.

        Regions with lower accuracy receive higher sampling weights
        in subsequent generations.
        """
        adjustments = {}
        mean_accuracy = sum(signal.accuracy_by_taxonomy.values()) /
    len(signal.accuracy_by_taxonomy)

        for node, accuracy in signal.accuracy_by_taxonomy.items():
            if accuracy < mean_accuracy:
                # Boost underperforming regions
                gap = mean_accuracy - accuracy
                adjustments[node] = 1.0 + (gap * 2)  # Up to 2x boost
            else:
                adjustments[node] = 1.0

        return adjustments
```

17.4.3 Adversarial Verification Layer

Add an adversarial detector to identify systematic biases:

Listing 17.6: Adversarial verification for distribution drift.

```python
class AdversarialVerifier:
    """Detect systematic biases in synthetic data."""

    async def detect_distribution_drift(
        self,
        synthetic_examples: list[TrainingExample],
        reference_distribution: dict[str, float],
    ) -> DriftReport:
        """
```

```
        Compare synthetic data distribution against reference.

        Detects:
        - Mode collapse (over-concentration in specific patterns)
        - Coverage gaps (missing taxonomy regions)
        - Complexity skew (imbalanced difficulty distribution)
        """
        pass

    async def adversarial_probe(
        self,
        synthetic_examples: list[TrainingExample],
        probe_queries: list[str],
    ) -> ProbeReport:
        """
        Use adversarial probes to test for memorization.

        Checks if generated data inadvertently reproduces
        patterns from the reasoning model's training data.
        """
        pass
```

17.4.4 SAP Integration: AI Core MLOps

Listing 17.7: SAP AI Core integration for closed-loop training.

```
class SAPAICoreIntegration:
    """Integration with SAP AI Core for MLOps feedback loop."""

    async def submit_training_job(
        self,
        training_data_path: str,
        model_config: dict,
    ) -> str:
        """Submit training job to SAP AI Core."""
        # Use SAP AI Core SDK
        pass

    async def get_evaluation_metrics(
        self,
        execution_id: str,
    ) -> DownstreamPerformanceSignal:
        """Retrieve evaluation metrics from completed training."""
        # Parse AI Core metrics
        pass

    async def trigger_retraining(
        self,
        signal: DownstreamPerformanceSignal,
        threshold: float = 0.85,
    ) -> Optional[str]:
        """Trigger retraining if performance below threshold."""
```

```
        if signal.overall_accuracy < threshold:
            # Generate additional targeted examples
            # Submit new training job
            pass
```

17.5 Enhancement 4: Programmable Workflow Engine

17.5.1 Current Limitation

Simula provides conceptual controllability but lacks:

- User-friendly interfaces for business users
- Declarative specification of dataset requirements
- Version control and reproducibility guarantees
- Audit trails for compliance

17.5.2 Declarative Dataset Specification Language

Design a YAML-based DSL for dataset specification:

Listing 17.8: Simula dataset specification DSL.

```yaml
# simula-dataset-spec.yaml
apiVersion: simula/v1
kind: DatasetSpecification
metadata:
  name: trial-balance-text2sql-v3
  domain: trial_balance
  owner: finance-ai-team
  created: 2026-04-21

spec:
  # Target dataset size
  target_count: 10000

  # Coverage requirements
  coverage:
    taxonomy_paths:
      - path: "query_type/aggregation/*"
        min_coverage: 0.15
      - path: "query_type/filtering/*"
        min_coverage: 0.20
      - path: "complexity/multi_table_join"
        min_coverage: 0.10

  # Complexity distribution
  complexity:
    distribution:
```

```yaml
      easy: 0.30
      medium: 0.45
      hard: 0.25
    calibration_anchors:
      - example_id: "expert_example_001"
        target_score: 0.85

  # Quality thresholds
  quality:
    critic_pass_rate: 0.90
    diversity_threshold: 0.70

  # Generation parameters
  generation:
    model: "gpt-4-turbo"
    temperature: 0.7
    best_of_n: 5
    enable_complexification: true
    complexification_ratio: 0.30

  # Validation
  validation:
    schema: "docs/schema/simula/training-example.schema.json"
    downstream_eval: true

  # Output
  output:
    format: jsonl
    path: "data/hana_generated/tb_text2sql_v3.jsonl"
    include_metadata: true
```

17.5.3 Version Control and Reproducibility

Listing 17.9: Git-like versioning for datasets.

```python
@dataclass
class DatasetVersion:
    """Version metadata for reproducibility."""

    version_id: str  # Semantic version or hash
    parent_version: Optional[str]

    # Generation configuration snapshot
    spec_snapshot: dict
    taxonomy_snapshot: dict

    # Reproducibility parameters
    random_seed: int
    model_version: str

    # Statistics
    example_count: int
```

```
    coverage_report: dict
    quality_report: dict

    # Audit trail
    created_by: str
    created_at: str
    approval_status: str  # draft, reviewed, approved
    approver: Optional[str]

class DatasetVersionControl:
    """Git-like version control for synthetic datasets."""

    def commit(
        self,
        examples: list[TrainingExample],
        spec: DatasetSpecification,
        message: str,
    ) -> DatasetVersion:
        """Commit a new dataset version."""
        pass

    def diff(
        self,
        version_a: str,
        version_b: str,
    ) -> DatasetDiff:
        """Compare two dataset versions."""
        pass

    def rollback(
        self,
        version_id: str,
    ) -> list[TrainingExample]:
        """Rollback to a previous version."""
        pass

    def audit_log(
        self,
        version_id: str,
    ) -> list[AuditEntry]:
        """Get complete audit trail for a version."""
        pass
```

17.5.4 SAP Integration: BTP and HANA Cloud

Listing 17.10: SAP BTP integration for workflow orchestration.

```
class SAPBTPWorkflowIntegration:
    """Integration with SAP BTP for workflow orchestration."""

    def __init__(self, btp_config: BTPConfig):
        self.object_store = btp_config.object_store  # SAP Object Store
```

```
        self.hana_client = HanaClient(btp_config.hana)
        self.workflow_service = btp_config.workflow_service

    async def execute_workflow(
        self,
        spec: DatasetSpecification,
    ) -> WorkflowExecution:
        """
        Execute Simula workflow on SAP BTP.

        Steps:
        1. Parse specification
        2. Extract schema from HANA Cloud
        3. Build taxonomies
        4. Generate examples
        5. Validate against schema
        6. Upload to HANA Cloud
        7. Trigger downstream training (optional)
        """
        pass

    async def store_dataset_version(
        self,
        version: DatasetVersion,
        examples: list[TrainingExample],
    ) -> str:
        """Store dataset version in SAP Object Store with HANA metadata."""
        # Store examples in Object Store
        object_path = f"simula/{version.version_id}/examples.jsonl"

        # Store metadata in HANA Cloud
        self.hana_client.insert_dataset_version(version)

        return object_path
```

17.6 Regulatory Framework Alignment

These enhancements are designed to align with emerging regulatory guidance on synthetic data:

17.6.1 UK FCA Synthetic Data Expert Group

The Financial Conduct Authority (FCA) Synthetic Data Expert Group identifies nine governance principles[1]:

1. **Transparency** — Enhancement 4 (audit trails, version control)
2. **Explainability** — Enhancement 2 (knowledge anchors, calibration)

[1] FCA, "Synthetic Data Expert Group Report," 2024

3. **Purpose limitation** — Enhancement 4 (declarative specifications)
4. **Data minimization** — Enhancement 1 (targeted generation)
5. **Accuracy** — Enhancement 3 (closed-loop validation)
6. **Security** — SAP BTP integration (access control)
7. **Accountability** — Enhancement 4 (ownership, approval workflows)
8. **Fairness** — Enhancement 3 (adversarial bias detection)
9. **Continuous monitoring** — Enhancement 3 (drift detection)

17.6.2 IEEE Synthetic Data White Paper

The IEEE white paper on synthetic data governance emphasizes[2]:

- **Privacy risk quantification** — Enhancement 3 (adversarial probing)
- **Fidelity-utility tradeoff** — Enhancement 3 (downstream validation)
- **Provenance tracking** — Enhancement 4 (version control)

17.6.3 World Economic Forum Guidelines

The WEF framework for synthetic data in financial services notes[3]:

- Cross-functional governance teams are essential (addressed in Chapter 2)
- Traceability and provenance are foundational (Enhancement 4)
- Continuous monitoring prevents quality degradation (Enhancement 3)

17.7 Implementation Checklist

[ ] Multi-modal taxonomy builder with cross-modal mappings

[ ] Modality-specific complexification metrics

[ ] SAP DOX integration for document extraction

[ ] Domain knowledge anchor injection mechanism

[ ] Vertical-specific complexity calibration

[ ] Closed-loop training feedback integration

[ ] Adversarial verification layer

[ ] SAP AI Core MLOps integration

[2] IEEE, "Synthetic Data: Generation, Validation, and Governance," 2025
[3] WEF, "Synthetic Data for Financial Services," 2024

- [ ] Declarative DSL parser
- [ ] Dataset version control system
- [ ] SAP BTP workflow orchestration
- [ ] HANA Cloud metadata storage

Chapter 18

Text-to-SQL Drift Detection Framework

18.1 Purpose and Scope

This chapter defines a comprehensive drift detection framework for Text-to-SQL pipelines. Drift occurs when the relationship between any of the following artifacts diverges from the expected baseline:

1. **HANA Table Schemas** — The source of truth for database structure
2. **Agent/User Prompts** — Natural language queries from users or agents
3. **Generated SQL** — The Text-to-SQL model output

The framework integrates with:

- Simula training data generation (Chapter 6)
- Spec-drift audit infrastructure (Chapter 4a of clinerules-agents)
- Singapore AI Safety metrics framework (Chapter 5 of singapore-ai-safety)

18.2 Drift Taxonomy

The Text-to-SQL drift taxonomy distinguishes three primary drift sources:

1. **Schema Drift** — Changes to HANA database structure not reflected in training
2. **User Prompt Drift** — Evolution of human user query patterns and vocabulary
3. **Agent Prompt Drift** — Changes in AI agent query generation behavior

18.2.1 Schema Drift Sources (TTS-DRIFT-001, TTS-DRIFT-002)

Definition: Schema drift occurs when the HANA Cloud database structure changes and the Text-to-SQL training data becomes stale or inconsistent.

Rating Criteria for Schema Drift Metrics:

Table 18.1: Text-to-SQL Drift Types by Source

Code	Type	Source	Description	Severity
TTS-DRIFT-001	Schema Drift	Schema	HANA schema changed without training update	CRITICAL
TTS-DRIFT-002	Coverage Drift	Schema	Training no longer covers schema adequately	HIGH
TTS-DRIFT-003	Semantic Drift	User/Agent	Prompt intent no longer aligns with SQL	CRITICAL
TTS-DRIFT-004	Terminology Drift	User	User vocabulary diverged from schema terms	MEDIUM
TTS-DRIFT-005	Complexity Drift	User/Agent	Query complexity differs from training	MEDIUM
TTS-DRIFT-006	Execution Drift	All	SQL execution success rate degraded	CRITICAL
TTS-DRIFT-007	Agent Behavior Drift	Agent	Agent query patterns changed significantly	HIGH
TTS-DRIFT-008	User Behavior Drift	User	User query patterns changed significantly	MEDIUM

Table 18.2: Schema Drift Trigger Events

Event	Weight	Impact	Mitigation
Column dropped	1.0	SQL references fail	Immediate retraining
Column type changed	0.5	Type errors in queries	Schema re-extraction
Column renamed	0.3	Minor query failures	Vocabulary update
Table added	0.2	Missing coverage	Incremental training
FK relationship changed	0.4	Invalid JOINs	Schema refresh

Table 18.3: Schema Drift Metric Quality Assessment

Metric	Accuracy	Coverage	Speed	Overall	Notes
TTS-M01 (SCR)	95%	High	Fast	**A**	Direct column comparison
TTS-M02 (SSS)	90%	High	Fast	**A**	Time-weighted staleness
TTS-M03 (CTMR)	85%	Medium	Medium	**B+**	Requires SQL parsing
TTS-M04 (FKCR)	90%	High	Medium	**A-**	FK metadata required

18.2.2 User Prompt Drift Sources (TTS-DRIFT-004, TTS-DRIFT-008)

Definition: User prompt drift occurs when human users change their query patterns, vocabulary, or complexity expectations over time.

Table 18.4: User Prompt Drift Indicators

Indicator	Description	Metric	Threshold
Vocabulary shift	New terms not in training data	TTS-M07	JSD > 0.15
Query complexity increase	Users asking harder questions	TTS-M11	KL > 0.10
Ambiguity increase	More vague or underspecified prompts	TTS-M08	ARR < 0.75
Domain expansion	Users querying new table areas	TTS-M01	SCR < 0.85
Intent mismatch	Users expect different results	TTS-M06	IPR < 0.90

Rating Criteria for User Prompt Drift Metrics:

Table 18.5: User Prompt Drift Metric Quality Assessment

Metric	Accuracy	Coverage	Speed	Overall	Notes
TTS-M05 (SAS)	80%	High	Slow	**B+**	Embedding-dependent
TTS-M06 (IPR)	85%	High	Slow	**B+**	Requires critic model
TTS-M07 (TDS)	90%	Medium	Fast	**A-**	JSD computation
TTS-M08 (ARR)	75%	Medium	Slow	**B**	Ambiguity detection hard

18.2.3 Agent Prompt Drift Sources (TTS-DRIFT-003, TTS-DRIFT-007)

Definition: Agent prompt drift occurs when AI agents (Cline, copilots, or automated systems) change their query generation behavior.

Table 18.6: Agent Prompt Drift Indicators

Indicator	Description	Metric	Threshold
Template evolution	Agent prompts follow new patterns	TTS-M05	SAS < 0.80
Instruction compliance	Agent adheres to system prompts	TTS-M06	IPR < 0.90
Query structure shift	SQL generation patterns changed	TTS-M11	CDD > 0.10
Tool calling patterns	Different tool invocation sequences	TTS-M09	ESR < 0.95
Context window usage	Agent uses context differently	TTS-M10	RFS < 0.85

Rating Criteria for Agent Prompt Drift Metrics:

18.2.4 Distinguishing User vs Agent Prompts

The framework must distinguish between human user prompts and AI agent prompts for accurate drift attribution:

Classification Method:

Listing 18.1: User vs Agent prompt classification

```
def classify_prompt_source(prompt: str, metadata: dict) -> str:
```

Table 18.7: Agent Prompt Drift Metric Quality Assessment

Metric	Accuracy	Coverage	Speed	Overall	Notes
TTS-M05 (SAS)	85%	High	Slow	**A-**	Effective for agent drift
TTS-M06 (IPR)	90%	High	Slow	**A**	Critical for agents
TTS-M09 (ESR)	95%	High	Fast	**A+**	Direct execution check
TTS-M10 (RFS)	85%	Medium	Slow	**B+**	Result comparison
TTS-M11 (CDD)	80%	Medium	Fast	**B+**	Distribution comparison

Table 18.8: User vs Agent Prompt Classification

Characteristic	User Prompts	Agent Prompts
Variability	High (natural language variety)	Low (templated patterns)
Vocabulary	Domain-specific, informal	Precise, instruction-following
Complexity	Variable (beginner to expert)	Consistent (system-configured)
Ambiguity	Common (underspecified)	Rare (explicit requirements)
Context usage	Limited (single query)	Rich (multi-turn, tools)
Error patterns	Typos, vague terms	Tool calling errors, format issues

```
    """
    Classify prompt as user or agent based on characteristics.

    Returns: "user", "agent", or "unknown"
    """
    # Check metadata first
    if "agent_id" in metadata or "tool_call_id" in metadata:
        return "agent"

    if "user_session_id" in metadata:
        # Additional heuristics for user prompts
        features = {
            "has_typos": detect_typos(prompt),
            "uses_informal_language": detect_informal(prompt),
            "high_variability": compute_perplexity(prompt) > 10,
            "explicit_instructions": has_system_instructions(prompt),
        }

        user_score = (features["has_typos"] +
                      features["uses_informal_language"] +
                      features["high_variability"])
        agent_score = features["explicit_instructions"]

        return "user" if user_score > agent_score else "agent"

    return "unknown"
```

18.2.5 Combined Drift Assessment Matrix

Reading the matrix:

- ✓= Metric is relevant for this drift source

Table 18.9: Comprehensive Drift Assessment Matrix

Metric	Schema	User Prompt	Agent Prompt	Weight
TTS-M01 (SCR)	✓	–	–	15%
TTS-M02 (SSS)	✓	–	–	10%
TTS-M03 (CTMR)	✓	–	–	5%
TTS-M04 (FKCR)	✓	–	–	10%
TTS-M05 (SAS)	–	✓	✓	15%
TTS-M06 (IPR)	–	✓	✓	10%
TTS-M07 (TDS)	–	✓	–	3%
TTS-M08 (ARR)	–	✓	–	5%
TTS-M09 (ESR)	✓	✓	✓	15%
TTS-M10 (RFS)	–	✓	✓	10%
TTS-M11 (CDD)	–	✓	✓	2%
TTS-M12 (TCD)	✓	–	–	–

- Weight = Contribution to composite TTS-EVAL score
- Metrics can be relevant to multiple sources

18.3 Mandatory Drift Metrics

18.3.1 Metric Overview

18.3.2 Schema Drift Metrics (TTS-M01 to TTS-M04)

TTS-M01: Schema Coverage Rate (SCR)

$$\boxed{SCR = \frac{|C_{\text{referenced}}|}{|C_{\text{schema}}|}} \tag{18.1}$$

Variables $C_{\text{referenced}}$ = columns referenced in training examples; C_{schema} = columns in HANA schema

Direction Higher is better

Threshold $SCR \geq 0.85$ required

Drift Link `TTS-DRIFT-002`

TTS-M02: Schema Staleness Score (SSS)

$$\boxed{SSS = \sum_{i \in \Delta_{\text{schema}}} w_i \cdot \text{age}(i)} \tag{18.2}$$

Variables Δ_{schema} = set of schema changes not in training; w_i = importance weight (1.0 column drop, 0.5 type change, 0.3 rename); $\text{age}(i)$ = days since change

Direction Lower is better

Threshold $SSS \leq 7.0$ required (approximately 1 week tolerance)

Drift Link `TTS-DRIFT-001`

Table 18.10: Text-to-SQL Drift Metrics (12 Metrics)

ID	Name	Definition	Drift Link
TTS-M01	Schema Coverage Rate (SCR)	Fraction of HANA columns referenced in training	TTS-DRIFT-002
TTS-M02	Schema Staleness Score (SSS)	Age-weighted schema changes not in training	TTS-DRIFT-001
TTS-M03	Column Type Mismatch Rate (CTMR)	Fraction of SQL with type-incompatible ops	TTS-DRIFT-001
TTS-M04	Foreign Key Consistency Rate (FKCR)	Fraction of JOINs following valid FKs	TTS-DRIFT-001
TTS-M05	Semantic Alignment Score (SAS)	Cosine similarity prompt-to-SQL embeddings	TTS-DRIFT-003
TTS-M06	Intent Preservation Rate (IPR)	Fraction of prompts correctly answered	TTS-DRIFT-003
TTS-M07	Terminology Drift Score (TDS)	Vocabulary divergence from schema terms	TTS-DRIFT-004
TTS-M08	Ambiguity Resolution Rate (ARR)	Fraction of ambiguous prompts disambiguated	TTS-DRIFT-003
TTS-M09	Execution Success Rate (ESR)	Fraction of SQL executing without error	TTS-DRIFT-006
TTS-M10	Result Fidelity Score (RFS)	Semantic similarity expected vs actual results	TTS-DRIFT-006
TTS-M11	Complexity Distribution Drift (CDD)	KL divergence training vs production complexity	TTS-DRIFT-005
TTS-M12	Taxonomy Coverage Drift (TCD)	Change in taxonomy node coverage over time	TTS-DRIFT-002

TTS-M03: Column Type Mismatch Rate (CTMR)

$$\boxed{CTMR = \frac{N_{\text{type-incompatible operations}}}{N_{\text{total column references}}}} \tag{18.3}$$

Direction Lower is better

Threshold $CTMR \leq 0.02$ required

Drift Link `TTS-DRIFT-001`

TTS-M04: Foreign Key Consistency Rate (FKCR)

$$\boxed{FKCR = \frac{N_{\text{valid FK joins}}}{N_{\text{total joins}}}} \tag{18.4}$$

Direction Higher is better

Threshold $FKCR \geq 0.95$ required

Drift Link `TTS-DRIFT-001`

18.3.3 Semantic Drift Metrics (TTS-M05 to TTS-M08)

TTS-M05: Semantic Alignment Score (SAS)

$$\boxed{SAS = \frac{1}{N}\sum_{j=1}^{N}\cos(E^{j}_{\text{prompt}}, E^{j}_{\text{sql}})} \tag{18.5}$$

Variables E_{prompt} = embedding of natural language prompt; E_{sql} = embedding of SQL intent (parsed query structure); cos() = cosine similarity

Direction Higher is better

Threshold $SAS \geq 0.80$ required

Drift Link `TTS-DRIFT-003`

TTS-M06: Intent Preservation Rate (IPR)

$$\boxed{IPR = \frac{N_{\text{correctly answered prompts}}}{N_{\text{total prompts evaluated}}}} \tag{18.6}$$

Direction Higher is better

Threshold $IPR \geq 0.90$ required

Drift Link `TTS-DRIFT-003`

TTS-M07: Terminology Drift Score (TDS)

$$\boxed{TDS = JSD(P_{\text{prompt_vocab}} \| P_{\text{schema_vocab}})} \tag{18.7}$$

Variables $P_{\text{prompt_vocab}}$ = term probability distribution in prompts; $P_{\text{schema_vocab}}$ = term distribution in schema (columns, tables); JSD = Jensen-Shannon Divergence

Direction Lower is better

Threshold $TDS \leq 0.15$ required

Drift Link `TTS-DRIFT-004`

TTS-M08: Ambiguity Resolution Rate (ARR)

$$\boxed{ARR = \frac{N_{\text{ambiguous prompts correctly resolved}}}{N_{\text{ambiguous prompts total}}}} \tag{18.8}$$

Direction Higher is better

Threshold $ARR \geq 0.75$ required

Drift Link `TTS-DRIFT-003`

18.3.4 Generation Quality Drift Metrics (TTS-M09 to TTS-M12)

TTS-M09: Execution Success Rate (ESR)

$$\boxed{ESR = \frac{N_{\text{SQL executed successfully}}}{N_{\text{SQL generated}}}} \tag{18.9}$$

Direction Higher is better

Threshold $ESR \geq 0.95$ required

Drift Link `TTS-DRIFT-006`

TTS-M10: Result Fidelity Score (RFS)

$$\boxed{RFS = \frac{1}{N}\sum_{j=1}^{N} \text{sim}(R^j_{\text{expected}}, R^j_{\text{actual}})} \tag{18.10}$$

Variables R_{expected} = expected query result (ground truth); R_{actual} = actual query result; sim() = semantic similarity

Direction Higher is better

Threshold $RFS \geq 0.85$ required

Drift Link `TTS-DRIFT-006`

TTS-M11: Complexity Distribution Drift (CDD)

$$\boxed{CDD = KL(P_{\text{train_complexity}} \| P_{\text{prod_complexity}})} \tag{18.11}$$

Variables Using Elo-calibrated complexity scores from Algorithm 3 (Chapter 8)

Direction Lower is better

Threshold $CDD \leq 0.10$ required

Drift Link `TTS-DRIFT-005`

TTS-M12: Taxonomy Coverage Drift (TCD)

$$\boxed{TCD = |LRC_{\text{baseline}} - LRC_{\text{current}}|} \tag{18.12}$$

Variables LRC = Level Ratio Coverage from Chapter 8

Direction Lower is better

Threshold $TCD \leq 0.05$ required (5% coverage change tolerance)

Drift Link `TTS-DRIFT-002`

18.3.5 Statistical Foundation Metrics (TTS-M13 to TTS-M15)

These metrics provide rigorous statistical foundations for drift detection, enabling hypothesis testing and multi-metric validation.

TTS-M13: Wasserstein Distance (W1D)

The **Wasserstein distance** (Earth Mover's Distance) quantifies the minimum "work" needed to transform the synthetic distribution Q into the real distribution P:

$$\boxed{W_1(P, Q) = \int_{-\infty}^{\infty} |F_P(x) - F_Q(x)| \, dx} \tag{18.13}$$

Variables F_P, F_Q = cumulative distribution functions of real and synthetic data respectively

Direction Lower is better

Threshold $W_{1,\text{norm}} \leq 0.10$ required (normalized to [0,1])

Implementation `scipy.stats.wasserstein_distance()`

Use Cases Continuous variables, complexity scores, numeric columns

Drift Link All drift types (cross-cutting)

Normalization formula:

$$W_{1,\text{norm}} = \frac{W_1(P, Q)}{\max(|x_{\max} - x_{\min}|, \epsilon)} \tag{18.14}$$

TTS-M14: Population Stability Index (PSI)

The **PSI** measures how a distribution has shifted across predefined bins, widely used in financial risk modeling for scorecard stability:

$$\boxed{PSI = \sum_{i=1}^{k} (P_i - Q_i) \cdot \ln\left(\frac{P_i}{Q_i}\right)} \tag{18.15}$$

Variables P_i = proportion in bin i for real data; Q_i = proportion in bin i for synthetic data; k = number of bins (typically 10)

Direction Lower is better

Threshold Industry standard:

- $PSI < 0.10$ — No significant drift
- $0.10 \leq PSI < 0.25$ — Moderate drift (investigate)
- $PSI \geq 0.25$ — Significant drift (action required)

Implementation Custom implementation with Laplace smoothing

Use Cases Categorical columns, discretized continuous variables, enums

Drift Link `TTS-DRIFT-001`, `TTS-DRIFT-002`

Laplace smoothing to avoid division by zero:

$$P_i' = \frac{P_i \cdot n + \alpha}{n + k\alpha}, \quad \alpha = 0.001 \tag{18.16}$$

TTS-M15: Kolmogorov-Smirnov Statistic (KSS)

The **KS test** is a non-parametric test measuring the maximum distance between two empirical CDFs:

$$\boxed{D_{KS} = \sup_x |F_P(x) - F_Q(x)|} \tag{18.17}$$

Variables F_P, F_Q = empirical CDFs; sup = supremum (maximum)

Direction Lower D_{KS} is better; higher p-value is better

Threshold p-value > 0.05 required (no significant drift at 95% confidence)

Implementation `scipy.stats.ks_2samp()`

Use Cases Continuous variables, location and shape differences

Drift Link `TTS-DRIFT-005`, `TTS-DRIFT-006`

Decision rule:

$$\text{Drift detected} \iff p\text{-value} \leq 0.05 \text{ (reject } H_0 : P = Q) \tag{18.18}$$

18.3.6 Statistical Metric Comparison

Table 18.11: Statistical Metrics Comparison

Metric	Applicability	Sensitivity	Theoretical Basis	Speed
W1D	Continuous, ordinal	Proportional to shifts	Optimal transport theory	Medium
PSI	Categorical, binned	Practical thresholds	KL-divergence variant	Fast
KSS	Continuous	Location & shape	Non-parametric CDF test	Fast
JSD (M07)	Both	Center-sensitive	Symmetric KL-divergence	Fast

18.4 Composite TTS-EVAL Score

The **TTS Evaluation Score** combines all drift metrics into a single composite score, following the pattern established in singapore-ai-safety:

$$\boxed{\begin{aligned} TTS_EVAL = 100 \times \Big(&0.15 \cdot SCR + 0.10 \cdot (1 - SSS_{\text{norm}}) + 0.10 \cdot FKCR \\ &+ 0.15 \cdot SAS + 0.10 \cdot IPR + 0.15 \cdot ESR \\ &+ 0.10 \cdot RFS + 0.05 \cdot ARR\Big) \\ -100 \times \Big(&0.05 \cdot CTMR + 0.03 \cdot TDS + 0.02 \cdot CDD\Big) \end{aligned}} \tag{18.19}$$

where $SSS_{\text{norm}} = \min(SSS/14.0, 1.0)$ normalizes staleness to a 0–1 range (14 days = maximum acceptable staleness).

18.4.1 TTS-EVAL Interpretation

Table 18.12: TTS-EVAL Score Interpretation

Score Band	Status	Action
$TTS_EVAL \geq 85$	GREEN	Production ready, no action
$70 \leq TTS_EVAL < 85$	AMBER	Investigate, schedule remediation
$55 \leq TTS_EVAL < 70$	AMBER	Urgent remediation required
$TTS_EVAL < 55$	RED	Kill-switch trigger, halt production

18.5 Statistical Evaluation Rules

Following the singapore-ai-safety specification, all metrics use the **95% Wilson Lower Confidence Bound** for "higher is better" metrics:

$$\boxed{LCB_{95} = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \quad \text{where } z = 1.96} \tag{18.20}$$

For "lower is better" metrics (CTMR, TDS, CDD, SSS), use the **95% Wilson Upper Confidence Bound**.

Production gate rule: A metric passes only if the confidence-bounded value meets the threshold.

18.6 Minimum Sample Sizes

18.7 User Population Sampling Strategy

Listing 18.2: User population sampler (simula_tts_drift_sampler.py)

```
@dataclass
class SamplingConfig:
    """Configuration for stratified user sampling."""

```

Table 18.13: Minimum Sample Sizes for TTS Drift Detection

Evaluation Context	Min Queries	Repetitions	Total	Frequency
Training Data Generation	500	1	500	Per batch
CI/CD Gate	100	3	300	Per PR
Production Monitoring	1000	1	1000	Daily
User Population Sampling	See Table 18.14	–	–	Rolling

Table 18.14: Stratified Sampling by User Population

Segment	Definition	Sample Rate	Frequency	Focus Metrics
Power Users	> 100 queries/day	100%	Real-time	All TTS-M01–M12
Regular Users	10–100/day	20%	Hourly	TTS-M05, M06, M09, M10
Occasional Users	< 10/day	5%	Daily	TTS-M09, M10
New Users	First 30 days	50%	Real-time	TTS-M06, M08 (ambiguity)

```
    power_user_threshold: int = 100  # queries/day
    regular_user_min: int = 10
    new_user_window_days: int = 30

    sampling_rates: dict[str, float] = field(default_factory=lambda: {
        "power_user": 1.0,
        "regular_user": 0.2,
        "occasional_user": 0.05,
        "new_user": 0.5,
    })

class UserPopulationSampler:
    """Stratified sampling for drift detection across user populations."""

    def __init__(self, config: SamplingConfig):
        self.config = config
        self._user_query_counts: dict[str, int] = {}
        self._user_first_seen: dict[str, datetime] = {}

    def classify_user(self, user_id: str) -> str:
        """Classify user into population segment."""
        first_seen = self._user_first_seen.get(user_id)
        if first_seen and (datetime.now() - first_seen).days <
    self.config.new_user_window_days:
            return "new_user"

```

```
        daily_count = self._user_query_counts.get(user_id, 0)
        if daily_count > self.config.power_user_threshold:
            return "power_user"
        elif daily_count >= self.config.regular_user_min:
            return "regular_user"
        return "occasional_user"

    def should_sample(self, user_id: str) -> bool:
        """Determine if this query should be sampled for drift analysis."""
        segment = self.classify_user(user_id)
        rate = self.config.sampling_rates[segment]
        return random.random() < rate
```

18.8 Training-Time Drift Evaluation

During Simula training data generation, drift metrics are computed to ensure generated examples remain aligned with the current HANA schema:

Listing 18.3: Training-time drift evaluator (simula_tts_drift_evaluator.py)

```
@dataclass
class TTSMetricReport:
    """Complete drift metric report."""

    # Schema drift metrics
    scr: float  # Schema Coverage Rate
    sss: float  # Schema Staleness Score
    ctmr: float  # Column Type Mismatch Rate
    fkcr: float  # Foreign Key Consistency Rate

    # Semantic drift metrics
    sas: float  # Semantic Alignment Score
    ipr: float  # Intent Preservation Rate
    tds: float  # Terminology Drift Score
    arr: float  # Ambiguity Resolution Rate

    # Generation quality metrics
    esr: float  # Execution Success Rate
    rfs: float  # Result Fidelity Score
    cdd: float  # Complexity Distribution Drift
    tcd: float  # Taxonomy Coverage Drift

    # Composite score
    tts_eval: float

    # Metadata
    evaluated_at: str
    sample_size: int
    confidence_level: float = 0.95

class TTSTrainingEvaluator:
    """Evaluate drift metrics during training data generation."""
```

```python

    def __init__(
        self,
        schema_registry: SchemaRegistry,
        embedding_model: str = "sentence-transformers/all-MiniLM-L6-v2",
    ):
        self.schema_registry = schema_registry
        self.embedding_model = embedding_model

    async def evaluate_batch(
        self,
        examples: list[TrainingExample],
        baseline: Optional[TTSMetricReport] = None,
    ) -> TTSMetricReport:
        """
        Evaluate drift metrics for a batch of training examples.

        Args:
            examples: Training examples to evaluate
            baseline: Optional baseline for drift comparison

        Returns:
            Complete drift metric report
        """
        # Schema drift metrics
        scr = self._compute_schema_coverage(examples)
        sss = self._compute_schema_staleness()
        ctmr = self._compute_type_mismatch_rate(examples)
        fkcr = self._compute_fk_consistency(examples)

        # Semantic drift metrics
        sas = await self._compute_semantic_alignment(examples)
        ipr = await self._compute_intent_preservation(examples)
        tds = self._compute_terminology_drift(examples)
        arr = await self._compute_ambiguity_resolution(examples)

        # Generation quality metrics
        esr = self._compute_execution_success(examples)
        rfs = await self._compute_result_fidelity(examples)

        # Distribution drift (requires baseline)
        cdd = 0.0
        tcd = 0.0
        if baseline:
            cdd = self._compute_complexity_drift(examples, baseline)
            tcd = self._compute_taxonomy_drift(examples, baseline)

        # Composite score
        tts_eval = self._compute_tts_eval(
            scr, sss, ctmr, fkcr, sas, ipr, tds, arr, esr, rfs, cdd
        )
```

```python

        return TTSMetricReport(
            scr=scr, sss=sss, ctmr=ctmr, fkcr=fkcr,
            sas=sas, ipr=ipr, tds=tds, arr=arr,
            esr=esr, rfs=rfs, cdd=cdd, tcd=tcd,
            tts_eval=tts_eval,
            evaluated_at=datetime.now().isoformat(),
            sample_size=len(examples),
        )

    def _compute_schema_coverage(self, examples: list[TrainingExample]) -> float:
        """Compute Schema Coverage Rate (SCR)."""
        schema_columns = set()
        for table in self.schema_registry.get_all_tables():
            for column in table.columns:
                schema_columns.add(f"{table.table_name}.{column.name}")

        referenced_columns = set()
        for example in examples:
            # Parse SQL to extract column references
            columns = self._extract_columns_from_sql(example.sql)
            referenced_columns.update(columns)

        if not schema_columns:
            return 1.0
        return len(referenced_columns & schema_columns) / len(schema_columns)

    def _compute_tts_eval(
        self,
        scr: float, sss: float, ctmr: float, fkcr: float,
        sas: float, ipr: float, tds: float, arr: float,
        esr: float, rfs: float, cdd: float,
    ) -> float:
        """Compute composite TTS-EVAL score."""
        # Normalize SSS to 0-1 range (14 days max)
        sss_norm = min(sss / 14.0, 1.0)

        # Positive contributions
        positive = (
            0.15 * scr +
            0.10 * (1 - sss_norm) +
            0.10 * fkcr +
            0.15 * sas +
            0.10 * ipr +
            0.15 * esr +
            0.10 * rfs +
            0.05 * arr
        )

```

```
        # Negative contributions (penalties)
        negative = (
            0.05 * ctmr +
            0.03 * tds +
            0.02 * cdd
        )

        return 100 * positive - 100 * negative
```

18.9 Production Monitoring

Listing 18.4: Production drift monitor (simula_tts_drift_monitor.py)

```
@dataclass
class DriftAlert:
    """Alert for detected drift."""

    alert_id: str
    severity: str  # "LOW", "MEDIUM", "HIGH", "CRITICAL"
    drift_type: str  # TTS-DRIFT-001 to TTS-DRIFT-006
    metric_code: str  # TTS-M01 to TTS-M12
    current_value: float
    threshold: float
    baseline_value: float
    delta: float
    timestamp: str
    user_segment: Optional[str] = None
    sample_queries: list[str] = field(default_factory=list)

class TTSProductionMonitor:
    """Monitor drift in production Text-to-SQL pipeline."""

    def __init__(
        self,
        baseline: TTSMetricReport,
        drift_threshold: float = 0.10,
        alert_callback: Optional[Callable[[DriftAlert], None]] = None,
    ):
        self.baseline = baseline
        self.drift_threshold = drift_threshold
        self.alert_callback = alert_callback
        self._rolling_metrics: dict[str, deque] = defaultdict(lambda:
    deque(maxlen=1000))

    async def monitor_query(
        self,
        prompt: str,
        generated_sql: str,
        execution_result: Optional[Any],
        user_id: str,
    ) -> Optional[DriftAlert]:
```

```
        """
        Monitor a single query for drift signals.

        Args:
            prompt: Natural language prompt
            generated_sql: Generated SQL query
            execution_result: Result of SQL execution (None if failed)
            user_id: User identifier for segmentation

        Returns:
            DriftAlert if drift detected, None otherwise
        """
        # Track execution success
        execution_success = execution_result is not None
        self._rolling_metrics["esr"].append(1.0 if execution_success else 0.0)

        # Compute rolling ESR
        current_esr = sum(self._rolling_metrics["esr"]) / len(self._rolling_metrics["esr"])
        esr_delta = abs(current_esr - self.baseline.esr)

        # Check for significant drift
        if esr_delta > self.drift_threshold:
            alert = DriftAlert(
                alert_id=str(uuid.uuid4()),
                severity="CRITICAL" if esr_delta > 0.2 else "HIGH",
                drift_type="TTS-DRIFT-006",
                metric_code="TTS-M09",
                current_value=current_esr,
                threshold=0.95,
                baseline_value=self.baseline.esr,
                delta=esr_delta,
                timestamp=datetime.now().isoformat(),
                user_segment=self._classify_user(user_id),
            )

            if self.alert_callback:
                self.alert_callback(alert)

            return alert

        return None

    def get_rolling_report(self) -> TTSMetricReport:
        """Get drift report based on rolling window of queries."""
        # Aggregate rolling metrics
        esr = sum(self._rolling_metrics["esr"]) / max(len(self._rolling_metrics["esr"]), 1)

        # Return partial report with available metrics
```

```
        return TTSMetricReport(
            scr=self.baseline.scr,  # Use baseline for schema metrics
            sss=self.baseline.sss,
            ctmr=self.baseline.ctmr,
            fkcr=self.baseline.fkcr,
            sas=self.baseline.sas,
            ipr=self.baseline.ipr,
            tds=self.baseline.tds,
            arr=self.baseline.arr,
            esr=esr,
            rfs=self.baseline.rfs,
            cdd=0.0,  # Requires full evaluation
            tcd=0.0,
            tts_eval=self._estimate_tts_eval(esr),
            evaluated_at=datetime.now().isoformat(),
            sample_size=len(self._rolling_metrics["esr"]),
        )
```

18.9.1 Schema Drift Agent and Retraining Trigger

A dedicated drift agent monitors versioned HANA schema snapshots, dictionary versions, and training artifact versions. The agent must compare the live schema registry against the latest approved training baseline on every scheduled check and on every schema registry commit.
Every trigger must write an auditable event containing the schema version, training artifact version, dictionary version, triggering metric, owner, and retraining decision. Retraining is blocked until the readiness assessment in Section 8.7 returns GREEN for the target audience or AMBER with an explicit waiver for agent-only publication.

18.9.2 Monitoring UI Requirements

The operations UI must expose schema, specification, and vocabulary drift as separate surfaces instead of collapsing them into one health indicator. The minimum dashboard fields are:

- current schema version, latest training schema version, and schema delta count;
- TTS-EVAL trend, per-metric failures, and confidence-bounded pass/fail state;
- unresolved vocabulary terms, dictionary miss rate, and top affected locales;
- human vs. agent readiness grade and artifact publication status;
- active retraining jobs, blocked deployments, and owner assignment.

18.10 CI/CD Integration

18.10.1 Training Data Generation Gate

The drift evaluator integrates with Simula's generation pipeline as a quality gate:

Table 18.15: Drift Agent Trigger Rules

Trigger	**Severity**	**Required Action**	**Retraining Scope**
Column dropped or type changed	CRITICAL	Halt affected query patterns, open schema-drift incident	Full affected domain
Column renamed or table added	HIGH	Re-extract schema, regenerate affected taxonomy nodes	Targeted incremental
Dictionary value added or changed	HIGH	Regenerate synonym/value mapping tests	Dictionary slice
Vocabulary drift over threshold	MEDIUM	Generate new human-facing examples and update aliases	Persona or locale slice
Agent prompt drift over threshold	MEDIUM	Refresh agent templates and re-run audience separation checks	Agent slice

Listing 18.5: CI/CD gate for TTS drift

```yaml
# In GitHub Actions workflow
- name: Run TTS Drift Evaluation
  run: |
    python -m src.training.pipeline.simula_tts_drift_evaluator \
      --examples data/hana_generated/*.jsonl \
      --schema-registry docs/schema/simula/hana-schema-entry.schema.json \
      --baseline data/baselines/tts_baseline.json \
      --output reports/tts_drift_report.json

- name: Check TTS-EVAL Score
  run: |
    TTS_EVAL=$(jq '.tts_eval' reports/tts_drift_report.json)
    if (( $(echo "$TTS_EVAL < 70" | bc -l) )); then
      echo "TTS-EVAL score $TTS_EVAL below threshold 70"
      exit 1
    fi
```

18.10.2 Drift Alert Workflow

Table 18.16: Drift Alert Response Actions

Severity	TTS-EVAL	Action	Timeline
CRITICAL	< 55	Kill-switch, halt production, page on-call	Immediate
HIGH	55–70	Block deployments, create P1 ticket	4 hours
MEDIUM	70–85	Create P2 ticket, schedule remediation	1 week
LOW	≥ 85	Log for tracking, no action required	–

18.11 HANA Cloud Telemetry Tables

Drift metrics are persisted to HANA Cloud for historical analysis:

Listing 18.6: TTS Drift telemetry tables

```sql
-- See docs/hana-cloud/TTS_DRIFT_TABLES_DDL.sql for complete DDL

-- Drift metrics history
CREATE COLUMN TABLE "FINSIGHT_GOV"."TTS_DRIFT_METRICS" (
    "METRIC_ID" NVARCHAR(64) NOT NULL,
    "TIMESTAMP" TIMESTAMP NOT NULL,
    "METRIC_CODE" NVARCHAR(20) NOT NULL,  -- TTS-M01 to TTS-M12
    "VALUE" DECIMAL(10,6),
    "THRESHOLD" DECIMAL(10,6),
    "PASSED" BOOLEAN,
    "USER_SEGMENT" NVARCHAR(50),
    "BATCH_ID" NVARCHAR(64),
    PRIMARY KEY ("METRIC_ID")
);

```

```sql
-- Schema snapshots for staleness tracking
CREATE COLUMN TABLE "FINSIGHT_GOV"."TTS_SCHEMA_SNAPSHOTS" (
    "SNAPSHOT_ID" NVARCHAR(64) NOT NULL,
    "SCHEMA_NAME" NVARCHAR(100),
    "TABLE_NAME" NVARCHAR(100),
    "COLUMN_HASH" NVARCHAR(64),
    "SNAPSHOT_AT" TIMESTAMP,
    "TRAINING_SYNC_AT" TIMESTAMP,
    PRIMARY KEY ("SNAPSHOT_ID")
);
```

18.12 Stochastic Surprise Gate (SSG) Framework

The SSG framework provides a unified, statistically rigorous approach to drift detection across all four drift areas. It treats each drift-sensitive artifact as a **time series** and uses a calibrated surprise score (Z-score) based on a Hull-White mean-reverting stochastic differential equation.

18.12.1 Mathematical Foundation

The SSG models the drift-sensitive observable x_t as evolving according to:

$$\boxed{dx_t = a(\theta(t) - x_t)dt + \sigma dW_t} \tag{18.21}$$

Variables a = mean reversion speed; $\theta(t)$ = time-dependent long-run mean; σ = volatility; W_t = Wiener process

Discretization Euler-Maruyama: $\hat{x}_{t+1} = x_t + a(\theta(t) - x_t)\Delta t$

18.12.2 Surprise Score Calculation

Step 1: Compute Residual (Innovation)

After observing x_t, the residual is:

$$\epsilon_t = x_t - \hat{x}_t \tag{18.22}$$

Step 2: Update Volatility Estimate

Use exponentially weighted moving variance:

$$\boxed{\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)\epsilon_t^2} \tag{18.23}$$

where $\lambda \in [0.95, 0.99]$ is the forgetting factor.

Step 3: Compute Z-Score (Surprise)

$$\boxed{Z_t = \frac{|\epsilon_t|}{\max(\sigma_t, \sigma_{\min})}} \tag{18.24}$$

where $\sigma_{\min} = 10^{-9}$ prevents division by zero.

Step 4: Apply Surprise Gate

$$\text{Drift detected} \iff Z_t > Z_{\text{threshold}} \tag{18.25}$$

Default $Z_{\text{threshold}} = 2.0$ (corresponds to $\approx 95\%$ confidence).

Step 5: Convert to p-Value

For statistical reporting, convert Z-score to two-tailed p-value:

$$\boxed{p = \operatorname{erfc}\left(\frac{Z_t}{\sqrt{2}}\right)} \tag{18.26}$$

18.12.3 SSG Surprise Metrics (TTS-M16 to TTS-M19)

Table 18.17: Stochastic Surprise Gate Metrics

ID	Name	Observable	Application
`TTS-M16`	Codebase Surprise Score (CSS)	Architecture constraint violations	Agent code drift
`TTS-M17`	Schema Surprise Score (SSS-HW)	Schema dissimilarity count	HANA schema drift
`TTS-M18`	Prompt Surprise Score (PSS)	Embedding distance from centroid	User/agent prompts
`TTS-M19`	Token Surprise Score (TSS)	$-\log_2 P(\text{token})/H$	TOON token stream

TTS-M16: Codebase Surprise Score (CSS)

For AI coding agents, measure deviation from architectural specification:

$$\boxed{Z_t^{\text{code}} = \frac{|e_t - \hat{e}_t|}{\sigma_t^{\text{code}}}} \tag{18.27}$$

Observable e_t Vector of: import violations, module boundary violations, file length violations, pattern mismatch counts

Predictive Model Hull-White with $\theta = 0$ (ideal state has zero violations)

Threshold $Z^{\text{code}} > 2.0$ triggers CI block

Mitigation Refresh `AGENTS.md`, request human review

TTS-M17: Schema Surprise Score (SSS-HW)

For HANA schema drift against Simula generator expectations:

$$\boxed{Z_t^{\text{schema}} = \frac{|d_t - \hat{d}_t|}{\sigma_t^{\text{schema}}}} \tag{18.28}$$

Observable d_t Dissimilarity vector: mismatched columns, type changes, missing indices, FK violations

Predictive Model Hull-White with time-dependent $\theta(t) = \theta_0 + \beta t$ to accommodate planned migrations

Threshold $Z^{\text{schema}} > 2.0$ triggers generator freeze

Mitigation Auto-adjust generator spec or emit alert for manual alignment

TTS-M18: Prompt Surprise Score (PSS)

For user/agent prompt drift from training distribution:

$$\boxed{Z_t^{\text{prompt}} = \frac{\|\text{embedding}_t - \mathbb{E}[\text{embedding}_t]\|_2}{\sigma_t^{\text{prompt}}}} \tag{18.29}$$

Observable Euclidean distance between prompt embedding and training centroid

Predictive Model Moving average of embeddings with mean-reverting dynamic

Threshold $Z^{\text{prompt}} > 2.0$ indicates out-of-distribution prompt

Mitigation Switch to fallback template, log for retraining

TTS-M19: Token Surprise Score (TSS)

For TOON token stream drift:

$$\boxed{Z_t^{\text{toon}} = \frac{S_t - \mathbb{E}[S]}{\sigma_t^{\text{toon}}} \quad \text{where} \quad S_t = \frac{-\log_2 P(\text{token}_t)}{H}} \tag{18.30}$$

Observable S_t Normalized token surprise (entropy-scaled negative log probability)

Token Model Adaptive n-gram with exponential forgetting

Threshold $Z^{\text{toon}} > 2.0$ indicates unexpected structure

Mitigation Trigger DSPy re-optimization, insert correction token

18.12.4 Significance Classification

Table 18.18: Z-Score Significance Levels

Z-Score Range	p-Value	Classification	Action
$Z \leq 1.0$	≥ 0.317	Normal	No action
$1.0 < Z \leq 2.0$	0.046–0.317	Noteworthy	Log for monitoring
$2.0 < Z \leq 3.0$	0.003–0.046	Significant	Trigger mitigation
$Z > 3.0$	< 0.003	Extreme	Immediate escalation

18.12.5 Online Calibration

The SSG uses recursive least squares (RLS) with exponential forgetting to adapt model parameters:

Listing 18.7: Online calibrator for Hull-White parameters

```python
@dataclass
class OnlineCalibrator:
    """Recursive calibrator for Hull-White model parameters."""

    forgetting_factor: float = 0.98  # lambda

    # Parameter estimates
    mean_reversion_estimate: float = 0.5
    long_run_mean_estimate: float = 0.0
    volatility_estimate: float = 0.1

    # RLS state
    _P: float = 1.0  # Covariance estimate

    def update(self, x_t: float, x_hat: float, epsilon: float):
        """Update parameter estimates using new observation."""
        # Update covariance with forgetting
        self._P = (1 / self.forgetting_factor) * self._P

        # Compute Kalman gain
        K = self._P / (1 + self._P)

        # Update parameter estimates
        self.mean_reversion_estimate += K * epsilon * (x_t -
    self.long_run_mean_estimate)

        # Update volatility using EWMA variance
        self.volatility_estimate = math.sqrt(
            self.forgetting_factor * self.volatility_estimate**2 +
            (1 - self.forgetting_factor) * epsilon**2
        )

        # Update covariance
        self._P = (1 - K) * self._P
```

18.12.6 Audit Trail Structure

Every surprise gate decision produces an immutable audit record:

Listing 18.8: Surprise audit trail schema

```python
@dataclass
class SurpriseAuditTrail:
    """Immutable audit record for surprise gate decisions."""

    # Identifiers
    audit_id: str
```

```
    timestamp: str
    observable_type: str  # "code", "schema", "prompt", "toon"

    # Observation
    observed_value: float
    predicted_value: float
    residual: float

    # Surprise metrics
    z_score: float
    p_value: float
    significance: str  # "normal", "noteworthy", "significant", "extreme"
    trigger: bool

    # Model state at decision time
    hull_white_state: dict[str, float]  # a, theta, sigma
    calibrator_version: str

    # Action taken
    mitigation_action: Optional[str]

    # Governance
    compliant_with: list[str]  # e.g., ["MAS-TRH-2024", "ISO-42001"]
```

18.13 Future Directions: TOON and DSPy Integration

Future Work

v1.3+ Scope
The following integrations are planned for future releases and are explicitly out of scope for v1.2 implementation.

18.13.1 Token-Oriented Object Notation (TOON)

TOON provides structured metadata that travels with AI-generated content. For Text-to-SQL drift detection, TOON metadata enables:

- **Provenance tracking** — Schema version, training data version, model version
- **Drift signals** — SCR at generation time, SAS at inference time
- **Governance** — Drift score, last check timestamp, alert status

Listing 18.9: TOON drift metadata structure

```
@dataclass
class TOONDriftMetadata:
    """TOON-compliant drift metadata for Text-to-SQL."""

```

```
    # Provenance
    schema_version: str
    training_data_version: str
    model_version: str

    # Drift Signals
    scr_at_generation: float
    sas_at_inference: float
    complexity_band: str  # "easy", "medium", "hard"

    # Traceability
    taxonomy_path: list[str]
    schema_tables_used: list[str]

    # Governance
    drift_score: float
    last_drift_check: str
    drift_alert_status: str  # "green", "amber", "red"
```

18.13.2 DSPy Integration

DSPy (Declarative Self-improving Language Programs) enables:

- **Programmatic prompt optimization** — Tune prompts based on drift metrics
- **Modular composition** — Separate schema extraction, intent, SQL generation
- **Self-correction** — Retry with refined prompts when drift detected

Listing 18.10: DSPy module with drift guard (future)

```
import dspy

class TextToSQLWithDriftGuard(dspy.Module):
    """DSPy module with integrated drift detection."""

    def __init__(self, schema_registry: SchemaRegistry):
        super().__init__()
        self.schema_extractor = dspy.ChainOfThought(
            "question -> relevant_tables, relevant_columns"
        )
        self.sql_generator = dspy.ChainOfThought(
            "question, schema_context -> sql_query"
        )
        self.drift_detector = dspy.Predict(
            "sql_query, schema_context -> drift_score, drift_reasons"
        )
        self.schema_registry = schema_registry

    def forward(self, question: str):
        # Step 1: Schema extraction
```

```
        schema_pred = self.schema_extractor(question=question)

        # Step 2: SQL generation
        sql_pred = self.sql_generator(
            question=question,
            schema_context=self._get_context(schema_pred.relevant_tables)
        )

        # Step 3: Drift detection
        drift_pred = self.drift_detector(
            sql_query=sql_pred.sql_query,
            schema_context=self._get_context(schema_pred.relevant_tables)
        )

        # Step 4: Self-correction if drift detected
        if drift_pred.drift_score > 0.2:
            sql_pred = self._regenerate_with_guidance(
                question, drift_pred.drift_reasons
            )

        return dspy.Prediction(
            sql=sql_pred.sql_query,
            drift_score=drift_pred.drift_score,
        )
```

18.14 Known Limitations & Roadmap

18.14.1 Review Summary (v1.2)

This specification has been reviewed against the following criteria. **Overall Rating: 7.5/10** — solid foundation with clear taxonomy and mathematical rigor, but requires empirical validation before production deployment.

Table 18.19: Specification Review Scores

Criterion	Score	Assessment
Completeness	7/10	Missing latency/cost metrics
Mathematical Rigor	9/10	Strong statistical foundation
Practicality	6/10	Computational cost concerns
Innovation	8/10	SSG is novel
Governance & Audit	9/10	Excellent audit trail design
Implementation Readiness	5/10	Needs empirical validation
Documentation Quality	8/10	Good cross-references

18.14.2 Known Limitations

18.14.3 Computational Optimizations

To address the computational cost concerns, the following optimizations are provided:

Table 18.20: Known Limitations and Mitigation Status

Area	Issue	Severity	Status
Empirical validation	No results from real HANA data or agent logs	HIGH	v1.3
Threshold justification	Thresholds not derived from error analysis	MEDIUM	v1.3
Computational cost	No overhead analysis for real-time monitoring	MEDIUM	Addressed
User/agent classification	Heuristic rules are brittle	LOW	v1.3
TOON + DSPy integration	Marked as future, conceptual only	MEDIUM	v1.3
Ambiguity resolution (ARR)	Detection method not specified	HIGH	v1.3
Calibration drift	No recovery from SSG calibration failure	MEDIUM	Addressed
Missing metrics	No latency drift or token cost drift	LOW	v2.0

Sliced Wasserstein Approximation

For real-time monitoring, use the **sliced Wasserstein distance** which projects distributions onto random 1-D subspaces:

$$\boxed{SW_1(P, Q) = \frac{1}{L} \sum_{\ell=1}^{L} W_1(P_\ell, Q_\ell)} \tag{18.31}$$

where P_ℓ and Q_ℓ are 1-D projections onto random unit vectors.

Complexity comparison:

- Full Wasserstein: $O(n^3 \log n)$ for exact, $O(n^2)$ for entropic
- Sliced Wasserstein: $O(L \cdot n \log n)$ where $L = 50$ projections typical

Implementation flag:

```
wasserstein_result = StatisticalDriftCalculator.compute_wasserstein(
    p_values, q_values,
    use_fast_approximation=True,  # Use sliced Wasserstein
    num_projections=50,
)
```

Tiered Metric Strategy

Table 18.21: Tiered Metrics by Latency Budget

Tier	Latency	Metrics	Use Case
Real-time	< 10ms	ESR, rolling counts	Per-query monitoring
Fast	< 100ms	PSI, KS, sliced Wasserstein	Batch (per minute)
Full	< 1s	Full Wasserstein, SSG, SAS	Offline analysis

18.14.4 Calibration Health Checks

To detect and recover from SSG calibration failure, the following checks are implemented:

Listing 18.11: Calibration health check

```python
@dataclass
class CalibrationHealth:
    """Health status of SSG calibration."""

    volatility: float
    volatility_z_score: float
    is_healthy: bool
    recovery_action: Optional[str]

class HullWhiteSurpriseGate:

    SIGMA_MIN_THRESHOLD: float = 1e-6
    SIGMA_MAX_THRESHOLD: float = 100.0

    def check_calibration_health(self) -> CalibrationHealth:
        """Check if SSG calibration is healthy."""
        sigma = self.state.volatility

        # Check for implausibly low volatility (collapsed)
        if sigma < self.SIGMA_MIN_THRESHOLD:
            return CalibrationHealth(
                volatility=sigma,
                volatility_z_score=float("inf"),
                is_healthy=False,
                recovery_action="reset_calibrator_sigma_collapsed",
            )

        # Check for implausibly high volatility (exploded)
        if sigma > self.SIGMA_MAX_THRESHOLD:
            return CalibrationHealth(
                volatility=sigma,
                volatility_z_score=float("inf"),
                is_healthy=False,
                recovery_action="reset_calibrator_sigma_exploded",
            )

        return CalibrationHealth(
            volatility=sigma,
            volatility_z_score=0.0,
            is_healthy=True,
            recovery_action=None,
        )

    def auto_recover_calibration(self) -> bool:
        """Auto-recover from calibration failure."""
        health = self.check_calibration_health()
        if not health.is_healthy:
            self.calibrator.reset()
            self.state.volatility = self.initial_volatility
            return True
```

```
        return False
```

18.14.5 Roadmap

Table 18.22: Implementation Roadmap

Version	Target	Deliverables	Priority
v1.2.1	2026-Q2	Sliced Wasserstein, calibration health checks	HIGH
v1.3	2026-Q3	Spider validation, threshold bootstrap, ARR module	HIGH
v1.3	2026-Q3	TOON/DSPy working implementation	MEDIUM
v1.3	2026-Q3	User/agent classification improvement	LOW
v2.0	2026-Q4	TTS-M20 (Latency), TTS-M21 (Token Cost)	MEDIUM
v2.0	2026-Q4	Multivariate Mahalanobis surprise	LOW

18.14.6 Pilot Deployment Strategy

Before production deployment, conduct a pilot with the following parameters:

1. **Traffic sample:** 1% of production HANA queries
2. **Metrics:** Lightweight only (PSI, KS, rolling ESR)
3. **Heavy metrics:** Defer Wasserstein and full SSG to offline batch
4. **Goal:** Collect empirical drift patterns for threshold tuning
5. **Duration:** 4 weeks minimum

18.15 Detailed Implementation Specifications

This section provides implementation-ready specifications for roadmap items.

18.15.1 Sliced Wasserstein Distance Implementation

Algorithm

Algorithm 1 Sliced Wasserstein Distance

Require: Samples $P = \{p_1, \ldots, p_m\}$, $Q = \{q_1, \ldots, q_n\}$, projections L
Ensure: Sliced Wasserstein distance $SW_1(P, Q)$

$sw \leftarrow 0$
for $\ell = 1$ to L **do**
  $\theta_\ell \leftarrow \text{random_unit_vector}(d)$ ▷ Sample direction
  $P_\ell \leftarrow \{\theta_\ell^\top p_i : p_i \in P\}$ ▷ Project P
  $Q_\ell \leftarrow \{\theta_\ell^\top q_j : q_j \in Q\}$ ▷ Project Q
  $P_\ell^{\text{sorted}} \leftarrow \text{sort}(P_\ell)$
  $Q_\ell^{\text{sorted}} \leftarrow \text{sort}(Q_\ell)$
  if $m \neq n$ **then** ▷ Interpolate if sizes differ
    $P_\ell^{\text{interp}}, Q_\ell^{\text{interp}} \leftarrow \text{interpolate_to_common_grid}(P_\ell, Q_\ell)$
    $w_\ell \leftarrow \frac{1}{k} \sum_{i=1}^{k} |P_\ell^{\text{interp}}[i] - Q_\ell^{\text{interp}}[i]|$
  else
    $w_\ell \leftarrow \frac{1}{m} \sum_{i=1}^{m} |P_\ell^{\text{sorted}}[i] - Q_\ell^{\text{sorted}}[i]|$
  end if
  $sw \leftarrow sw + w_\ell$
end for
return sw/L

Python Implementation

Listing 18.12: Sliced Wasserstein implementation

```python
import numpy as np
from typing import Optional

def compute_sliced_wasserstein(
    p_values: np.ndarray,
    q_values: np.ndarray,
    num_projections: int = 50,
    seed: Optional[int] = None,
) -> float:
    """
    Compute sliced Wasserstein distance.

    Args:
        p_values: Array of shape (m,) or (m, d) for d-dimensional data
        q_values: Array of shape (n,) or (n, d)
        num_projections: Number of random projections (L)
        seed: Random seed for reproducibility

    Returns:
        Sliced Wasserstein distance
    """
    rng = np.random.default_rng(seed)
```

```python

    # Handle 1D case
    if p_values.ndim == 1:
        p_values = p_values.reshape(-1, 1)
    if q_values.ndim == 1:
        q_values = q_values.reshape(-1, 1)

    d = p_values.shape[1]
    m, n = len(p_values), len(q_values)

    # Common grid size for interpolation
    grid_size = max(m, n)

    total_distance = 0.0

    for _ in range(num_projections):
        # Sample random direction on unit sphere
        theta = rng.standard_normal(d)
        theta = theta / np.linalg.norm(theta)

        # Project samples
        p_proj = p_values @ theta
        q_proj = q_values @ theta

        # Sort projections
        p_sorted = np.sort(p_proj)
        q_sorted = np.sort(q_proj)

        # Interpolate to common grid if sizes differ
        if m != n:
            p_interp = np.interp(
                np.linspace(0, 1, grid_size),
                np.linspace(0, 1, m),
                p_sorted
            )
            q_interp = np.interp(
                np.linspace(0, 1, grid_size),
                np.linspace(0, 1, n),
                q_sorted
            )
            w_l = np.mean(np.abs(p_interp - q_interp))
        else:
            w_l = np.mean(np.abs(p_sorted - q_sorted))

        total_distance += w_l

    return total_distance / num_projections
```

Table 18.23: Sliced Wasserstein Parameters

Parameter	Default	Range	Rationale
`num_projections`	50	20–200	50 gives $< 5\%$ variance
`seed`	None	Any int	Set for reproducibility in CI
`threshold`	0.10	0.05–0.20	Same as full W1D

Parameters and Thresholds

18.15.2 Calibration Health Check Implementation

Health Check Algorithm

Algorithm 2 SSG Calibration Health Check

Require: SSG state $S = (a, \theta, \sigma_t)$, history window H
Ensure: Health status, recovery action

$\sigma_{\min} \leftarrow 10^{-6}$, $\sigma_{\max} \leftarrow 100.0$
$a_{\min} \leftarrow 0.01$, $a_{\max} \leftarrow 10.0$
if $\sigma_t < \sigma_{\min}$ **then**
    return (`UNHEALTHY`, "sigma_collapsed", `RESET`)
end if
if $\sigma_t > \sigma_{\max}$ **then**
    return (`UNHEALTHY`, "sigma_exploded", `RESET`)
end if
if $a < a_{\min}$ or $a > a_{\max}$ **then**
    return (`UNHEALTHY`, "mean_reversion_invalid", `RESET`)
end if
$\sigma_{\text{history}} \leftarrow [\sigma_{t-k} : k \in H]$ ▷ Collect historical volatility
$\mu_\sigma \leftarrow \text{mean}(\sigma_{\text{history}})$
$\text{sd}_\sigma \leftarrow \text{std}(\sigma_{\text{history}})$
$z_\sigma \leftarrow |\sigma_t - \mu_\sigma| / \max(\text{sd}_\sigma, 10^{-9})$
if $z_\sigma > 3.0$ **then**
    return (`WARNING`, "sigma_drifted", `LOG_ONLY`)
end if
return (`HEALTHY`, None, None)

Python Implementation

Listing 18.13: Calibration health check implementation

```python
from dataclasses import dataclass
from enum import Enum
from typing import Optional, List
from collections import deque

class HealthStatus(Enum):
    HEALTHY = "healthy"
    WARNING = "warning"
    UNHEALTHY = "unhealthy"
```

```python

class RecoveryAction(Enum):
    NONE = "none"
    LOG_ONLY = "log_only"
    RESET = "reset"
    ESCALATE = "escalate"

@dataclass
class CalibrationHealth:
    """Detailed health status of SSG calibration."""

    status: HealthStatus
    reason: Optional[str]
    action: RecoveryAction

    # Current state
    volatility: float
    mean_reversion: float

    # Diagnostic metrics
    volatility_z_score: float
    volatility_history_mean: float
    volatility_history_std: float
    observations_since_reset: int

    def to_audit_record(self) -> dict:
        """Convert to audit trail record."""
        return {
            "status": self.status.value,
            "reason": self.reason,
            "action": self.action.value,
            "volatility": self.volatility,
            "mean_reversion": self.mean_reversion,
            "volatility_z_score": self.volatility_z_score,
            "volatility_history_mean": self.volatility_history_mean,
            "volatility_history_std": self.volatility_history_std,
            "observations_since_reset": self.observations_since_reset,
        }

class CalibrationHealthChecker:
    """Check and maintain SSG calibration health."""

    # Thresholds
    SIGMA_MIN = 1e-6
    SIGMA_MAX = 100.0
    A_MIN = 0.01
    A_MAX = 10.0
    Z_WARNING_THRESHOLD = 3.0

    def __init__(self, history_window: int = 100):
        self.history_window = history_window
```

```
        self._sigma_history: deque = deque(maxlen=history_window)
        self._observations = 0

    def record_observation(self, sigma: float) -> None:
        """Record a volatility observation."""
        self._sigma_history.append(sigma)
        self._observations += 1

    def check_health(
        self,
        sigma: float,
        mean_reversion: float,
    ) -> CalibrationHealth:
        """
        Check calibration health.

        Args:
            sigma: Current volatility estimate
            mean_reversion: Current mean reversion speed

        Returns:
            CalibrationHealth with status and recommended action
        """
        # Check hard bounds on sigma
        if sigma < self.SIGMA_MIN:
            return CalibrationHealth(
                status=HealthStatus.UNHEALTHY,
                reason="sigma_collapsed",
                action=RecoveryAction.RESET,
                volatility=sigma,
                mean_reversion=mean_reversion,
                volatility_z_score=float("inf"),
                volatility_history_mean=0.0,
                volatility_history_std=0.0,
                observations_since_reset=self._observations,
            )

        if sigma > self.SIGMA_MAX:
            return CalibrationHealth(
                status=HealthStatus.UNHEALTHY,
                reason="sigma_exploded",
                action=RecoveryAction.RESET,
                volatility=sigma,
                mean_reversion=mean_reversion,
                volatility_z_score=float("inf"),
                volatility_history_mean=0.0,
                volatility_history_std=0.0,
                observations_since_reset=self._observations,
            )

        # Check mean reversion bounds
```

```
        if mean_reversion < self.A_MIN or mean_reversion > self.A_MAX:
            return CalibrationHealth(
                status=HealthStatus.UNHEALTHY,
                reason="mean_reversion_invalid",
                action=RecoveryAction.RESET,
                volatility=sigma,
                mean_reversion=mean_reversion,
                volatility_z_score=0.0,
                volatility_history_mean=0.0,
                volatility_history_std=0.0,
                observations_since_reset=self._observations,
            )

        # Check volatility stability (need history)
        if len(self._sigma_history) >= 10:
            import statistics
            mu = statistics.mean(self._sigma_history)
            sd = statistics.stdev(self._sigma_history) if len(self._sigma_history) > 1 else 1e-9
            z = abs(sigma - mu) / max(sd, 1e-9)

            if z > self.Z_WARNING_THRESHOLD:
                return CalibrationHealth(
                    status=HealthStatus.WARNING,
                    reason="sigma_drifted",
                    action=RecoveryAction.LOG_ONLY,
                    volatility=sigma,
                    mean_reversion=mean_reversion,
                    volatility_z_score=z,
                    volatility_history_mean=mu,
                    volatility_history_std=sd,
                    observations_since_reset=self._observations,
                )

            return CalibrationHealth(
                status=HealthStatus.HEALTHY,
                reason=None,
                action=RecoveryAction.NONE,
                volatility=sigma,
                mean_reversion=mean_reversion,
                volatility_z_score=z,
                volatility_history_mean=mu,
                volatility_history_std=sd,
                observations_since_reset=self._observations,
            )

        # Not enough history yet
        return CalibrationHealth(
            status=HealthStatus.HEALTHY,
            reason="insufficient_history",
            action=RecoveryAction.NONE,
```

```
            volatility=sigma,
            mean_reversion=mean_reversion,
            volatility_z_score=0.0,
            volatility_history_mean=0.0,
            volatility_history_std=0.0,
            observations_since_reset=self._observations,
        )
```

18.15.3 Spider Dataset Validation Specification

Dataset Requirements

Table 18.24: Spider Dataset Validation Parameters

Parameter	Value	Source
Dataset	Spider v1.0	`https://yale-lily.github.io/spider`
Split	dev (1,034 examples)	Standard benchmark
Databases	20 databases	Cross-domain coverage
Complexity levels	Easy, Medium, Hard, Extra	Per Spider taxonomy

Synthetic Drift Injection Protocol

Listing 18.14: Drift injection for validation

```
@dataclass
class DriftInjectionConfig:
    """Configuration for synthetic drift injection."""

    # Schema drift parameters
    column_drop_rate: float = 0.05  # 5% of columns dropped
    column_rename_rate: float = 0.10  # 10% renamed
    type_change_rate: float = 0.03  # 3% type changes

    # Prompt drift parameters
    vocabulary_shift_rate: float = 0.15  # 15% new terms
    complexity_shift: float = 0.20  # 20% harder queries

    # Temporal parameters
    drift_start_fraction: float = 0.30  # Drift starts at 30%
    drift_ramp_fraction: float = 0.20  # Full drift by 50%

class SpiderDriftInjector:
    """Inject synthetic drift into Spider dataset."""

    def __init__(self, config: DriftInjectionConfig):
        self.config = config
        self._original_schemas: dict[str, dict] = {}

    def inject_schema_drift(
```

```
26          self,
27          schema: dict,
28          drift_intensity: float,
29      ) -> tuple[dict, list[str]]:
30          """
31          Inject schema drift.
32
33          Args:
34              schema: Original database schema
35              drift_intensity: 0.0 (no drift) to 1.0 (full drift)
36
37          Returns:
38              (modified_schema, list_of_changes)
39          """
40          changes = []
41          modified = copy.deepcopy(schema)
42
43          for table in modified["tables"]:
44              for col in table["columns"]:
45                  r = random.random()
46
47                  # Column drop
48                  if r < self.config.column_drop_rate * drift_intensity:
49                      col["_dropped"] = True
50                      changes.append(f"DROP {table['name']}.{col['name']}")
51                      continue
52
53                  # Column rename
54                  if r < (self.config.column_drop_rate +
    self.config.column_rename_rate) * drift_intensity:
55                      old_name = col["name"]
56                      col["name"] = f"{old_name}_v2"
57                      changes.append(f"RENAME {table['name']}.{old_name} ->
    {col['name']}")
58                      continue
59
60                  # Type change
61                  if r < (self.config.column_drop_rate +
    self.config.column_rename_rate +
62                          self.config.type_change_rate) * drift_intensity:
63                      old_type = col["type"]
64                      col["type"] = self._mutate_type(old_type)
65                      changes.append(f"TYPE {table['name']}.{col['name']}:
    {old_type} -> {col['type']}")
66
67          # Filter dropped columns
68          for table in modified["tables"]:
69              table["columns"] = [c for c in table["columns"] if not
    c.get("_dropped")]
70
71          return modified, changes
```

```python

    def inject_prompt_drift(
        self,
        prompt: str,
        drift_intensity: float,
    ) -> str:
        """Inject vocabulary drift into prompt."""
        words = prompt.split()
        for i, word in enumerate(words):
            if random.random() < self.config.vocabulary_shift_rate * drift_intensity:
                words[i] = self._synonym_or_noise(word)
        return " ".join(words)
```

Validation Metrics Protocol

1. **Baseline Phase** (examples 1–300):
   - No drift injected
   - Compute baseline TTS-EVAL, store as $TTS_EVAL_{\text{baseline}}$
   - Record all 19 metric baselines
2. **Drift Ramp Phase** (examples 301–500):
   - Linearly increase drift intensity from 0.0 to 1.0
   - Record TTS-EVAL at each 50-example checkpoint
   - Record first-detection example index for each metric
3. **Full Drift Phase** (examples 501–1034):
   - Drift intensity = 1.0 (constant)
   - Compute final TTS-EVAL, compare to baseline
   - Record false positive rate (alerts during baseline)
   - Record false negative rate (missed drift during ramp)

Expected Outputs

Table 18.25: Spider Validation Expected Outputs

Output	File	Contents
Baseline report	`spider_baseline.json`	All 19 metrics at t=300
Drift detection curve	`spider_drift_curve.csv`	TTS-EVAL vs example index
False positive rate	`spider_fpr.json`	FPR per metric (should be $< 5\%$)
False negative rate	`spider_fnr.json`	FNR per metric (should be $< 10\%$)
Latency report	`spider_latency.json`	P50, P95, P99 per metric

18.15.4 Threshold Bootstrap Analysis Specification

Bootstrap Protocol

Algorithm 3 Threshold Bootstrap via Percentile Method

Require: Clean dataset D (no drift), metric function f, target FPR α
Ensure: Threshold τ such that $P(f(x) > \tau | \text{no drift}) \leq \alpha$

$B \leftarrow 1000$ ▷ Bootstrap iterations
$n \leftarrow |D|$
$S \leftarrow []$ ▷ Bootstrap metric samples
for $b = 1$ to B **do**
  $D_b^* \leftarrow \text{sample_with_replacement}(D, n)$
  $m_b \leftarrow f(D_b^*)$ ▷ Compute metric on bootstrap sample
  $S.\text{append}(m_b)$
end for
if metric direction is "higher is better" **then**
  $\tau \leftarrow \text{percentile}(S, \alpha \times 100)$ ▷ Lower tail
else
  $\tau \leftarrow \text{percentile}(S, (1 - \alpha) \times 100)$ ▷ Upper tail
end if
return τ

Python Implementation

Listing 18.15: Threshold bootstrap implementation

```python
import numpy as np
from typing import Callable, List, Tuple
from dataclasses import dataclass

@dataclass
class BootstrapResult:
    """Result of bootstrap threshold analysis."""

    metric_name: str
    direction: str  # "higher_better" or "lower_better"

    # Bootstrap distribution
    bootstrap_mean: float
    bootstrap_std: float
    bootstrap_percentiles: dict[int, float]  # {5: 0.82, 50: 0.90, 95: 0.95}

    # Derived thresholds at different FPR levels
    threshold_fpr_01: float  # 1% FPR
    threshold_fpr_05: float  # 5% FPR
    threshold_fpr_10: float  # 10% FPR

    # Validation
    actual_fpr: float  # Measured FPR on holdout
    num_bootstrap_samples: int
```

```python

def bootstrap_threshold(
    data: List[dict],
    metric_fn: Callable[[List[dict]], float],
    metric_name: str,
    direction: str,
    num_bootstrap: int = 1000,
    target_fpr: float = 0.05,
    seed: int = 42,
) -> BootstrapResult:
    """
    Compute threshold via bootstrap.

    Args:
        data: Clean dataset (no drift)
        metric_fn: Function to compute metric from data
        metric_name: Name of the metric
        direction: "higher_better" or "lower_better"
        num_bootstrap: Number of bootstrap iterations
        target_fpr: Target false positive rate
        seed: Random seed

    Returns:
        BootstrapResult with threshold and diagnostics
    """
    rng = np.random.default_rng(seed)
    n = len(data)

    # Bootstrap sampling
    bootstrap_metrics = []
    for _ in range(num_bootstrap):
        indices = rng.choice(n, size=n, replace=True)
        sample = [data[i] for i in indices]
        m = metric_fn(sample)
        bootstrap_metrics.append(m)

    bootstrap_metrics = np.array(bootstrap_metrics)

    # Compute percentiles
    percentiles = {
        1: np.percentile(bootstrap_metrics, 1),
        5: np.percentile(bootstrap_metrics, 5),
        10: np.percentile(bootstrap_metrics, 10),
        25: np.percentile(bootstrap_metrics, 25),
        50: np.percentile(bootstrap_metrics, 50),
        75: np.percentile(bootstrap_metrics, 75),
        90: np.percentile(bootstrap_metrics, 90),
        95: np.percentile(bootstrap_metrics, 95),
        99: np.percentile(bootstrap_metrics, 99),
    }

```

```python
    # Derive thresholds based on direction
    if direction == "higher_better":
        # Threshold is lower bound (alert if below)
        threshold_01 = np.percentile(bootstrap_metrics, 1)
        threshold_05 = np.percentile(bootstrap_metrics, 5)
        threshold_10 = np.percentile(bootstrap_metrics, 10)
    else:
        # Threshold is upper bound (alert if above)
        threshold_01 = np.percentile(bootstrap_metrics, 99)
        threshold_05 = np.percentile(bootstrap_metrics, 95)
        threshold_10 = np.percentile(bootstrap_metrics, 90)

    return BootstrapResult(
        metric_name=metric_name,
        direction=direction,
        bootstrap_mean=float(np.mean(bootstrap_metrics)),
        bootstrap_std=float(np.std(bootstrap_metrics)),
        bootstrap_percentiles=percentiles,
        threshold_fpr_01=threshold_01,
        threshold_fpr_05=threshold_05,
        threshold_fpr_10=threshold_10,
        actual_fpr=0.0,  # To be filled after validation
        num_bootstrap_samples=num_bootstrap,
    )
```

Threshold Documentation Template

Each threshold must be documented with:

Listing 18.16: Threshold documentation schema

```json
{
  "metric_code": "TTS-M01",
  "metric_name": "Schema Coverage Rate",
  "current_threshold": 0.85,
  "threshold_derivation": {
    "method": "bootstrap_percentile",
    "dataset": "Spider v1.0 dev split (n=1034)",
    "bootstrap_iterations": 1000,
    "target_fpr": 0.05,
    "bootstrap_mean": 0.912,
    "bootstrap_std": 0.023,
    "derived_threshold": 0.867,
    "chosen_threshold": 0.85,
    "rationale": "Rounded down to 0.85 for operational margin"
  },
  "validation": {
    "holdout_dataset": "Spider train subset (n=500)",
    "measured_fpr": 0.042,
    "measured_fnr": 0.08,
    "validated_at": "2026-04-23"
  }
}
```

18.15.5 Ambiguity Resolution Rate (ARR) Module Specification

Ambiguity Classification

Table 18.26: Ambiguity Types for Text-to-SQL

Type	Example	Resolution Strategy
Lexical	"Show sales" (which sales table?)	Schema lookup, clarification
Scope	"Show recent orders" (what is recent?)	Default + flag
Aggregation	"Total revenue" (sum? count?)	Context inference
Temporal	"Last month" (relative to when?)	Timestamp context
Entity	"Show customer info" (which customer?)	Require filter

Ambiguity Detector Architecture

Listing 18.17: ARR module architecture

```python
from dataclasses import dataclass
from enum import Enum
from typing import List, Optional

class AmbiguityType(Enum):
    LEXICAL = "lexical"
    SCOPE = "scope"
    AGGREGATION = "aggregation"
    TEMPORAL = "temporal"
    ENTITY = "entity"
    NONE = "none"

@dataclass
class AmbiguityDetection:
    """Result of ambiguity detection."""

    is_ambiguous: bool
    ambiguity_type: AmbiguityType
    confidence: float
    ambiguous_span: Optional[str]
    suggested_clarifications: List[str]
    resolution_applied: Optional[str]

class AmbiguityDetector:
    """
    Detect and classify ambiguity in Text-to-SQL prompts.

    Uses a two-stage approach:
    1. Rule-based heuristics for common patterns
    2. Small classifier for edge cases
    """

    # Heuristic patterns
```

```
    LEXICAL_PATTERNS = [
        r"\b(show|get|display)\s+\w+\b",  # Vague verbs
        r"\b(info|data|stuff)\b",  # Vague nouns
    ]

    SCOPE_PATTERNS = [
        r"\b(recent|latest|new|old)\b",  # Relative time
        r"\b(some|few|many|all)\b",  # Vague quantities
    ]

    TEMPORAL_PATTERNS = [
        r"\b(last|this|next)\s+(week|month|year|quarter)\b",
        r"\b(yesterday|today|tomorrow)\b",
    ]

    def __init__(
        self,
        schema_terms: set[str],
        classifier_model: Optional[str] = "microsoft/deberta-v3-small",
        classifier_threshold: float = 0.7,
    ):
        """
        Initialize detector.

        Args:
            schema_terms: Set of valid schema terms (tables, columns)
            classifier_model: HuggingFace model for edge cases
            classifier_threshold: Confidence threshold for classifier
        """
        self.schema_terms = {t.lower() for t in schema_terms}
        self.classifier_model = classifier_model
        self.classifier_threshold = classifier_threshold
        self._classifier = None  # Lazy load

    def detect(self, prompt: str, schema_context: dict) ->
    AmbiguityDetection:
        """
        Detect ambiguity in a prompt.

        Args:
            prompt: Natural language query
            schema_context: Available schema information

        Returns:
            AmbiguityDetection result
        """
        import re

        prompt_lower = prompt.lower()

        # Check lexical ambiguity (unknown terms)
```

```
        words = set(re.findall(r'\b\w+\b', prompt_lower))
        unknown_terms = words - self.schema_terms - self._stopwords()

        if len(unknown_terms) > 3:  # Many unknown terms
            return AmbiguityDetection(
                is_ambiguous=True,
                ambiguity_type=AmbiguityType.LEXICAL,
                confidence=0.8,
                ambiguous_span=", ".join(list(unknown_terms)[:3]),
                suggested_clarifications=[
                    f"Did you mean: {self._suggest_term(t)}" for t in
list(unknown_terms)[:2]
                ],
                resolution_applied=None,
            )

        # Check scope ambiguity
        for pattern in self.SCOPE_PATTERNS:
            match = re.search(pattern, prompt_lower)
            if match:
                return AmbiguityDetection(
                    is_ambiguous=True,
                    ambiguity_type=AmbiguityType.SCOPE,
                    confidence=0.7,
                    ambiguous_span=match.group(),
                    suggested_clarifications=[
                        "Please specify an exact count or range",
                        "Consider adding a numeric limit"
                    ],
                    resolution_applied="default_limit_100",
                )

        # Check temporal ambiguity
        for pattern in self.TEMPORAL_PATTERNS:
            match = re.search(pattern, prompt_lower)
            if match:
                return AmbiguityDetection(
                    is_ambiguous=True,
                    ambiguity_type=AmbiguityType.TEMPORAL,
                    confidence=0.9,
                    ambiguous_span=match.group(),
                    suggested_clarifications=[
                        "Please specify exact dates",
                        f"Interpreting '{match.group()}' relative to
current date"
                    ],
                    resolution_applied="relative_to_current_timestamp",
                )

        # Fall back to classifier for edge cases
        if self._classifier is None:
```

```
            self._load_classifier()

        if self._classifier:
            result = self._classifier_predict(prompt)
            if result["is_ambiguous"] and result["confidence"] > self.classifier_threshold:
                return AmbiguityDetection(
                    is_ambiguous=True,
                    ambiguity_type=AmbiguityType(result["type"]),
                    confidence=result["confidence"],
                    ambiguous_span=None,
                    suggested_clarifications=["Consider providing more specific terms"],
                    resolution_applied=None,
                )

        # No ambiguity detected
        return AmbiguityDetection(
            is_ambiguous=False,
            ambiguity_type=AmbiguityType.NONE,
            confidence=0.95,
            ambiguous_span=None,
            suggested_clarifications=[],
            resolution_applied=None,
        )
```

ARR Metric Computation

Listing 18.18: ARR metric computation

```
def compute_arr(
    prompts: List[str],
    schema_context: dict,
    ground_truth_resolutions: List[str],
    detector: AmbiguityDetector,
) -> float:
    """
    Compute Ambiguity Resolution Rate.

    Args:
        prompts: List of prompts to evaluate
        schema_context: Schema information
        ground_truth_resolutions: Expected SQL for each prompt
        detector: Ambiguity detector instance

    Returns:
        ARR score in [0, 1]
    """
    ambiguous_count = 0
    correctly_resolved = 0

    for prompt, expected_sql in zip(prompts, ground_truth_resolutions):
```

```
        detection = detector.detect(prompt, schema_context)

        if detection.is_ambiguous:
            ambiguous_count += 1

            # Check if resolution was correct
            if detection.resolution_applied:
                # Generate SQL with resolution and compare
                resolved_sql = generate_sql_with_resolution(
                    prompt, detection.resolution_applied
                )
                if sql_equivalent(resolved_sql, expected_sql):
                    correctly_resolved += 1

    if ambiguous_count == 0:
        return 1.0  # No ambiguous prompts = perfect score

    return correctly_resolved / ambiguous_count
```

18.16 Implementation Checklist

- [x] Define 19 TTS drift metrics (TTS-M01 to TTS-M19)
- [x] Specify calculation formulas with statistical bounds
- [x] Define TTS-EVAL composite score
- [x] Specify user population sampling strategy
- [x] Implement `simula_tts_drift_evaluator.py`
- [x] Implement `simula_tts_drift_monitor.py`
- [x] Implement `simula_tts_drift_sampler.py`
- [x] Create HANA Cloud DDL for drift telemetry
- [x] Create JSON schemas for drift artifacts
- [x] Update spec-code-mapping.yaml
- [x] Implement HullWhiteSurpriseGate (SSG)
- [x] Add calibration health checks
- [ ] Integrate with CI/CD pipeline
- [ ] Spider dataset validation
- [ ] Threshold bootstrap analysis
- [ ] Ambiguity resolution module (ARR)

Chapter 19

Financial Domain Multi-Persona Prompt Generation

19.1 Purpose and Scope

This chapter defines a generation pipeline for creating synthetic training prompts in the **financial domain language**. The pipeline generates diverse, high-quality natural language prompts that reflect how different **human populations** and **AI agent co-workers** communicate about financial concepts.

> **Key Insight**
>
> **Key Distinction:** This chapter generates *prompts and queries* about financial statements, not the financial data itself. The focus is on the **language and communication patterns** of financial professionals and AI agents with varying levels of domain expertise.

19.1.1 Integration Points

This chapter integrates with:

- **Chapter 18** — TTS Drift Detection Framework (per-persona drift monitoring)
- **Chapter 5 of singapore-ai-safety** — AI Safety Metrics (fairness, explainability, data quality by persona)
- **Chapter 4a of clinerules-agents** — Agent Spec-Drift Auditing (schema expert vs generalist agent drift)

19.1.2 Core Design Principles

Following Simula's reasoning-first methodology:

1. **Persona as first-class axis** — Generation varies by communicator type
2. **Expertise-aware templates** — Schema experts use precise terms; novices are vague

3. **Dual-reviewer validation** — Prompts validated against persona-appropriate standards
4. **Controlled diversity** — All persona × concept × intent combinations covered
5. **Drift-monitorable** — Baseline distributions enable per-persona drift detection

19.2 Persona Taxonomy

19.2.1 Persona Classification Axes

Each communicator is classified along two orthogonal axes:

Table 19.1: Persona Expertise Axes

Axis	Definition	Levels
Schema Knowledge	Understanding of database structure, tables, columns	None, Low, Medium, H
Process Knowledge	Understanding of business goals, workflows, outcomes	None, Low, Medium, H

19.2.2 Human User Personas

Table 19.2: Human User Persona Definitions

Persona	Schema	Process	Communication Style	Code
Business Executive	Low	High	High-level, outcome-focused, time-sensitive	HU-EXEC
Financial Analyst	Medium	High	Analytical, comparative, ratio-oriented	HU-ANALYST
Accountant	High	High	Precise, standards-compliant, audit-ready	HU-ACCT
New Hire	Low	Low	Exploratory, uncertain, needs guidance	HU-NEW
Operations Manager	Low	Medium	Process-focused, operational metrics	HU-OPS
External Auditor	High	Medium	Compliance-focused, evidence-seeking	HU-AUDIT

19.2.3 AI Agent Worker Personas

19.2.4 Persona Communication Patterns

Listing 19.1: Example prompts by persona

```
# HU-EXEC (Business Executive) - Low schema, High process
"What's our revenue growth this quarter compared to last year?"
"Are we on track to hit the margin target?"

# HU-ANALYST (Financial Analyst) - Medium schema, High process
"Show gross margin by segment with YoY comparison"
```

Table 19.3: AI Agent Worker Persona Definitions

Persona	Schema	Process	Communication Style	Code
Schema Expert Agent	Expert	Medium	Precise column refs, SQL-like structure	AG-SCHEMA
Process Orchestrator	Medium	Expert	Workflow-oriented, task sequencing	AG-ORCH
Generalist Helper	Low	High	Natural language, may use synonyms	AG-GEN
Data Validation Agent	Expert	Low	Assertion-style, boolean outcomes	AG-VAL
Report Generator	Medium	Medium	Template-driven, structured output	AG-REPORT
Anomaly Detector	High	Medium	Statistical, threshold-based queries	AG-ANOM

```
"What's the EBITDA trend over the last 4 quarters?"

# HU-ACCT (Accountant) - High schema, High process
"Verify that total assets equals liabilities plus equity for FY2025"
"Reconcile operating income with EBITDA adjustments"

# AG-SCHEMA (Schema Expert Agent) - Expert schema, Medium process
"SELECT SUM(revenue) FROM income_statement WHERE fiscal_year = 2025
 AND segment_code = 'NAM' GROUP BY quarter"

# AG-GEN (Generalist Helper) - Low schema, High process
"Can you get me the sales numbers for this year?"
"What do our financials look like?"

# HU-NEW (New Hire) - Low schema, Low process
"Where do I find the revenue data?"
"What's a balance sheet?"
```

19.3 Financial Domain Taxonomy

19.3.1 Hierarchical Concept Structure

Listing 19.2: Financial domain taxonomy (JSON)

```
{
  "version": "1.0",
  "root": "FinancialDomain",
  "nodes": {
    "StatementType": {
      "children": ["IncomeStatement", "BalanceSheet", "CashFlowStatement"]
    },
    "IncomeStatement": {
      "children": ["Revenue", "COGS", "GrossProfit", "OperatingExpenses",
                   "OperatingIncome", "InterestExpense", "TaxExpense",
    "NetIncome"],
      "line_items": {
```

```
        "Revenue": {"aliases": ["sales", "top line", "turnover"]},
        "COGS": {"aliases": ["cost of sales", "cost of goods"]},
        "GrossProfit": {"aliases": ["gross margin"]},
        "NetIncome": {"aliases": ["bottom line", "profit", "earnings"]}
      }
    },
    "BalanceSheet": {
      "children": ["Assets", "Liabilities", "Equity"],
      "constraints": ["Assets = Liabilities + Equity"]
    },
    "QueryIntent": {
      "children": ["Retrieval", "Comparison", "Validation", "Aggregation",
                   "Exploration", "Forecasting"]
    },
    "Scenario": {
      "children": ["RoutineReview", "AdhocQuestion", "AuditRequest",
                   "BoardMeeting", "RegulatoryFiling"]
    }
  }
}
```

19.3.2 Alias Mapping for Terminology Tolerance

Schema experts use canonical terms; others may use aliases:

Table 19.4: Financial Term Aliases by Persona Expertise

Canonical Term	Expert (HU-ACCT)	Analyst (HU-ANALYST)	Novice (HU-NEW)
Revenue	Revenue	Revenue, Sales	Sales, Money coming in
COGS	Cost of Goods Sold	COGS, Cost of Sales	Costs, What we spend
Gross Profit	Gross Profit	Gross Margin	Profit
EBITDA	EBITDA	EBITDA	Earnings, Profit
Accounts Receivable	A/R, Receivables	Receivables	Money owed to us

19.3.3 Data Dictionary and Value Synonym Mapping

Aliases are not limited to financial concepts. The prompt generator must also train explicit mappings from surface language to canonical values so that user phrases are resolved deterministically instead of inferred by the model.

Dictionary entries must be versioned with the training run and linked to schema paths. Each entry records `surface_forms`, `canonical_value`, `locale`, `owner`, and `valid_from`. If a prompt contains a surface form that is not in the dictionary or schema registry, the generator must emit a clarification example or reject the training example; it must not invent a mapping.

Table 19.5: Example Data Dictionary Mappings

Schema Field	Surface Forms	Canonical Value	Training Requirement
`country_code`	India, Bharat, Indian entity	`IN`	Include positive and negative examples for country filters
`country_code`	Singapore, SG entity, Singapore branch	`SG`	Include locale-specific prompt variants
`currency_code`	dollars, US dollars, USD	`USD`	Disambiguate from SGD, AUD, and local reporting currency
`period`	Q1, first quarter, quarter one	`Q1`	Include fiscal-calendar context where applicable
`business_segment`	wealth, WM, wealth management	`WEALTH`	Require owner-approved segment vocabulary

19.4 Multi-Persona Prompt Generation Pipeline

19.4.1 Stage 1: Mix Generation

A **mix** is a concrete combination of:

- Persona (from Tables 19.2 and 19.3)
- Financial concept (from Section 19.3)
- Query intent (Retrieval, Comparison, Validation, etc.)
- Scenario (Routine, Adhoc, Audit, etc.)

Listing 19.3: Example mix

```
{
  "mix_id": "mix-001",
  "persona": {
    "code": "HU-ANALYST",
    "schema_knowledge": "medium",
    "process_knowledge": "high"
  },
  "financial_concept": "GrossProfit",
  "query_intent": "Comparison",
  "scenario": "RoutineReview",
```

```
  "accounting_standard": "US-GAAP"
}
```

19.4.2 Stage 2: Persona-Aware Meta-Prompt Templates

Each persona has a distinct template style:

Listing 19.4: Meta-prompt templates by persona

```
TEMPLATES = {
    "HU-EXEC": """
        Generate a brief, outcome-focused question about {concept}.
        The executive wants to know status or trend, not details.
        Use business language, not technical terms.
        Max 15 words.
    """,

    "HU-ANALYST": """
        Generate an analytical query about {concept}.
        Include comparison dimension (YoY, segment, budget vs actual).
        May use financial ratios. Medium technical depth.
        20-40 words.
    """,

    "HU-ACCT": """
        Generate a precise accounting query about {concept}.
        Use correct line item names per {standard}.
        May reference specific accounts or reconciliation.
        Include validation or audit perspective.
    """,

    "AG-SCHEMA": """
        Generate a structured query about {concept}.
        Use explicit column names and table references.
        May include SQL-like syntax or precise filters.
        Highly technical, unambiguous.
    """,

    "AG-GEN": """
        Generate a natural language query about {concept}.
        The agent knows the goal but may not know exact schema terms.
        May use synonyms or vague language.
        Conversational tone.
    """,

    "HU-NEW": """
        Generate an exploratory question about {concept}.
        The user is learning and may ask basic questions.
        Uncertain phrasing, may need guidance.
        Simple vocabulary.
    """
}
```

19.4.3 Stage 3: Complexification by Expertise

Complexity is **calibrated to persona expertise**:

Table 19.6: Complexity Levels by Persona

Persona	c=1	c=2	c=3	c=4	c=5
HU-EXEC	✓	✓	–	–	–
HU-ANALYST	–	✓	✓	✓	–
HU-ACCT	–	–	✓	✓	✓
AG-SCHEMA	–	–	✓	✓	✓
AG-GEN	✓	✓	✓	–	–
HU-NEW	✓	✓	–	–	–

Complexity definitions for financial prompts:

Table 19.7: Complexity Level Definitions

Level	Name	Characteristics
1	Basic	Single concept, no aggregation, top-level only
2	Standard	Single concept with filter or time range
3	Detailed	Multi-concept, segment breakdown, ratios
4	Complex	Multi-period comparison, nested aggregation, reconciliation
5	Expert	Multi-statement analysis, adjustments, audit-level detail

19.4.4 Stage 4: Dual-Reviewer Validation

Validation criteria are **persona-dependent**:

Listing 19.5: Critic prompt for persona-aware validation

```python
def validate_prompt(prompt: str, mix: dict) -> ValidationResult:
    """
    Validate a generated prompt against persona-appropriate criteria.
    """
    persona = mix["persona"]["code"]
    schema_level = mix["persona"]["schema_knowledge"]

    # Retrieve persona-specific rules
    rules = VALIDATION_RULES[persona]

    violations = []

    # Check terminology precision
    if schema_level in ["high", "expert"]:
        if uses_vague_terms(prompt):
            violations.append("terminology_too_vague_for_expert")

    # Check complexity appropriateness
    actual_complexity = estimate_complexity(prompt)
    expected_range = COMPLEXITY_RANGES[persona]
```

Table 19.8: Validation Rules by Persona Type

Check	**HU-EXEC**	**HU-ANALYST**	**HU-ACCT**	**AG-SCHEMA**	**Rationale**
Uses canonical terms	Soft	Medium	Strict	Strict	Experts should be precise
Schema refs valid	N/A	Soft	Strict	Strict	Schema knowledge varies
Ambiguity allowed	Yes	Some	No	No	Novices expected to be vague
Complexity appropriate	Low	Med-High	High	High	Match expertise level
Grammar correct	Soft	Medium	Strict	N/A	Agents may be terse
Intent clear	Medium	Strict	Strict	Strict	All should be answerable

```
    if actual_complexity not in expected_range:
        violations.append(f"complexity_mismatch: {actual_complexity} not in {expected_range}")

    # Check ambiguity
    if not rules["ambiguity_allowed"] and is_ambiguous(prompt):
        violations.append("ambiguity_not_allowed_for_persona")

    # Check intent clarity
    if not has_clear_intent(prompt):
        violations.append("intent_unclear")

    return ValidationResult(
        passed=len(violations) == 0,
        violations=violations,
        persona=persona,
        prompt=prompt,
    )
```

19.5 Persona-Aware Metrics

19.5.1 New Metrics (PER-M01 to PER-M04)

Table 19.9: Persona-Aware Generation Metrics

ID	Name	Formula	Threshold	Direction
PER-M01	Persona Coverage Rate	$\frac{\lvert\text{personas with} \geq N \text{ prompts}\rvert}{\lvert\text{all personas}\rvert}$	$\geq 95\%$	Higher better
PER-M02	Expertise Consistency	$\frac{\lvert\text{prompts matching persona expertise}\rvert}{\lvert\text{total prompts}\rvert}$	$\geq 90\%$	Higher better
PER-M03	Cross-Persona Fairness	$\max_i \lvert p_i - \bar{p}\rvert$ where p_i = proportion for persona i	$\leq 10\%$	Lower better
PER-M04	Per-Persona Drift Score	$Z_t^{\text{persona}_i}$ (SSG Z-score per persona)	$Z < 2.0$	Lower better

PER-M01: Persona Coverage Rate

$$\boxed{PCR = \frac{\lvert\{p : N_p \geq N_{\min}\}\rvert}{\lvert P\rvert}} \tag{19.1}$$

Variables P = set of all defined personas; N_p = number of prompts for persona p; $N_{\min}$ = minimum prompts per persona (default 100)

Threshold $PCR \geq 0.95$ required

PER-M02: Expertise Consistency Rate

$$\boxed{ECR = \frac{N_{\text{prompts matching expected complexity/terminology}}}{N_{\text{total}}}} \tag{19.2}$$

Definition A prompt "matches" if its complexity and terminology precision fall within the expected range for its persona

Threshold $ECR \geq 0.90$ required

PER-M03: Cross-Persona Fairness

$$\boxed{CPF = \max_{i \in P} |p_i - \bar{p}| \quad \text{where} \quad \bar{p} = \frac{1}{|P|}} \tag{19.3}$$

Definition Maximum deviation from uniform persona distribution

Threshold $CPF \leq 0.10$ required (no persona overrepresented by $> 10\%$)

PER-M04: Per-Persona Drift Score

Extends the SSG framework (Section 18.12) to compute a separate Z-score for each persona:

$$\boxed{Z_t^{(p)} = \frac{|x_t^{(p)} - \hat{x}_t^{(p)}|}{\sigma_t^{(p)}}} \tag{19.4}$$

Observable $x_t^{(p)}$ Prompt embedding distance from persona p's training centroid

Threshold $Z_t^{(p)} < 2.0$ for each persona (95% confidence)

Action If any persona exceeds threshold, emit persona-specific drift alert

19.6 Integration with TTS Drift Framework

19.6.1 Extended User Population Sampling

The sampling strategy from Chapter 18 is extended with persona weights:

Table 19.10: Extended Sampling Strategy by Persona

Persona Code	Type	Sample Rate	Frequency	Focus Metrics
HU-EXEC	Human	100%	Real-time	PER-M02, TTS-M09
HU-ANALYST	Human	50%	Hourly	TTS-M05, TTS-M07, PER-M02
HU-ACCT	Human	20%	Hourly	TTS-M03, TTS-M04, PER-M02
HU-NEW	Human	100%	Real-time	TTS-M08 (ARR), PER-M02
AG-SCHEMA	Agent	100%	Real-time	TTS-M01, TTS-M17, PER-M04
AG-ORCH	Agent	50%	Batch	TTS-M18, PER-M04
AG-GEN	Agent	20%	Batch	TTS-M07, TTS-M18, PER-M04

19.6.2 Persona-Attributed Drift Alerts

Drift alerts from Chapter 18 are extended with persona attribution:

Listing 19.6: Extended drift alert with persona

```python
@dataclass
class PersonaDriftAlert(DriftAlert):
    """Drift alert with persona attribution."""

    # Inherited fields
    alert_id: str
    severity: str
    drift_type: str
    metric_code: str
    current_value: float
    threshold: float

    # Persona extension
    persona_code: str
    persona_type: str  # "human" or "agent"
    schema_knowledge: str
    process_knowledge: str

    # Cross-persona context
    other_personas_affected: list[str]
    is_persona_specific: bool  # True if only this persona drifted

    def to_audit_trail(self) -> dict:
        return {
            **super().to_dict(),
            "persona": {
                "code": self.persona_code,
                "type": self.persona_type,
                "schema_knowledge": self.schema_knowledge,
                "process_knowledge": self.process_knowledge,
            },
            "cross_persona": {
                "other_affected": self.other_personas_affected,
                "persona_specific": self.is_persona_specific,
            },
        }
```

19.7 Integration with AI Safety Framework

19.7.1 Mapping to Singapore AI Safety Metrics

19.7.2 Safety Thresholds by Persona Type

Human and agent personas have different safety tolerances:

Table 19.11: Persona Metrics Mapped to AI Safety Framework

AI Safety Metric	Persona Metric	Application
S-01 (Accuracy)	PER-M02	Prompts accurately reflect persona expertise level
S-02 (Robustness)	PER-M04	Stable performance across persona drift scenarios
S-04 (Fairness)	PER-M03	Equal coverage and quality across personas
S-06 (Explainability)	Audit trail	Persona classification rationale is logged
S-08 (Data Quality)	PER-M01, M02	Training data covers all personas appropriately

Table 19.12: Safety Thresholds by Persona Type

Metric	Human Threshold	Agent Threshold	Rationale
Ambiguity tolerance	Higher	Lower	Humans naturally vague
Schema precision	Variable	Strict	Agents should be precise
Drift sensitivity	Medium	High	Agent drift more concerning
Complexity bounds	Wide	Narrow	Human tasks vary more

19.8 Orchestration Pipeline

19.8.1 Configuration File

Listing 19.7: financial_prompt_config.yaml

```yaml
# Financial Domain Prompt Generation Configuration
domain: "Financial professional and agent communication"
taxonomy_version: "1.0"
output_dir: "./generated_prompts"

personas:
  humans:
    - code: HU-EXEC
      schema_knowledge: low
      process_knowledge: high
      target_count: 1000
    - code: HU-ANALYST
      schema_knowledge: medium
      process_knowledge: high
      target_count: 2000
    - code: HU-ACCT
      schema_knowledge: high
      process_knowledge: high
      target_count: 1500
    - code: HU-NEW
      schema_knowledge: low
      process_knowledge: low
      target_count: 500
  agents:
    - code: AG-SCHEMA
      schema_knowledge: expert
      process_knowledge: medium
      target_count: 1500
```

```yaml
    - code: AG-ORCH
      schema_knowledge: medium
      process_knowledge: expert
      target_count: 1000
    - code: AG-GEN
      schema_knowledge: low
      process_knowledge: high
      target_count: 1000

global_diversity:
  min_coverage: 0.95
  sampling_strategy: "stratified"

local_diversity:
  prompts_per_mix: 5
  diversity_threshold: 0.85

complexification:
  enabled: true
  persona_aware: true

quality_control:
  critic_model: "gpt-4"
  persona_validation: true
  acceptance_rules:
    critical_failures: ["intent_unclear", "persona_mismatch"]
    warn_on_complexity_violation: true

metrics:
  per_m01_threshold: 0.95
  per_m02_threshold: 0.90
  per_m03_threshold: 0.10
  per_m04_z_threshold: 2.0
```

19.8.2 Pipeline Implementation

Listing 19.8: simula_financial_prompt_generator.py

```python
from dataclasses import dataclass, field
from typing import List, Dict, Optional
import itertools

@dataclass
class Persona:
    """A communicator persona with expertise levels."""
    code: str
    type: str  # "human" or "agent"
    schema_knowledge: str
    process_knowledge: str
    target_count: int
    complexity_range: tuple[int, int]

```

```python
@dataclass
class PromptMix:
    """A combination of factors for prompt generation."""
    mix_id: str
    persona: Persona
    financial_concept: str
    query_intent: str
    scenario: str
    accounting_standard: str = "US-GAAP"

@dataclass
class GeneratedPrompt:
    """A generated prompt with metadata."""
    prompt_id: str
    text: str
    mix: PromptMix
    complexity_level: int
    template_used: str
    validation_result: Optional[dict] = None

class FinancialPromptGenerator:
    """Generate diverse prompts for financial domain communication."""

    def __init__(self, config_path: str):
        self.config = load_yaml(config_path)
        self.taxonomy = self._load_taxonomy()
        self.personas = self._load_personas()
        self.teacher = LLMClient(model="gpt-4")
        self.critic = LLMClient(model="gpt-4")
        self.embedder = SentenceTransformer("all-MiniLM-L6-v2")

    def run(self) -> List[GeneratedPrompt]:
        """Execute the full generation pipeline."""

        # Stage 1: Generate all mixes
        mixes = self._generate_mixes()
        print(f"Generated {len(mixes)} mixes")

        # Stage 2: Generate prompts per mix
        all_prompts = []
        for mix in mixes:
            prompts = self._generate_prompts_for_mix(mix)
            all_prompts.extend(prompts)
        print(f"Generated {len(all_prompts)} raw prompts")

        # Stage 3: Apply persona-aware complexification
        complexified = []
        for prompt in all_prompts:
            levels = self._get_complexity_levels(prompt.mix.persona)
            for level in levels:
                cp = self._complexify(prompt, level)
```

```
                complexified.append(cp)
        print(f"Complexified to {len(complexified)} prompts")

        # Stage 4: Validate with persona-aware critic
        validated = []
        for prompt in complexified:
            result = self._validate(prompt)
            if result.passed:
                prompt.validation_result = result.to_dict()
                validated.append(prompt)
        print(f"Validated {len(validated)} prompts")

        # Compute metrics
        metrics = self._compute_metrics(validated)
        self._save_artifacts(validated, metrics)

        return validated

    def _generate_mixes(self) -> List[PromptMix]:
        """Generate Cartesian product of personas x concepts x intents."""
        concepts = self.taxonomy["IncomeStatement"]["children"] + \
                   self.taxonomy["BalanceSheet"]["children"]
        intents = self.taxonomy["QueryIntent"]["children"]
        scenarios = self.taxonomy["Scenario"]["children"]

        mixes = []
        for persona in self.personas:
            for concept, intent, scenario in itertools.product(concepts,
    intents, scenarios):
                mix = PromptMix(
                    mix_id=f"mix-{len(mixes):05d}",
                    persona=persona,
                    financial_concept=concept,
                    query_intent=intent,
                    scenario=scenario,
                )
                mixes.append(mix)

        return mixes

    def _generate_prompts_for_mix(self, mix: PromptMix) ->
    List[GeneratedPrompt]:
        """Generate diverse prompts for a single mix."""
        template = TEMPLATES[mix.persona.code]
        base_prompt = template.format(
            concept=mix.financial_concept,
            intent=mix.query_intent,
            scenario=mix.scenario,
            standard=mix.accounting_standard,
        )

```

```
        # Generate multiple diverse versions
        n = self.config["local_diversity"]["prompts_per_mix"]
        prompts = []
        for i in range(n):
            response = self.teacher.complete(
                f"Generate a unique example for: {base_prompt}"
            )
            prompts.append(GeneratedPrompt(
                prompt_id=f"{mix.mix_id}-{i:02d}",
                text=response,
                mix=mix,
                complexity_level=2,  # Default, will be adjusted
                template_used=mix.persona.code,
            ))

        # Deduplicate by embedding similarity
        return self._deduplicate(prompts)

    def _validate(self, prompt: GeneratedPrompt) -> ValidationResult:
        """Validate prompt against persona-appropriate criteria."""
        return validate_prompt(prompt.text, prompt.mix.__dict__)

    def _compute_metrics(self, prompts: List[GeneratedPrompt]) -> dict:
        """Compute PER-M01 to PER-M04."""
        # PER-M01: Persona Coverage Rate
        counts = {}
        for p in prompts:
            counts[p.mix.persona.code] = counts.get(p.mix.persona.code, 0) + 1
        per_m01 = sum(1 for c in counts.values() if c >= 100) / len(self.personas)

        # PER-M02: Expertise Consistency
        consistent = sum(1 for p in prompts if self._is_consistent(p))
        per_m02 = consistent / len(prompts) if prompts else 0

        # PER-M03: Cross-Persona Fairness
        proportions = [counts.get(p.code, 0) / len(prompts) for p in self.personas]
        mean_prop = 1 / len(self.personas)
        per_m03 = max(abs(p - mean_prop) for p in proportions)

        return {
            "PER-M01": per_m01,
            "PER-M02": per_m02,
            "PER-M03": per_m03,
            "PER-M04": {},  # Computed during drift monitoring
        }
```

19.9 Implementation Checklist

- [x] Define persona taxonomy (human + agent)
- [x] Define financial domain taxonomy (concepts, intents, scenarios)
- [x] Create persona-aware prompt templates
- [x] Define PER-M01 to PER-M04 metrics
- [x] Specify validation rules by persona
- [x] Define integration with TTS drift framework
- [x] Define integration with AI Safety framework
- [ ] Implement `simula_financial_prompt_generator.py`
- [ ] Implement `simula_financial_prompt_critic.py`
- [ ] Create JSON schemas for artifacts
- [ ] Create HANA DDL for persona telemetry
- [ ] Integrate with CI/CD pipeline

19.10 Cross-References

- **TTS Drift Metrics (Chapter 18)** — TTS-M05 (SAS), TTS-M07 (TDS), TTS-M08 (ARR), TTS-M18 (PSS) apply to generated prompts
- **SSG Framework (Section 18.12)** — Per-persona Hull-White calibration for PER-M04
- **AI Safety Metrics (Chapter 5 of singapore-ai-safety)** — S-01, S-02, S-04, S-06, S-08 mapped to persona metrics
- **Spec-Drift Auditing (Chapter 4a of clinerules-agents)** — DRIFT-009, DRIFT-010 for agent persona drift

Appendix A

Implementation Gap Review and Agent Rule Pack

Warning

Point-in-Time Assessment
This appendix is a snapshot of implementation status, not a normative specification.
Last Updated: 2026-04-21
Next Review Due: 2026-05-21 (or after completion of any v1.2 milestone)

If this document is more than 4 weeks old without updates, re-verify findings before relying on them.

A.1 Review scope and constraints

This chapter records a repository review performed against the *Simula Training Data Framework* specification PDF and the current code under `/Users/user/Documents/sap-oss/src` as of 2026-04-21. The review covered four targets:

1. the normative Simula implementation path in `src/training/`;
2. the implementation-agent and maintenance rule location under `src/training/`;
3. the operational frontend and API wrapper in `src/generativeUI/training-webcomponents-ngx/`
4. runtime monitoring and alerting support for Simula runs.

Two delivery constraints shaped the outcome:

- This workstream was instructed to change only files under `docs/`. No source changes were made under `src/`.
- The intended agent location is `src/training/`. Because source-tree edits were out of scope for this task, the implementation-agent and runtime-monitor definitions are captured here as canonical proposed artefacts to be materialised later under `src/training/.clinerules` and a companion runtime monitor rule file in the same subtree.

A.2 Executive assessment

Table A.1: Current implementation status against the Simula specification.

Area	Status	Assessment
Default CLI and pipeline entrypoints	**Red**	The repository default still executes the legacy CSV/template/Spider pipeline, not the HANA-direct Simula flow described in chapters 4–6.
Simula core modules	**Amber**	HANA extraction, taxonomy building, generation, and evaluator modules exist, but they are only partially integrated and several evaluation components fail at runtime because their data contracts drifted.
Evaluation framework	**Red**	Complexity calibration, coverage, diversity, taxonomy quality, and critic calibration are present as isolated modules, but they are not orchestrated by the runtime path that produces datasets.
Frontend training console	**Amber/Red**	The shell, governance surfaces, and HANA explorer exist, but the pipeline UX still models the older workflow and does not expose Simula-specific artefacts or evaluator outputs.
Runtime monitoring and alerting	**Amber/Red**	Generic health and notification primitives exist, but there is no Simula-specific runtime monitor that watches spec-critical outputs, thresholds, and evaluator failures.
`src/training` agent rule materialisation	**Amber**	The canonical agent location is `src/training`, but the rule files are documented here rather than created in source because this task was constrained to `docs/`.

A.3 Validation performed

The review did not rely only on static reading. The following checks were run against the current repository state:

- `pytest -q src/training/pipeline/tests/test_pipeline.py` passed (**20/20**); this confirms that the *legacy* pipeline remains internally healthy.
- `pytest -q packages/api-server/tests/test_main.py -k 'hana_stats_falls_back_to_ or hana_query_returns_preview_rows_when_unconfigured'` passed (**2/2**); this confirms that preview fallback behaviour is functioning as designed in the current API wrapper.
- Runtime reproductions demonstrated that `SimulaCriticValidator` and `SimulaCoverageEval` are not execution-safe against the current `TrainingExample` and `Taxonomy` classes: the observed failures were `TypeError: TrainingExample.__init__()`

`got an unexpected keyword argument 'schema_context'` and `AttributeError: 'Taxonomy' object has no attribute 'id'`.

A.4 Gap-to-Milestone Mapping Summary

The following table provides a direct mapping from each P0 gap identified in this appendix to the specific project milestone(s) it blocks. This enables prioritized remediation planning aligned with the project roadmap in Chapter 15.

Table A.2: P0 Gap to Milestone Blocking Relationship

Gap ID	Severity	Blocks Milestone	Impact Summary
SIM-GAP-001	P0	M01, M04	Default CLI runs legacy pipeline; blocks extraction pipeline certification and environment CLI
SIM-GAP-002	P0	M04, M06	API/job wiring to legacy generators; blocks environment CLI and TB reference run
SIM-GAP-003	P0	M02, M03	Evaluator contract drift; blocks taxonomy engine and generation engine certification
UI-GAP-001	P0	M06, M07	Pipeline UI models old workflow; blocks TB reference run sign-off and v1.2 release
P1 gaps (for reference):			
SIM-GAP-004	P1	M03	Evaluation framework not in runtime path
SIM-GAP-005	P1	M03	Adaptive controls not applied in generator
SIM-GAP-006	P1	M04, M06	Output contract mismatch across runner/API/UI
SIM-GAP-007	P2	M07	Tests only prove legacy path
UI-GAP-002	P1	M06	Simulated telemetry (not real metrics)
UI-GAP-003	P1	M06	Preview/production distinction unclear
UI-GAP-004	P1	M07	No Simula-specific UI for evaluation artefacts
UI-GAP-005	P2	M07	Notifications generic, not Simula-specific

Critical Path Impact: The four P0 gaps collectively block milestones M01 through M07, meaning no v1.2 release is possible until all P0 gaps are resolved. The recommended remediation order in §A.9 sequences these fixes to unblock the critical path as early as possible.

A.5 Findings in `src/training`

SIM-GAP-001 (P0): the default execution path is still the legacy pipeline

Blocks Milestone: M01 (Extraction pipeline), M04 (Environment CLI)

Observed state. `src/training/pipeline/main.py` still exposes the legacy command set (`extract-schema`, `parse-templates`, `expand`, `format-spider`, `upload`), and the default Makefile target still executes the same CSV/template/Spider flow.

Evidence.

- `src/training/pipeline/main.py:3-13, 30-39, 136-141`
- `src/training/pipeline/Makefile:4-11, 29-30, 35-82`

- By contrast, the Simula orchestration path lives in `src/training/scripts/run_hana_pipeli` `51-196` and is not the repository default.

Impact. An operator following the current default entrypoint runs the old pipeline, not the spec-defined HANA extraction, taxonomy generation, and reasoning-driven synthesis flow. This is the single largest end-to-end compliance gap.

Required remediation. Promote the Simula path to the authoritative CLI/runtime entrypoint, and demote or quarantine the legacy template pipeline behind an explicit compatibility switch.

SIM-GAP-002 (P0): API and job orchestration are wired to the legacy generators

Blocks Milestone: M04 (Environment CLI), M06 (TB reference run)

Observed state. The training console backend does not call `run_hana_pipeline.py`. Instead:

- the pipeline websocket worker runs `make all` in `src/training/pipeline/`;
- the training-generation job runner executes `schema_pipeline/data_generator.py` followed by `prepare_training_data.py`.

Evidence.

- `packages/api-server/src/main.py:2904-3019`
- `packages/api-server/src/main.py:2222-2357`

Impact. Even if the Simula modules were complete, the deployed frontend/API stack would still launch the older data path and publish the older artefact contract. The Simula implementation therefore remains present in code but absent from the operational surface.

Required remediation. Replace the API worker commands with the Simula runner and its report artefacts; keep the current legacy commands only for rollback or migration support.

SIM-GAP-003 (P0): evaluation modules have drifted away from the current data contracts

Blocks Milestone: M02 (Taxonomy engine), M03 (Generation engine)

Observed state. The current `TrainingExample` dataclass contains only `id`, `question`, `sql`, `domain`, `table`, `difficulty`, `taxonomy_mix`, `meta_prompt`, `is_complexified`, and `critic_verdict`. However:

- `SimulaCriticValidator` constructs `TrainingExample` objects using nonexistent fields such as `schema_context`, `taxonomy_path`, `complexity_score`, `critic_passed`, and `generation_metadata`;
- `SimulaCoverageEvaluator` expects each `Taxonomy` to expose `id` and `factor`, and each `TaxonomyNode` to expose `id`; the current taxonomy classes expose `factor_name` and `name`/`description`/`children` instead.

Evidence.

- `src/training/pipeline/simula_data_generator.py:32-44`
- `src/training/pipeline/simula_critic_validator.py:212-238, 268-272`
- `src/training/pipeline/simula_coverage_evaluator.py:157-170, 203-215`
- Runtime reproductions raised the concrete exceptions recorded in the validation section above.

Impact. The spec's evaluation chapter is not merely unintegrated; two of its core evaluators are currently unusable against the live Simula data model. This blocks critic calibration, coverage measurement, and any automatic gap-filling workflow built on top of them.

Required remediation. Unify the `TrainingExample`, `Taxonomy`, and evaluator contracts before further orchestration work. Until those contracts are repaired, downstream evaluation integration should be treated as blocked.

SIM-GAP-004 (P1): the Simula orchestration script stops before the full evaluation framework

Observed state. `run_hana_pipeline.py` performs three phases:

1. HANA schema extraction;
2. taxonomy construction;
3. example generation and file write-out.

It writes `extracted_schema.json`, `taxonomies.json`, a timestamped training JSONL, and `pipeline_stats.json`. It does *not* call the evaluator modules for complexity calibration, coverage, diversity, taxonomy quality, critic calibration, or adaptive gap filling.

Evidence.

- `src/training/scripts/run_hana_pipeline.py:51-196`
- Search results show that `SimulaComplexityCalibrator`, `SimulaCoverageEvaluator`, `SimulaCriticValidator`, `SimulaDiversityAnalyzer`, `SimulaDiversityOptimizer`, and `SimulaTaxonomyEvaluator` are defined and re-exported, but not invoked from any orchestration path.

Impact. The implementation currently reaches chapter 5 (*Generation Engine*) but not chapter 7 (*Evaluation Framework*) in an operational sense. The spec's quality gates therefore exist largely as library code and prose, not as emitted run artefacts.

Required remediation. Extend the Simula runner so that every completed generation run emits all evaluation reports required by the spec and records their threshold outcomes into governance and monitoring surfaces.

SIM-GAP-005 (P1): adaptive controls are specified in configuration but not applied in the generator

Observed state. `GenerationConfig` defines `get_adaptive_complexity_ratio()` and `get_adaptive_critic_threshold()`, but the main generation loop continues to use the raw `complexity_ratio` and a plain double-critic pass/fail call without consulting those helpers.

Evidence.

- `src/training/pipeline/simula_config.py:269-326`
- `src/training/pipeline/simula_data_generator.py:494-500, 599-614`
- Repository-wide search found no call sites for `get_adaptive_complexity_ratio()` or `get_adaptive_critic_threshold()` outside their definitions.

Impact. Adaptive features introduced in the v1.2 spec are represented as configuration helpers only. The runtime path does not yet change its behaviour in response to teacher strength or measured complexity.

Required remediation. Thread the adaptive controls through generation, critic acceptance, and report emission so the pipeline can prove that the v1.2 adaptive logic is active.

SIM-GAP-006 (P1): output contracts do not line up across the Simula runner, API, and UI

Observed state. The Simula runner writes timestamped files such as `simula_training_{timestam` under `data/hana_generated/`. The API and frontend continue to expect the older path `training/pipeline/output/train.jsonl`.

Evidence.

- `src/training/pipeline/simula_data_generator.py:672-716`
- `packages/api-server/src/main.py:2960-2963, 3007-3012, 3354-3360`
- `apps/angular-shell/src/app/pages/pipeline/pipeline.component.ts:549-554`

Impact. Even after the Simula runner is wired in, the frontend and API will not discover the artefacts they are looking for unless the output contract is normalised.

Required remediation. Choose one authoritative artefact layout for Simula runs and apply it consistently across runner, backend, governance records, preview endpoints, and UI components.

SIM-GAP-007 (P2): automated tests still prove only the legacy pipeline path

Observed state. The repository contains passing unit tests for the legacy pipeline modules, but there are no corresponding tests for `run_hana_pipeline.py` or the end-to-end Simula evaluator stack.

Evidence.

- `src/training/pipeline/tests/test_pipeline.py`

- The passing test run recorded in the validation section above exercised only those legacy tests.

Impact. The project currently has high confidence in the deprecated path and low confidence in the normative one.

Required remediation. Add unit and integration coverage for extraction, taxonomy, critic filtering, evaluator compatibility, output contracts, and the API worker that launches the Simula flow.

A.6 Findings in `src/generativeUI/training-webcomponents-ng`

UI-GAP-001 (P0): the pipeline UI still models the old six-stage workflow

Blocks Milestone: M06 (TB reference run), M07 (v1.2 release tag)

Observed state. The pipeline page shows the following stages: `Preconvert`, `Extract Schema`, `Parse Templates`, `Expand`, `Build SQL`, and `Format`. Stage transitions are inferred from keywords such as `converting`, `parsing`, and `formatting`.

Evidence.

- `apps/angular-shell/src/app/pages/pipeline/pipeline.component.ts:333-339`

- `apps/angular-shell/src/app/pages/pipeline/pipeline.component.ts:488-497`

Impact. The operational UI tells users that the product pipeline is still the preconvert/template/Spider flow. It does not surface the Simula phase model from this spec: HANA extraction, taxonomy building, data generation, and evaluation.

Required remediation. Replace the stage model with the Simula phase graph and emit stage-specific artefacts and metrics from the backend.

UI-GAP-002 (P1): runtime telemetry in the pipeline page is simulated rather than sourced from the backend

Observed state. The pipeline page's worker grid and throughput readout are generated from random numbers whenever the pipeline state is `running`.

Evidence.

- `apps/angular-shell/src/app/pages/pipeline/pipeline.component.ts:369-379`

Impact. The page provides motion, but not observability. It cannot support a runtime-monitoring role because the visual telemetry is not derived from actual execution state.

Required remediation. Bind the concurrency, throughput, and stage timing displays to real worker metrics, log events, and evaluator outcomes.

UI-GAP-003 (P1): the frontend/API operational surface is still wired to preview or compatibility paths

Observed state. The HANA explorer and related backend endpoints deliberately fall back to preview data when credentials or connectivity are missing. The API wrapper exposes this cleanly, but the Simula pipeline has no explicit distinction between a live production run and a preview-backed demonstration surface.

Evidence.

- `packages/api-server/src/main.py:3030-3038, 3149-3197, 3349-3395`
- `apps/angular-shell/src/app/pages/hana-explorer/hana-explorer.component.ts:17-2 73-82, 112-118`
- API preview tests passed, confirming that preview mode is an intended and working behaviour in the current stack.

Impact. Preview fallback is appropriate for resilience, but the current product surface does not yet enforce the sharper contract the Simula spec needs: production corpus generation must be visibly live, never silently satisfied by preview artefacts.

Required remediation. Add explicit live-vs-preview run state, block governed production launches when required dependencies are only in preview mode, and surface that policy directly in the UI.

UI-GAP-004 (P1): there is no Simula-specific UI for taxonomy and evaluation artefacts

Observed state. The frontend contains generic dashboard, analytics, registry, governance, HANA explorer, and data-product screens, but there is no route or component dedicated to:

- extracted schema inspection as a Simula run artefact;
- taxonomy visualisation and taxonomy quality review;
- coverage, diversity, critic calibration, or Elo complexity reports;
- student-teacher gap tracking or adaptive-complexity decisions.

Evidence.

- `apps/angular-shell/src/app/app.routes.ts:1-92`
- Repository-wide searches for `SimulaCoverageEvaluator`, `SimulaCriticValidator`, `SimulaComplexityCalibrator`, `SimulaDiversityAnalyzer`, and `SimulaTaxonomyEvaluator` found no corresponding Angular surfaces.

Impact. The spec's evaluator outputs cannot yet be reviewed in the operational console, which means human sign-off, explainability, and runtime triage still depend on raw files or logs rather than first-class product screens.

Required remediation. Add explicit Simula result views or extend existing governance and registry pages so that each spec-defined artefact becomes inspectable.

UI-GAP-005 (P2): notification and alerting are generic, not Simula-specific

Observed state. Notification CRUD exists in the API, but the shell notification tray is currently composed from generic health, pipeline state, and GPU pressure held in the store. No Simula-specific alert rules are registered for evaluator failures, missing artefacts, contract drift, or unexpected preview mode.

Evidence.

- `packages/api-server/src/notification_api.py`
- `packages/api-server/src/main.py:590`
- `apps/angular-shell/src/app/components/shell/shell.component.ts:645-691`

Impact. The plumbing for notifications exists, but the actual runtime monitor requested for Simula has not yet been encoded as alerting logic.

Required remediation. Back the notification surface with rules derived from Simula run artefacts and thresholds, not only generic platform health.

A.7 Proposed implementation-agent rule file

Because this review was restricted to `docs/`, the following content is the canonical proposed implementation-agent rule file. It should be materialised at `src/training/.clinerules` once source-tree edits are allowed.

```
# ==============================================================================
# .clinerules - Simula implementation and maintenance agent
# ==============================================================================

Mission:
- Implement and maintain the Simula training data framework in src/training.
- Treat the Simula PDF spec as normative over legacy pipeline behaviour.

Repository reality check:
- The canonical agent location is src/training.
- The default runtime path is still the legacy CSV/template pipeline.
- Simula modules already exist in src/training/pipeline and must become the
  authoritative execution path.

Read these files first on every Simula task:
- docs/pdf/specs/simula-training-spec.pdf
- docs/latex/specs/simula/chapters/12-implementation-instructions.tex
- src/training/scripts/run_hana_pipeline.py
- src/training/pipeline/main.py
- src/training/pipeline/Makefile
- src/training/pipeline/simula_data_generator.py
- src/training/pipeline/simula_taxonomy_builder.py
- src/training/pipeline/hana_schema_extractor.py
- src/training/pipeline/simula_critic_validator.py
- src/training/pipeline/simula_coverage_evaluator.py

Non-negotiable implementation rules:
1. Promote the Simula HANA-direct path over the legacy template path.
2. Keep a single canonical data contract across TrainingExample, Taxonomy,
   evaluation reports, API payloads, and UI artefacts.
3. Do not add new evaluator fields in one module without updating every
   producer and consumer.
4. Wire evaluation into the runtime path:
   - complexity calibration
   - critic calibration
   - taxonomy quality
   - coverage
```

```
   - diversity
   - student-teacher gap reporting
5. Emit stable artefacts for every run:
   - extracted_schema.json
   - taxonomies.json
   - training_examples.jsonl (or one agreed canonical JSONL name)
   - pipeline_stats.json
   - evaluation reports
6. Make API and frontend consume the same artefact contract as the runner.
7. Preserve legacy pipeline code only as an explicit compatibility path.

Required tests before considering a change complete:
- unit tests for any changed Simula module
- contract test for TrainingExample and Taxonomy serialization
- integration test for the runner entrypoint used by the API
- regression test proving frontend/API artefact paths still resolve

Definition of done:
- the default pipeline command launches the Simula flow
- evaluator modules execute without contract errors
- governance captures run status and evaluation outputs
- frontend surfaces the real Simula artefacts instead of legacy placeholders
```

A.8 Proposed runtime-monitor agent rule file

The runtime-monitor agent below is the canonical proposed companion to the implementation agent. It should be materialised later at a path such as `src/training/.clinerules.runt` or another agreed operational-agent location.

```
# ==============================================================================
# .clinerules - Simula runtime monitor and alerting agent
# ==============================================================================

Mission:
- Monitor live Simula runs and raise actionable alerts.
- Distinguish live production runs from preview/demo fallbacks.

Primary evidence sources:
- governance training runs and gate checks
- /pipeline/status and websocket pipeline logs
- /jobs and job websocket updates
- HANA health and mode (live vs preview)
- vLLM / TurboQuant health
- emitted run artefacts in the agreed output directory

Treat these as critical alert conditions:
1. Pipeline state = error or job status = failed.
2. HANA extraction completed with zero tables.
3. Taxonomy build completed with zero taxonomies.
4. Example generation completed with zero examples.
5. Expected artefact missing:
   - extracted_schema.json
   - taxonomies.json
   - training JSONL
   - pipeline_stats.json
6. Any evaluator raises a contract error or runtime exception.
7. Production-governed run is operating in preview mode.
8. Coverage or taxonomy quality falls below agreed thresholds.
9. Critic calibration degrades:
   - p(accept | correct) <= 0.80
   - p(accept | corrupted) >= 0.30
10. Diversity falls outside target band:
   - global diversity materially below target
   - local diversity materially below target

Recommended severities:
- Critical:
  failed run, preview mode on a production run, missing core artefact,
  contract drift in evaluator modules
- High:
```

```
  zero extracted tables, zero taxonomies, zero examples,
  critic calibration failure
- Medium:
  low coverage, low diversity, repeated governance blocks

Required alert payload:
- run_id
- job_id
- team or tenant
- stage
- severity
- exact failing condition
- last relevant log lines
- affected artefact path
- recommended next action

Response rules:
1. Open a notification and attach it to the owning user/team.
2. Append a governance note for Critical and High alerts.
3. Never report "healthy" if the run is only in preview mode.
4. Prefer precise evidence over generic summaries.
5. Collapse duplicate alerts for the same run and condition.
```

A.9 Sequenced remediation order

The findings above should be addressed in the following order:

1. Replace the default execution path and API worker wiring so the Simula runner becomes authoritative.
2. Repair the `TrainingExample`/`Taxonomy`/evaluator contract drift.
3. Normalise run artefact naming and paths across runner, API, governance, and frontend.
4. Integrate the evaluation framework into the operational run.
5. Replace simulated UI telemetry with real runtime metrics and evaluator outputs.
6. Materialise the two proposed agent rule files once source-tree edits outside `docs/` are permitted.

Appendix B

SAP Integration Guide

This appendix provides detailed technical guidance for integrating Simula with SAP technology stack components. It complements the high-level integration points described in Chapter 2 and Chapter 16.

B.1 SAP HANA Cloud Integration

B.1.1 Connection Configuration

Simula requires read-only access to SAP HANA Cloud for schema extraction.

Listing B.1: HANA Cloud connection configuration

```
# Environment variables for HANA Cloud connection
SIMULA_HANA_HOST=your-hana-instance.hana.ondemand.com
SIMULA_HANA_PORT=443
SIMULA_HANA_USER=SIMULA_SERVICE_USER
SIMULA_HANA_PASSWORD_SECRET=hana-password  # Reference to BTP Credential
    Store
SIMULA_HANA_ENCRYPT=true
SIMULA_HANA_SCHEMA=TRAINING_SOURCE

# Python connection using hdbcli
from hdbcli import dbapi

connection = dbapi.connect(
    address=os.environ["SIMULA_HANA_HOST"],
    port=int(os.environ["SIMULA_HANA_PORT"]),
    user=os.environ["SIMULA_HANA_USER"],
    password=get_secret(os.environ["SIMULA_HANA_PASSWORD_SECRET"]),
    encrypt=True,
    sslValidateCertificate=True
)
```

B.1.2 Required Database Permissions

The service user requires the following minimum privileges:

Table B.1: HANA Cloud permissions for Simula service user

Permission	Purpose
SELECT on SYS.TABLES	List available tables
SELECT on SYS.COLUMNS	Extract column metadata
SELECT on SYS.CONSTRAINTS	Extract foreign key relationships
SELECT on SYS.INDEXES	Identify indexed columns
SELECT on target schemas	Read table/view definitions
CATALOG READ	Access system views

Listing B.2: Granting minimum permissions

```
-- Create service user
CREATE USER SIMULA_SERVICE_USER PASSWORD "***" NO
    FORCE_FIRST_PASSWORD_CHANGE;

-- Grant catalog read
GRANT CATALOG READ TO SIMULA_SERVICE_USER;

-- Grant schema access (repeat for each source schema)
GRANT SELECT ON SCHEMA "TRAINING_SOURCE" TO SIMULA_SERVICE_USER;

-- For metadata extraction
GRANT SELECT ON SYS.TABLES TO SIMULA_SERVICE_USER;
GRANT SELECT ON SYS.COLUMNS TO SIMULA_SERVICE_USER;
GRANT SELECT ON SYS.CONSTRAINTS TO SIMULA_SERVICE_USER;
GRANT SELECT ON SYS.INDEXES TO SIMULA_SERVICE_USER;
```

B.1.3 HANA-Specific Type Mapping

Table B.2: HANA type to Simula schema type mapping

HANA Type	Simula Type	Notes
TINYINT, SMALLINT, INTEGER, BIGINT	integer	—
DECIMAL, SMALLDECIMAL	decimal	Preserve precision/scale
REAL, DOUBLE	float	—
VARCHAR, NVARCHAR	string	Note max length
DATE	date	Format: YYYY-MM-DD
TIME	time	Format: HH:MM:SS
TIMESTAMP, SECONDDATE	timestamp	ISO 8601 format
BOOLEAN	boolean	—
VARBINARY, BLOB	binary	Excluded from SQL generation
CLOB, NCLOB	text	Large text handling
ST_GEOMETRY, ST_POINT	geometry	See §B.1.4

B.1.4 Spatial Type Handling

HANA spatial types require special handling during schema extraction and SQL generation:

Listing B.3: Spatial type detection and handling

```python
SPATIAL_TYPES = {
    "ST_GEOMETRY", "ST_POINT", "ST_LINESTRING", "ST_POLYGON",
    "ST_MULTIPOINT", "ST_MULTILINESTRING", "ST_MULTIPOLYGON",
    "ST_GEOMETRYCOLLECTION"
}

def extract_column_metadata(cursor, table_name: str) -> list[ColumnMetadata]:
    """Extract column metadata with spatial type handling."""
    query = """
        SELECT COLUMN_NAME, DATA_TYPE_NAME, LENGTH, SCALE, IS_NULLABLE
        FROM SYS.COLUMNS
        WHERE TABLE_NAME = ?
        ORDER BY POSITION
    """
    cursor.execute(query, (table_name,))

    columns = []
    for row in cursor.fetchall():
        col_type = row[1]
        is_spatial = col_type in SPATIAL_TYPES

        columns.append(ColumnMetadata(
            name=row[0],
            data_type=col_type,
            simula_type="geometry" if is_spatial else map_hana_type(col_type),
            length=row[2],
            scale=row[3],
            nullable=row[4] == "TRUE",
            is_spatial=is_spatial,
            spatial_srid=get_srid(cursor, table_name, row[0]) if is_spatial else None
        ))

    return columns

def get_srid(cursor, table_name: str, column_name: str) -> int | None:
    """Get SRID for spatial column."""
    query = """
        SELECT SRS_ID FROM SYS.ST_GEOMETRY_COLUMNS
        WHERE TABLE_NAME = ? AND COLUMN_NAME = ?
    """
    cursor.execute(query, (table_name, column_name))
    row = cursor.fetchone()
    return row[0] if row else 4326  # Default to WGS84
```

SQL Generation Guidelines for Spatial Columns:

- Use `ST_AsText()` for human-readable output
- Use `ST_Distance()` for proximity queries

- Use `ST_Within()` for containment predicates
- Avoid including raw geometry in `SELECT *` queries

B.2 SAP AI Core Integration

B.2.1 Deployment Configuration

Simula can be deployed as an SAP AI Core serving endpoint:

Listing B.4: SAP AI Core serving configuration

```yaml
# ai-core-serving.yaml
apiVersion: ai.sap.com/v1alpha1
kind: ServingTemplate
metadata:
  name: simula-generator
  labels:
    scenarios.ai.sap.com/id: "simula-training"
    ai.sap.com/version: "1.2.0"
spec:
  template:
    apiVersion: serving.kubeflow.org/v1alpha2
    spec:
      predictor:
        imagePullSecrets:
          - name: docker-registry-secret
        containers:
          - name: simula
            image: sap-oss/simula:1.2.0
            ports:
              - containerPort: 8080
                protocol: TCP
            env:
              - name: SIMULA_VLLM_ENDPOINT
                valueFrom:
                  secretKeyRef:
                    name: simula-secrets
                    key: vllm-endpoint
              - name: SIMULA_HANA_HOST
                valueFrom:
                  configMapKeyRef:
                    name: simula-config
                    key: hana-host
            resources:
              requests:
                memory: "4Gi"
                cpu: "2"
              limits:
                memory: "8Gi"
                cpu: "4"
```

B.2.2 Training Pipeline Integration

Register Simula outputs with SAP AI Core MLOps:

Listing B.5: Registering training data with AI Core

```
from ai_core_sdk.models import Artifact, ArtifactScenario

def register_training_dataset(
    ai_core_client,
    dataset_path: str,
    run_id: str,
    metadata: dict
) -> str:
    """Register generated dataset as AI Core artifact."""

    artifact = Artifact(
        name=f"simula-training-data-{run_id}",
        kind="dataset",
        url=dataset_path,  # S3/Object Store URL
        scenario_id="simula-training",
        labels={
            "run_id": run_id,
            "examples_count": str(metadata["total_examples"]),
            "coverage_score": str(metadata["coverage"]["global"]),
            "diversity_score": str(metadata["diversity"]["global"]),
            "generation_date": metadata["timestamp"]
        }
    )

    result = ai_core_client.artifact.create(artifact)
    return result.id
```

B.3 SAP BTP Services Integration

B.3.1 Credential Store

Store sensitive credentials in SAP BTP Credential Store:

Listing B.6: Retrieving secrets from BTP Credential Store

```
import requests
from cfenv import AppEnv

def get_secret(secret_name: str) -> str:
    """Retrieve secret from BTP Credential Store."""
    env = AppEnv()
    cred_store = env.get_service(name="credential-store")

    response = requests.get(

    f"{cred_store.credentials['url']}/api/v1/credentials/{secret_name}",
        headers={
            "Authorization": f"Bearer {get_token(cred_store)}",
```

```
            "Content-Type": "application/json"
        }
    )
    response.raise_for_status()
    return response.json()["value"]
```

B.3.2 Audit Log Service

Emit audit events to SAP Audit Log Service:

Listing B.7: Audit logging integration

```
from sap_audit_logging import AuditLogger, SecurityMessage

audit_logger = AuditLogger()

def log_generation_event(
    run_id: str,
    user_id: str,
    action: str,
    details: dict
):
    """Log generation event to SAP Audit Log Service."""

    message = SecurityMessage(
        user=user_id,
        action=action,
        object_type="simula.training.dataset",
        object_id=run_id,
        attributes=[
            {"name": "examples_count", "old": None, "new": str(details.get("count"))},
            {"name": "source_schema", "old": None, "new": details.get("schema")},
            {"name": "quality_passed", "old": None, "new": str(details.get("passed"))}
        ]
    )

    audit_logger.log_security_message(message)
```

B.3.3 Alert Notification Service

Configure alerts for quality gate failures:

Listing B.8: Alert notification for quality failures

```
import requests

def send_quality_alert(
    ans_endpoint: str,
    run_id: str,
    failures: list[dict]
):
```

```
    """Send alert via SAP Alert Notification Service."""

    alert = {
        "eventType": "simula.quality.failure",
        "severity": "WARNING",
        "category": "NOTIFICATION",
        "subject": f"Simula quality gate failure: {run_id}",
        "body": format_failure_report(failures),
        "resource": {
            "resourceName": "simula-generator",
            "resourceType": "application"
        },
        "tags": {
            "run_id": run_id,
            "failure_count": str(len(failures))
        }
    }

    response = requests.post(
        f"{ans_endpoint}/cf/producer/v1/resource-events",
        json=alert,
        headers={"Content-Type": "application/json"}
    )
    response.raise_for_status()
```

B.4 SAP Analytics Cloud Integration

B.4.1 Cost Tracking Dashboard

Export generation metrics to SAP Analytics Cloud:

Listing B.9: Exporting metrics to Analytics Cloud

```
def export_metrics_to_sac(
    sac_client,
    run_metrics: dict,
    cost_data: dict
):
    """Export run metrics to SAP Analytics Cloud model."""

    data_row = {
        "run_id": run_metrics["run_id"],
        "date": run_metrics["timestamp"][:10],
        "examples_generated": run_metrics["total_examples"],
        "llm_calls": cost_data["total_llm_calls"],
        "llm_tokens_input": cost_data["tokens"]["input"],
        "llm_tokens_output": cost_data["tokens"]["output"],
        "cost_usd": cost_data["estimated_cost_usd"],
        "cost_per_example": cost_data["estimated_cost_usd"] /
    run_metrics["total_examples"],
        "coverage_score": run_metrics["coverage"]["global"],
        "diversity_score": run_metrics["diversity"]["global"],
```

```
        "critic_acceptance_rate": run_metrics["critic"]["acceptance_rate"]
    }

    sac_client.import_data(
        model_id="simula_generation_metrics",
        data=[data_row]
    )
```

B.4.2 Recommended Dashboard Widgets

Table B.3: SAC dashboard widget recommendations

Widget	Type	Metrics
Generation Volume	Time Series	Examples/day, cumulative total
Cost Analysis	Stacked Bar	Cost by component (LLM, compute, storage)
Quality Trends	Line Chart	Coverage, diversity, critic rate over time
Use Case Distribution	Pie Chart	Examples by taxonomy factor
Cost Efficiency	KPI Card	Cost per example, trend arrow
Quality Gate Status	Traffic Light	Current run pass/fail status

B.5 SAP GRC Integration

B.5.1 Policy Management

Register synthetic data policies in SAP GRC:

Listing B.10: GRC policy registration

```yaml
# grc-policy-registration.yaml
policy:
  id: "SYNTH-DATA-001"
  name: "Synthetic Training Data Generation Policy"
  category: "Data Governance"
  owner: "AI Platform Team"

  controls:
    - id: "CTRL-001"
      name: "Quality Gate Verification"
      description: "All generated datasets must pass quality gates"
      test_procedure: "Verify coverage >= 80%, diversity >= 65%"
      frequency: "Per generation run"

    - id: "CTRL-002"
      name: "Provenance Logging"
      description: "All generation runs must have complete provenance"
      test_procedure: "Verify audit trail completeness"
      frequency: "Per generation run"

    - id: "CTRL-003"
      name: "Privacy Screening"
      description: "Generated data must pass privacy checks"
```

```
      test_procedure: "Run membership inference test"
      frequency: "Before production use"
```

B.5.2 Compliance Evidence Collection

Automatically collect compliance evidence:

Listing B.11: GRC evidence collection

```
def collect_compliance_evidence(run_id: str, metrics: dict) -> dict:
    """Collect compliance evidence for GRC reporting."""

    return {
        "run_id": run_id,
        "timestamp": datetime.utcnow().isoformat(),
        "controls": {
            "CTRL-001": {
                "status": "PASS" if metrics["coverage"]["global"] >= 0.80
                          and metrics["diversity"]["global"] >= 0.65 else
    "FAIL",
                "evidence": {
                    "coverage": metrics["coverage"]["global"],
                    "diversity": metrics["diversity"]["global"]
                }
            },
            "CTRL-002": {
                "status": "PASS" if metrics.get("provenance_complete")
    else "FAIL",
                "evidence": {
                    "audit_events": metrics.get("audit_event_count", 0),
                    "provenance_hash": metrics.get("provenance_hash")
                }
            }
        }
    }
```

B.6 SAP Build Apps Integration

For self-service generation interfaces, Simula exposes a REST API compatible with SAP Build Apps:

B.6.1 API Endpoints

Table B.4: Simula REST API endpoints for Build Apps

Endpoint	Method	Purpose
`/api/v1/schemas`	GET	List available schemas
`/api/v1/taxonomies`	GET, POST	List/create taxonomies
`/api/v1/runs`	GET, POST	List/create generation runs
`/api/v1/runs/{id}`	GET	Get run status and results
`/api/v1/runs/{id}/cancel`	POST	Cancel running generation
`/api/v1/artifacts/{id}`	GET	Download generated artifacts
`/api/v1/metrics`	GET	Get aggregated metrics

B.6.2 Build Apps Data Entity

Listing B.12: Build Apps data entity for generation runs

```
{
  "name": "SimulaRun",
  "fields": [
    {"name": "id", "type": "text", "key": true},
    {"name": "name", "type": "text"},
    {"name": "status", "type": "enum", "values": ["pending", "running",
   "completed", "failed"]},
    {"name": "target_count", "type": "number"},
    {"name": "generated_count", "type": "number"},
    {"name": "coverage_score", "type": "number"},
    {"name": "diversity_score", "type": "number"},
    {"name": "created_at", "type": "datetime"},
    {"name": "completed_at", "type": "datetime"},
    {"name": "error_message", "type": "text"}
  ],
  "dataSource": {
    "type": "REST",
    "baseUrl": "https://simula-api.your-domain.com",
    "authentication": "OAuth2",
    "endpoints": {
      "getAll": "/api/v1/runs",
      "get": "/api/v1/runs/{id}",
      "create": "/api/v1/runs"
    }
  }
}
```

B.7 Quick Reference: Environment Variables

Table B.5: Complete environment variable reference

Variable	Default	Description
HANA Cloud Connection		
SIMULA_HANA_HOST	(required)	HANA Cloud host
SIMULA_HANA_PORT	443	HANA Cloud port
SIMULA_HANA_USER	(required)	Service user
SIMULA_HANA_PASSWORD_SECRET	(required)	Credential Store reference
SIMULA_HANA_SCHEMA	(required)	Default schema
LLM Configuration		
SIMULA_VLLM_ENDPOINT	(required)	vLLM TurboQuant endpoint
SIMULA_TEACHER_MODEL	deepseek-coder-33b	Teacher model ID
SIMULA_STUDENT_MODEL	codellama-7b	Student model ID
SIMULA_LLM_TIMEOUT	60	Request timeout (seconds)
Taxonomy Configuration		
SIMULA_TAXONOMY_DEPTH	3	Default taxonomy depth
SIMULA_BEST_OF_N	5	Proposals per node
SIMULA_TAXONOMY_TEMPERATURE	0.8	Expansion temperature
Generation Configuration		
SIMULA_TARGET_EXAMPLES	10000	Default target count
SIMULA_COMPLEXIFICATION_RATIO	0.5	Complexification fraction
SIMULA_SAMPLING_STRATEGY	uniform	Node sampling strategy
SIMULA_SCHEMA_MAX_TOKENS	4096	Schema context budget
SIMULA_MAX_COLUMNS_PER_TABLE	30	Column limit per table
Quality Thresholds		
SIMULA_COVERAGE_THRESHOLD_L1	1.0	Level 1 coverage
SIMULA_COVERAGE_THRESHOLD_L2	0.95	Level 2 coverage
SIMULA_COVERAGE_THRESHOLD_L3	0.80	Level 3 coverage
SIMULA_DIVERSITY_THRESHOLD	0.65	Global diversity
SIMULA_CRITIC_ACCEPT_MIN	0.80	Min P(accept|correct)
SIMULA_CRITIC_REJECT_MAX	0.30	Max P(accept|corrupted)
SAP Integration		
SIMULA_AI_CORE_URL	—	AI Core endpoint
SIMULA_AUDIT_LOG_URL	—	Audit Log Service
SIMULA_ANS_URL	—	Alert Notification Service
SIMULA_SAC_MODEL_ID	—	Analytics Cloud model

Appendix C

References

C.1 Source Documents

Table C.1: Source Documents

Document	Path
Simula Research Paper	`docs/6171_Reasoning_Driven_Syntheti.pdf`
Trial Balance Spec	`docs/technical-reports/04-trial-balance-ai-specification.tex`
Arabic AP Spec	`docs/arabic/structured/` (external)

C.2 Implementation Files

Table C.2: Implementation Files

File	Lines
`src/training/pipeline/simula_config.py`	Configuration dataclasses
`src/training/pipeline/simula_llm_client.py`	LLM client with Best-of-N, double-critic
`src/training/pipeline/hana_schema_extractor.py`	HANA MCP extraction
`src/training/pipeline/simula_taxonomy_builder.py`	Algorithm 1 implementation
`src/training/pipeline/simula_data_generator.py`	Algorithm 2 implementation
`src/training/pipeline/simula_complexity_calibrator.py`	Algorithm 3 (Elo scoring)
`src/training/pipeline/simula_coverage_evaluator.py`	Level Ratio Coverage
`src/training/pipeline/simula_diversity_analyzer.py`	Global + local diversity
`src/training/pipeline/simula_critic_validator.py`	Critic calibration
`src/training/scripts/run_hana_pipeline.py`	CLI entry point
`src/training/scripts/split_by_complexity.py`	Complexity split (Figure 7)

C.3 Tools and Dependencies

Table C.3: Tools and Dependencies

Tool	Version	Purpose
vLLM TurboQuant	≥0.8.x	LLM inference backend
AI Core PAL	1.x	HANA MCP server
Python	3.11+	Runtime
asyncio	stdlib	Async orchestration
httpx	0.27+	HTTP client

C.4 Academic References

1. Elo, A. E., & Sloan, S. (1978). *The Rating of Chess Players, Past and Present.* Arco.

2. Yu, P., et al. (2023). "Large Language Models as Simulated Economic Agents." *arXiv preprint arXiv:2305.09857.*

3. Liu, Y., et al. (2024). "Simula: Reasoning-Driven Synthetic Data Generation for Domain-Specific Knowledge Distillation." *OpenReview NALsdGEPhB*.

4. Brown, T., et al. (2020). "Language Models are Few-Shot Learners." *Advances in Neural Information Processing Systems*, 33.

5. Wei, J., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS 2022*.

C.5 Related Specifications

Arabic AP Specification
`docs/latex/specs/arabic/arabic-ap-spec.tex`

Trial Balance Specification
`docs/latex/specs/tb/tb-review-spec.tex`

Regulations Framework Specification
`docs/latex/specs/regulations/regulations-spec.tex`

Agent Rule Pack Specification
`docs/latex/specs/clinerules-agents/clinerules-agents-spec.tex`

C.6 Schema Registry

All schemas referenced in this specification are registered in the unified schema registry:

Registry Index `docs/schema/registry.json`

Simula Schemas `docs/schema/simula/`

- `training-example.schema.json`
- `taxonomy.schema.json`
- `hana-schema-entry.schema.json`
- `config.schema.json`
- `meta-prompt.schema.json`

Common Types `docs/schema/common/base-types.schema.json`

Shared Enums `docs/schema/common/enums.yaml`

C.7 External Standards

JSON Schema Draft-07 (`https://json-schema.org/draft-07/schema`)

ODPS 4.1 SAP ODPS Data Product Specification v4.1

MAS MGF Monetary Authority of Singapore Model Governance Framework for AI

AI Verify 2.0 Singapore AI governance testing framework

Moonshot CI/CD 1.1.0 Red-teaming and benchmark toolkit

Appendix D

Global Wave-Dependency Manifest

D.1 Critical Path Visualization (Waves 1–3)

The following chart defines the normative sequencing for implementation agents. A task in a later wave cannot be marked COMPLETE until its prerequisites in an earlier wave are validated.

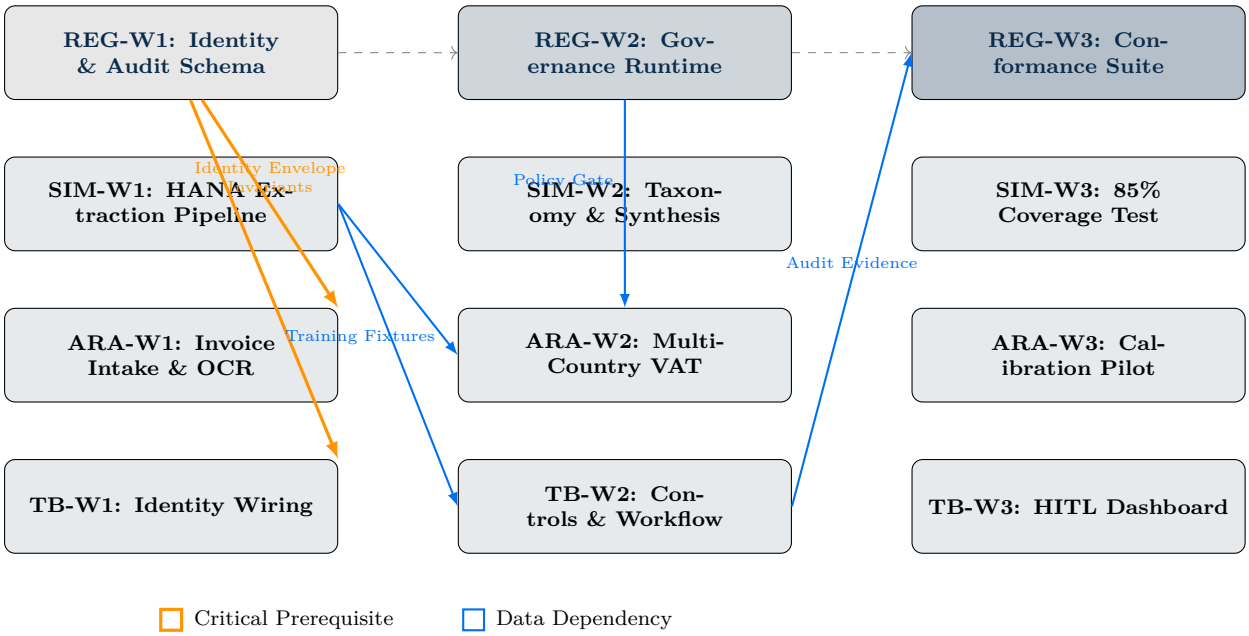

Figure D.1: Cross-specification dependency graph: The blueprint for platform rollout.

Table D.1: Normative Cross-Domain Prerequisites.

Target Task	Prerequisite	Ref ID	Rationale
ARA-W1	REG-W1 (Identity)	`REG-IDENTITY-01`	Invoice intake must be audit-traced from Day 1.
TB-W2	SIM-W1 (Extraction)	`SIM-EXTRACT-01`	Anomaly detection requires HANA schema metadata.
REG-W2	SIM-W1 (Registry)	`SIM-REGISTRY-01`	Governance monitoring relies on the schema registry.
ARA-W2	REG-W2 (Pillar 1)	`REG-MGF-P1`	VAT release is an HITL gate governed by MAS Pillar 1.
TB-W3	ARA-W2 (Posted Invoices)	`ARA-AP-POST`	Review dashboard depends on posted AP data from Arabic domain.

D.2 Inter-Domain Prerequisite Matrix

Dependency Enforcement

Implementation agents are strictly forbidden from "Fast-Forwarding" into Wave 2 if Wave 1 critical prerequisites (marked in **Gold**) are in a GAP or PARTIAL state.